The Zcash Arboretum

Zcash for Bitcoiners

how a shielded transaction is verified without being seen

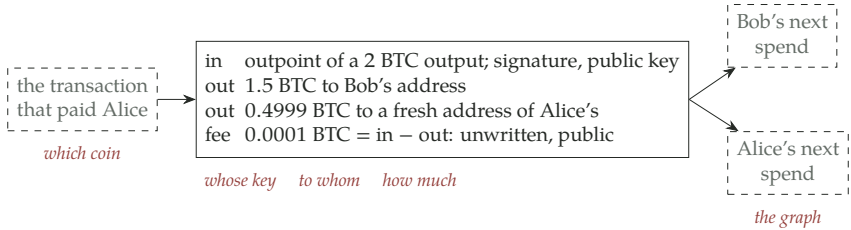
m@rek.onl

NU7 draft: 6 October 2026

Part A

What Bitcoin publishes, and what privacy must mean

Alice pays Bob in Bitcoin: what the chain publishes



- Every output carries its spending condition and its amount; every input names an old output by identifier. Names cost outside information; the graph costs nothing.
- Reuse an address and its history merges; pay with change and the non-round output to a fresh address is usually yours. Both follow from the record, not from a careless wallet.
- The same shape — one 2-coin input, 1.5 to Bob, change back — returns as the shielded running example, with none of the labels.

Ironwood Guide, "Requirements on a shielded payment"

The validator's four checks

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]
    check sig(input.sig, coin.script)
    delete utxo[input.prev_out]
  check sum(in) >= sum(out)
```

(1) coin exists	reads the outpoint
(2) authorised	reads script and key
(3) not spent twice	reads the outpoint
(4) conserved	reads every amount

- The whole state is one table, the set of unspent outputs; a spend is a lookup, a signature check, and a deletion.
- Every check reads a plaintext field of the transaction or of the output it names. Nothing is inferred; everything is looked at.
- The fee is the fourth check's slack, $\Sigma\,\text{in} - \Sigma\,\text{out}$: not a field, yet public to the satoshi.

Ironwood Guide, "Requirements on a shielded payment"; "The note commitment tree"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

– Hide the amount: check 4 dies — nothing to add up.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

- Hide the amount: check 4 dies — nothing to add up.
- Hide the input reference: checks 1 and 3 die — nothing to look up, nothing to delete.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

- Hide the amount: check 4 dies — nothing to add up.
- Hide the input reference: checks 1 and 3 die — nothing to look up, nothing to delete.
- Hide the key: check 2 dies — nothing to verify against.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

- Hide the amount: check 4 dies — nothing to add up.
- Hide the input reference: checks 1 and 3 die — nothing to look up, nothing to delete.
- Hide the key: check 2 dies — nothing to verify against.
- Hide the receiver: nothing dies here — the output's script is what check 2 reads when Bob spends it, so that spend's check 2 dies.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

Hide a field, lose a check

```
validate(tx):
  for input in tx.inputs:
    coin = utxo[input.prev_out]          (1) coin exists
    check sig(input.sig, coin.script)    (2) authorised
    delete utxo[input.prev_out]          (3) not spent twice
  check sum(in) >= sum(out)              (4) conserved
```

- Hide the amount: check 4 dies — nothing to add up.
- Hide the input reference: checks 1 and 3 die — nothing to look up, nothing to delete.
- Hide the key: check 2 dies — nothing to verify against.
- Hide the receiver: nothing dies here — the output's script is what check 2 reads when Bob spends it, so that spend's check 2 dies.

Privacy, for this workshop: hide the sender, the receiver and the amount, while a stranger still performs all four checks.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"

What a privacy design must supply

the validator must	the tool	Bitcoin's version
accept a claim about data it cannot see	a zero-knowledge proof: the statement public, the witness (the prover's secret input, not Bitcoin's witness field) private	look at the data
know the hidden data cannot change under it	a commitment: hiding, and binding	the output, in the clear
notice the same hidden coin a second time	a nullifier: a keyed, one-way tag of the coin, published at spend	delete it from the set
add up amounts it cannot read	a homomorphic commitment: sealed envelopes that add	add the numbers

The proof checks the claim; the other three tools make the claim about something fixed, unrepeatable, summable.

Crypto Guide, "Soundness, zero knowledge, and succinctness"; "Commit and reveal"; *Ironwood Guide*, "Nullifiers"

Sealed envelopes that add

The fourth tool in one line: two commitments add without either being opened.

$$([v_1]\,V + [r_1]\,R) + ([v_2]\,V + [r_2]\,R) = [v_1 + v_2]\,V + [r_1 + r_2]\,R$$

- This is the Confidential Transactions trick: amounts hidden, their sum still checkable.
- It holds only modulo the group order; here the 64-bit range check that closes the gap lives inside the proof.

Crypto Guide, "The Pedersen commitment"; *Ironwood Guide*, "Value commitments and the binding signature"; "The Action statement"

Notes versus UTXOs: what a coin is

	Bitcoin UTXO	Zcash note
owner	a script, usually one key hash, on chain	an address $(d, \mathsf{pk_d})$, inside the note
value	the amount, on chain	$v \in [0, 2^{64})$ zatoshi, inside the note
spending condition	script: signatures, timelocks, hashlocks	a key, full stop — no script, no timelocks, no HTLCs

- The owner and the value are fields of the note, and the note is never published; Bitcoin writes both on the output.
- Threshold authority exists (FROST, later) and is invisible on chain: one ordinary signature under one per-spend key.

Ironwood Guide, "Notes"; "Randomised validating keys"; *FROST Guide*, "Spend authorisation under randomised validating keys"

Notes versus UTXOs: what the chain records

	Bitcoin UTXO	Zcash note
on-chain identifier	the outpoint	a commitment `cm` to the note, stored as `cmx`
spent marker	removed from the UTXO set	a nullifier `nf`, published at spend
set structure	one set: insert, delete	an append-only tree of `cmx`, depth 32, and a set of `nf`: append only, both

The note never touches the chain: recorded are a commitment to it, an encrypted copy for its recipient, and at spend a tag derived from it.

Ironwood Guide, "Notes and note commitments"; "The note commitment tree"; "Nullifiers"

The three-line mental model

A shielded spend is three claims about a note nobody sees:

1. *it exists* — a commitment to it is a leaf under a public root, proved with leaf, position and path all private;
2. *it is spent now* — its serial number, the nullifier, is revealed, and the chain refuses to see it twice;
3. *the books balance* — commitments to the amounts add up to the declared net flow.

$$R_{\text{mem}} = \{\, (\underbrace{rt}_{\text{public}} ; \underbrace{(\mathsf{cmx},\ i,\ path)}_{\text{private}}) \ :\ \mathsf{Verify}(rt, i, \mathsf{cmx}, path) = \mathsf{accept} \,\}$$

A Bitcoin spend names its coin and signs. A shielded spend names nothing: it proves the first line, reveals the second, and signs for the third — and, as in Bitcoin, the spender signs, under a key the proof ties to the note (check 2). Everything else in the workshop is machinery for these three and for that signature.

Crypto Guide, "Privacy-preserving membership via zero-knowledge paths"; "A shielded spend, end to end"; *Ironwood Guide*, "Value commitments and the binding signature"

Four checks, four mechanisms

	the check	Bitcoin	shielded Zcash
(1)	coin exists	look the outpoint up in the UTXO set	a commitment in an append-only tree; a membership proof under a public root
(3)	not spent twice	delete it from the set	a nullifier, published once, refused the second time
(4)	conserved	add the amounts up	value commitments that add; a binding signature over their sum
(2)	authorised	run the script over the signature	key knowledge proved inside the proof; a signature under a re-randomised key

Each right-hand cell names what replaces one Bitcoin check; why the Bitcoin way is gone was settled on *Hide a field, lose a check*. Each part of the workshop reopens with both questions.

Ironwood Guide, "Requirements on a shielded payment"; "The Action statement"

The route

1. **The toolbox** — each primitive from the Bitcoin instance the room already holds.
2. **The data model** — notes, commitments, the tree, the nullifier: checks 1 and 3.
3. **Keys and addresses** — who can spend, who can see.
4. **The Action statement** — what the proof proves, clause by clause; then a break.
5. **Balance and authority** — checks 4 and 2, on the running example.
6. **The proof system** — from a statement to a few kilobytes.
7. **Delivery and light clients** — how Bob finds his money.
8. **The transaction and the turnstile** — the bytes on the chain, and the pools.
9. **Consensus** — why one fork wins, what the chain must remember.
10. **Limits, frontier, takeaways.**

One example threads it: Alice pays Bob 1.5 ZEC from a 2 ZEC note.

Checkpoint

Which of the four checks does a nullifier serve, and what breaks if two different notes could share one?

Checkpoint

Which of the four checks does a nullifier serve, and what breaks if two different notes could share one?

- Check 3, not spent twice. The nullifier is "delete from the UTXO set" without naming the coin: published once, refused the second time.
- Two notes, one nullifier: whichever is spent first burns the other, and the second owner's honest spend is refused as a double spend — an attack on earlier designs, in which forged notes could be made to share one.
- How the chain makes sure no two accepted notes can share one — every new note inherits, as its seed, the serial of the note spent beside it — is Part C's business.

Ironwood Guide, "Nullifier uniqueness and unlinkability"

Part B

The toolbox, from Bitcoin’s primitives up

Hash functions: one primitive, three notions

A fixed public function $H\colon \{0,1\}^* \to \{0,1\}^n$ that compresses.

- *Preimage resistance*: given y, no efficient adversary finds any x' with $H(x') = y$.
- *Second-preimage resistance*: given x, none finds $x' \neq x$ colliding with it.
- *Collision resistance*: none finds any pair $x \neq x'$ with $H(x) = H(x')$. The adversary picks both, so this is the strongest demand — and the one commitments, Merkle trees and Fiat–Shamir need.
- Generic cost: a collision after about $2^{n/2}$ evaluations (the birthday bound), a preimage after 2^n. Hence 256-bit digests for 128-bit collision security; RIPEMD-160 sits at 2^{80}.

- Bitcoin: SHA-256d over block headers and every txid; $\mathsf{hash160} = \mathrm{RIPEMD160} \circ \mathrm{SHA256}$ inside addresses.
- Zcash keeps every one of those in its transparent layer, and adds three hashes for the shielded one.

Crypto Guide, "Security notions: preimage, second-preimage, and collision resistance"; "The birthday bound on generic collision finding"; "The transparent layer's hashes"

Zcash's hashes: SHA-256 is not replaced

hash	kind	deployed use
SHA-256d	bit-oriented, Bitcoin's	header difficulty filter (after Equihash); transparent layer: legacy txids, Base58Check
BLAKE2b	bit-oriented; personalised natively, the key prepended to the input	PRFexpand (the key tree); note-encryption KDF; ZIP-244 digests (txid, sighash)
Poseidon	algebraic: $x \mapsto x^5$ over $\mathbb{F}_{p_{\mathsf{Pallas}}}$	the nullifier PRF, inside the circuit
Sinsemilla	algebraic: curve additions on Pallas	note commitment; Merkle node hash — both inside the circuit

Bitcoin's hashes stay where Bitcoin put them; the shielded protocol adds one bit-oriented hash for work off the circuit, two algebraic ones for work inside it.

Ahead: Poseidon and Sinsemilla in Part C; BLAKE2b in Parts D, F and H.

Consensus Guide, "Equihash and the memoryless model"; *Crypto Guide*, "The transparent layer's hashes"; "PRFs from hash functions: length extension and keyed BLAKE2"; "Arithmetisation-friendly hashing: Poseidon"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"

Why in-circuit hashing wants algebraic hashes

- A zero-knowledge proof re-expresses every step of a computation as polynomial constraints over a prime field; a field multiplication is one constraint, a Boolean operation is several plus range checks.
- SHA-256 and BLAKE2 shuffle and XOR 32- and 64-bit words: foreign to the field, tens of thousands of constraints per evaluation.
- Poseidon: the round is add constants, raise the state to x^5 (every element in a full round, one element in a partial round), multiply by a fixed matrix; the only non-linear step is x^5. A 2-to-1 hash costs a few hundred constraints. Orchard's instance: state width 3, 8 full and 56 partial rounds.
- Sinsemilla: per 10-bit chunk, one lookup into a table of $2^{10} = 1024$ hashed-to-curve points and two incomplete curve additions — native operations, and collision resistance that reduces to the discrete logarithm.
- Bitcoin never hashes inside a proof; the first shielded generation did, with SHA-256, and its prover paid for it.

Crypto Guide, "Arithmetisation-friendly hashing: Poseidon"; "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Cost inside a proof"; protocol specification, "JoinSplit Statement (Sprout)"

Curves: the same shape, different primes

Short Weierstrass $y^2 = x^3 + b$ over a prime field; the points form a group of prime order, cofactor 1; the discrete logarithm in it is the standing assumption.

- Bitcoin: secp256k1, $b = 7$ over $p = 2^{256} - 2^{32} - 977$, a 256-bit prime order. The audience's keys, ECDSA and BIP-340 live here.
- Zcash: Pallas and Vesta, $b = 5$ over two 255-bit primes p_{Pallas}, p_{Vesta}, with $\#\mathsf{Pallas} = p_{\mathsf{Vesta}}$ and $\#\mathsf{Vesta} = p_{\mathsf{Pallas}}$: the scalar field of each is the base field of the other. About 126 bits of discrete-log security.
- Every Pallas and Vesta generator the protocol uses is hashed to the curve (secp256k1's base point is a SEC 2 constant): no relation between any two is known to anyone, and no ceremony for this stack (the Sapling pool still rests on its Groth16 MPC, Part G).
- Pallas carries every point in an Action — commitments, nullifier base, keys, ephemeral keys, signatures; Vesta carries the proof's polynomial commitments.

Ahead: the division of labour between the two curves in Part G.

Crypto Guide, "Instantiation on elliptic curves; the Pasta curves"; "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; *Math Guide*, "Pallas and Vesta assembled"

Why not secp256k1: two-adicity

A proof system of the PLONK family interpolates each circuit column over the 2^k-th roots of unity of its field. Those exist iff $2^k \mid p - 1$; the largest such k is the field's *two-adicity*.

field	bits	two-adicity of $p - 1$	largest power-of-two domain
Pallas base field	255	32	2^{32}
Vesta base field	255	32	2^{32}
secp256k1 base field	256	1	2
secp256k1 scalar field	256	6	64

- The Orchard circuit has 2^{11} rows; neither secp256k1 field offers a domain of that size. The Pasta primes were searched for with two-adicity 32 and the cycle as design goals.
- The secp256k1 rows are the deck's own computation, not a volume figure.

Ahead: rows, roots of unity and the polynomial identity in Part G.

Math Guide, "Roots of unity and evaluation domains", Remark "Two-adicity and FFT-friendly primes"; "Base fields, scalar fields, and the Pasta cycle"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Commitments: a sealed envelope

A commitment $c = \mathsf{Commit}(m; r)$ with a random trapdoor r: *hiding* — from c, two candidate messages are indistinguishable; *binding* — no c opens to two messages. The two cannot both be perfect; the deployed schemes are perfectly hiding, binding under the discrete logarithm. The Pedersen commitment on Pallas, with V, R hashed to the curve:

$$\mathsf{Commit}(v; r) := [v]\,V + [r]\,R.$$

For uniform r the point is uniform whatever v is; two openings of one c solve $\log_V R$, which nobody holds.

- Bitcoin: this envelope is Confidential Transactions'; an address is a hash commitment to a public key or a script, opened at spend.
- Zcash: cv seals an Action's value change (Pedersen, so it adds); cm seals a whole note — an algebraic hash plus a blinding term $[\mathsf{rcm}]\,H_D$ of the same Pedersen shape, over its own hashed generator, so the circuit recomputes it cheaply.

Crypto Guide, "Hiding and binding"; "The Pedersen commitment"; "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Sinsemilla: hash, commitment, and short forms"

The envelope adds: the Confidential Transactions bridge

$$\mathsf{Commit}(v_1; r_1) + \mathsf{Commit}(v_2; r_2) = \mathsf{Commit}(v_1 + v_2;\ r_1 + r_2),$$

so, for a set of inputs and outputs,

$$\sum \mathsf{Commit}_{\text{in}} - \sum \mathsf{Commit}_{\text{out}} = \Big[\sum v_{\text{in}} - \sum v_{\text{out}}\Big] V + [\bar{r}]\, R,$$

and the V term vanishes exactly when the values balance — modulo the group order. A range check on every value closes the wrap.

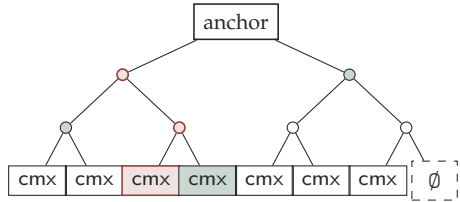
- Confidential Transactions (Elements): the same equation on the chain; a range proof beside each commitment; the receiver is told the opening. Amounts hidden, keys and graph in the clear.
- Here: the 64-bit range check and the opening (v, rcv) are clauses of one zero-knowledge proof, alongside the sender's authorisation and the receiver's note; the net trapdoor $\bar{r}$ becomes a signing key.

Ahead: clause A4 in Part E; the balance walk-through in Part E′.

Crypto Guide, "The Pedersen commitment"; "Binding signatures as a proof of knowledge of a discrete logarithm"

Merkle trees: from the txid tree to the note commitment tree

- Bitcoin: each block hashes its txids pairwise up to a root in the header; an SPV proof is the leaf plus one sibling per level, and the leaf — the txid — is public.
- Zcash: the leaves are cmx in creation order; depth 32, room for 2^{32} notes; the node hash is Sinsemilla, tagged with its layer; the root after each block is that block's *anchor*.
- A membership proof: 32 siblings and 32 direction bits, verified by 32 hashes — recomputed here inside the proof.



red: the leaf and its route, private; green: the siblings, private; the anchor alone is public

Crypto Guide, "Authentication paths and membership proofs"; *Ironwood Guide*, "The Merkle hash and the tree"

What changes: append only, historical roots, private path

- *Append only.* One leaf at a time; the *frontier* — at most 32 carried labels — updates the root in 32 hashes. Nothing is ever deleted; the "deleted" half of the state is a separate set.
- *History binding.* A proof formed against the root at insertion stays valid against that root forever, and consensus accepts the anchor of any earlier main-chain block.
- *Private path.* Revealing (leaf, index, siblings) would say which note is spent; so the path is a witness of the zero-knowledge proof and only the anchor is a statement:

$$R_{\text{mem}} = \{\ (rt;\ (\mathsf{cmx}, i, path)) : \mathsf{Verify}(rt, i, \mathsf{cmx}, path) = \mathsf{accept}\ \}.$$

- Bitcoin contrast: an SPV proof names the txid it proves; this proof names only the root, and hides the leaf among every leaf beneath it.

Ahead: the tree, anchors and the nullifier set in Part C; clause A3 in Part E.

Crypto Guide, "Append-only and incrementally updatable trees"; "Privacy-preserving membership via zero-knowledge paths"; *Ironwood Guide*, "The note commitment tree"

Pseudorandom functions: a key that fakes randomness

A keyed function F_k is *pseudorandom* if, for a secret uniform k, no efficient adversary with oracle access — any inputs, adaptively — tells F_k from a truly random function.

- The algorithm is public; only k is hidden. The adversary never sees k, only input–output pairs of her choosing.
- Bitcoin: HMAC-SHA512 inside BIP-32 child derivation — a PRF the audience already trusts to make child keys look unrelated.
- Zcash, off the circuit: BLAKE2b, personalised, as $\mathsf{PRFexpand}$ — one call per sub-secret, $\mathsf{sk} \to \mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}}$, and later the note's randomness from its seed.
- Zcash, inside the circuit: Poseidon over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, keyed by its first input,

$$F_{\mathsf{nk}}(\rho) = \mathsf{PoseidonHash}(\mathsf{nk}, \rho),$$

  state width 3, S-box x^5, 8 full and 56 partial rounds.

Ahead: $\mathsf{PRFexpand}$ builds the key ladder in Part D.

Crypto Guide, "Pseudorandom functions and permutations"; "PRFs from hash functions: length extension and keyed BLAKE2"; "Arithmetisation-friendly hashing: Poseidon"

What a nullifier needs from a PRF

The nullifier, previewed:

$$\mathsf{nf} = \mathsf{Extract}([\,F_{\mathsf{nk}}(\rho) + \psi\,]\,G^{\mathsf{nf}} + \mathsf{cm}).$$

Extract: the x-coordinate; ρ, ψ: per-note randomness fixed at creation (Part C); G^{nf}: a hashed generator.

Two properties, and they are separate:

- *Uniqueness*: every input $(\mathsf{nk}, \rho, \psi, \mathsf{cm})$ is fixed once the note exists, so the tag is deterministic; two distinct notes sharing a tag yield a discrete-log relation on Pallas. No PRF property is used here — the forger holds nk.
- *Unlinkability*: without nk the term $[F_{\mathsf{nk}}(\rho) + \psi]\,G^{\mathsf{nf}}$ masks cm with a value that looks random — by Poseidon's PRF security or, independently, by DDH on Pallas. Either assumption suffices.
- Bitcoin contrast: the spent marker is the outpoint itself, unique and public; a nullifier must be unique *and* say nothing.

Ahead: the nullifier in full in Part C; clause A5 in Part E.

Ironwood Guide, "Nullifiers"; "Nullifier uniqueness"

Key agreement: the ECDH of silent payments

On a prime-order curve with base point P: a secret a, a public $[a]P$, and $\mathrm{Agree}(a, Q) = [a]Q$, so that $[a]([b]P) = [b]([a]P)$.

- Bitcoin (BIP-352): the sender's input key and the receiver's scan key agree on a secret that tweaks the output key. Nothing is encrypted; the receiver scans for outputs that decode under the shared secret.
- Zcash: the address carries a static key $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ over a per-address base $\mathsf{g_d}$, itself hashed from the diversifier. The sender draws an ephemeral esk (from the note's seed, by $\mathsf{PRFexpand}$) and publishes $\mathsf{epk} = [\mathsf{esk}]\,\mathsf{g_d}$:

$$\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{epk}.$$

- Static–ephemeral: the recipient was never online, and the sender learns nothing beyond the address. Cofactor 1 leaves only the on-curve, non-identity check on epk.

Ahead: note encryption step by step in Part F.

Crypto Guide, "Elliptic-curve Diffie–Hellman"; "Shielded-note delivery as deployed hybrid encryption"; *Ironwood Guide*, "Encryption to the recipient"

Encryption on top: KDF, AEAD, wrong-key rejection

- *KDF*: one personalised hash call, $K_{\text{enc}} = \text{BLAKE2b-256}(\text{“Zcash_OrchardKDF”};\ s \| epk)$; folding epk in binds the key to this ephemeral.
- *AEAD*: ChaCha20-Poly1305 under the 256-bit key, nonce all zero and associated data empty — safe because each key encrypts exactly once — with a 128-bit tag.
- *Wrong-key rejection*: decryption under an independent key returns $\perp$ except with probability about 2^{-128}. This is what lets a wallet try every output on the chain with its own key. The scheme is not key-committing, so the wallet also recomputes cm from the plaintext and compares.
- Bitcoin never encrypts anything on chain: scripts and amounts are plaintext by design. Here the recipient, the value and the memo exist on chain only inside the ciphertext.
- Two ciphertexts per Action: one to the recipient, one to the sender's own outgoing key.

Ahead: trial decryption and compact blocks in Part F.

Crypto Guide, “Key-derivation functions”; “The ChaCha20-Poly1305 AEAD”; *Ironwood Guide*, “Encryption to the recipient”

Signatures: ECDSA, Schnorr, RedPallas

- Bitcoin: ECDSA (r, s) over secp256k1, DER plus a sighash-type byte; under Taproot, BIP-340 Schnorr, $s = k + e\,d$ with $e = H(x(R) \,\|\, x(X) \,\|\, m)$.
- Zcash, transparent: the same ECDSA (sighash as message, DER, low-s). No Taproot, no BIP-340 in the transparent layer.
- Zcash, shielded: *RedPallas* — Schnorr over Pallas with the base point P a parameter of the scheme. Keys $X = [x]P$;

$$\begin{aligned} \text{Sign:}\quad & r = H(T \,\|\, X \,\|\, m) \text{ for 80 fresh random bytes } T,\ R = [r]P, \\ & c = H(R \,\|\, X \,\|\, m), \quad s = r + c\,x; \\ \text{Verify:}\quad & [s]P = R + [c]X, \end{aligned}$$

  with H BLAKE2b-512, personalised, reduced modulo the group order.
- Two instantiations, differing only in P: the spend-authorisation base $G^{\mathsf{SpendAuth}}$, and the value-commitment base R.

Crypto Guide, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; "RedDSA: re-randomisable Schnorr for Orchard"

The need: one spend key per wallet, not per coin

- Bitcoin: one key per output. Reuse is a wallet's choice, and each reuse links two payments.
- Zcash: one spend validating key per wallet, $\mathsf{ak} = [\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$, and every diversified address shares it:

$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk_d} = [\mathsf{ivk}]\, \mathsf{g_d}.$$

- An un-randomised spend would publish ak — the same key for every note the wallet ever spends. Linkage worse than address reuse, and structural rather than chosen.
- Hence a fresh disguise per spend. The public key must be shiftable without the secret; a Schnorr key is the homomorphic image $[x]P$ of its secret, so it can be.

Ahead: the ladder that makes every address share ak in Part D.

Crypto Guide, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Viewing keys"; "Diversified addresses"

Key re-randomisation, additive

For a fresh uniform α per spend — added, never multiplied in:

$$\mathsf{rsk} := \mathsf{ask} + \alpha, \qquad \mathsf{rk} := \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}} = [\mathsf{rsk}]\, G^{\mathsf{SpendAuth}}.$$

- *Consistency*: $(\mathsf{rsk}, \mathsf{rk})$ is an ordinary key pair; a signature under rsk verifies under rk.
- *Unlinkability*: for uniform α the key rk is uniform on the group whatever ak is — perfect, no assumption, provided α is fresh and stays secret.
- Signing under rk needs both ask and α. The circuit proves rk is a re-randomisation of the ak bound into the note's address; the node checks the signature under rk.
- Bitcoin bridge: a non-hardened BIP-32 child is pubkey + $[\text{tweak}]\, G$ too — but anyone holding the parent xpub (key, chain code, index) recomputes its tweak; this one is uniform and never published.

Ahead: clauses A6/A7 in Part E; the signature on the wire in Part E′.

Crypto Guide, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Randomised validating keys"

Preview: a signature under a key nobody holds unless the values balance

Sum the Actions' value commitments and fold in the public balance:

$$\mathsf{bvk} = \sum \mathsf{cv} - [\mathsf{valueBalance}]\, V \quad \text{(public)}.$$

- Every group element is some multiple of R; what matters is who can compute the multiplier. If the hidden values balance, the V terms cancel, $\mathsf{bvk} = [\sum \mathsf{rcv}]\, R$, and the author knows $\mathsf{bsk} = \sum \mathsf{rcv}$. If not, writing bvk as any multiple $[\beta]\, R$ would reveal $\log_R V$, which nobody holds.
- The *binding signature*: RedPallas under bsk, verified under bvk, over the sighash. A Schnorr signature proves knowledge of a discrete logarithm; here knowing it means "the values balance". Range checks in the proof lift balance modulo the group order to balance over the integers.
- Bitcoin's check 4 reads the amounts; here they stay sealed and a signature certifies their sum.

Ahead: the running example's balance, in a toy group, in Part E′.

Crypto Guide, "Binding signatures as a proof of knowledge of a discrete logarithm"

Zero-knowledge proofs, first contact

A relation R of pairs (x, w): the *statement* x is public, the *witness* w private; the claim is "I know w with $(x, w) \in R$".

- *Complete*: an honest prover holding w convinces.
- *Sound*: no prover convinces on a false x — an *argument* if only efficient provers are excluded, the price of succinctness.
- *Zero knowledge*: a simulator without w reproduces the verifier's whole view; nothing beyond x leaks.
- *Knowledge sound*: whoever convinces can be rewound into surrendering w. Needed here: "some spendable note exists in the tree" is true for everyone; "I hold one" is the claim.

Crypto Guide, "Languages, relations, and witnesses"; "Interactive protocols, proofs, and arguments"; "Knowledge soundness and extractors"; "The simulation paradigm and zero knowledge"

A signature on a statement

- Interaction removed by Fiat–Shamir: each challenge becomes a hash of the statement and the transcript so far, so one string carries the whole exchange and anyone can re-derive the challenges.
- Proof of knowledge + Fiat–Shamir = signature. The audience's Schnorr signature is exactly this: a proof of knowledge of x in $X = [x]G$, with the message folded into the challenge hash.
- Bitcoin verifies one equation per signature; this verifier runs one fixed circuit's verifying key against the statement, per proof.

Crypto Guide, "The Fiat–Shamir transform: from interactive to non-interactive"; "SNARKs: succinct non-interactive arguments of knowledge"

A SNARK: succinct, hence an argument

- A SNARK is the same move on a whole statement — "this note is in the tree, this nullifier is its, these values balance" — with the proof size and the verifier's time sublinear in the computation.
- Halo 2, as deployed: the proof is; the verifier's linear part is shared across a batch of proofs (Part G).
- Succinctness forces an *argument*: a proof shorter than the witness cannot pin the witness down against an unbounded prover, so soundness holds against efficient provers, under an assumption.

Ahead: the statement in Part E; the proof system in Part G.

Crypto Guide, "SNARKs: succinct non-interactive arguments of knowledge"; *Ironwood Guide*, "The Action circuit and the Halo 2 proof"

Proof sizes, with units

generation	system	one proof covers	bytes	running example
Sprout	BCTV14	one JoinSplit: 2 in, 2 out	296 B	1 JoinSplit, 296 B
Sapling	Groth16	one Spend or one Output description	192 B each	1 spend + 2 outputs, 576 B
Orchard	Halo 2	the whole bundle, a Actions	$2720 + 2272\,a$ B	$a = 2$ in the Ironwood pool, 7264 B

- Curves: BN254, then BLS12-381, then Pallas/Vesta. Pairing proofs are constant-size; the growth to Halo 2 is the price of no trusted setup. Sprout's row is its original system; it later moved to Groth16.
- Alice's transaction: one proof for both Actions, not two — 4992 bytes plus 2272 for the second.

Ahead: where the bytes go in Part G; the transaction's size in Part H.

Ironwood Guide, "Ironwood component"; *Crypto Guide*, "SNARKs: succinct non-interactive arguments of knowledge"; protocol specification, "JoinSplit Descriptions"; "Constants"

What the proof hides, and what it does not

- The transcript reveals nothing beyond the public inputs of the Action — anchor, cv, nf, rk, cmx, flags. That is the whole of zero knowledge.
- Necessary, not sufficient: the public fields must not link on their own, and the proof cannot help them.
- What keeps them unlinkable, field by field:
  - the commitments cm, cv: hiding — uniform for a uniform trapdoor;
  - the nullifier nf: the keyed PRF — random-looking without nk;
  - the key rk: re-randomisation — uniform for a uniform α;
  - the ephemeral key epk and the ciphertexts: key agreement plus the KDF and AEAD.
- Bitcoin contrast: a Bitcoin signature hides its key's secret and nothing else; the transaction around it is plaintext by design. Here every published field is a disguise, and the proof certifies that the disguises are consistent.

Ironwood Guide, "Privacy of an Action"; *Crypto Guide*, "Hiding and binding"; "Key re-randomisation and unlinkability"

The instantiation table

primitive	Bitcoin	Zcash	in the Action; part
hash	SHA-256d, hash160	BLAKE2b; Poseidon; Sinsemilla	keys, nf, cmx; C, D
curve	secp256k1	Pallas (Action), Vesta (proof)	every point; G
commitment	CT Pedersen; key/script hash	cv Pedersen; cm Sinsemilla	cv, cmx; C, E
Merkle tree	txid tree	commitment tree, depth 32	anchor, A3; C, E
PRF	HMAC-SHA512	PRFexpand; Poseidon	nf, A5; C, D
key agreement	BIP-352 ECDH	ECDH to g_d	epk; F
encryption	none	ChaCha20-Poly1305	ciphertexts; F
signature	ECDSA; BIP-340	ECDSA (transparent); RedPallas	rk, two signatures; E, E′
ZK proof	none	Halo 2 zk-SNARK per bundle	the proof; E, G

Crypto Guide, "The transparent layer's hashes"; "Instantiation on elliptic curves; the Pasta curves"; "Shielded-note delivery as deployed hybrid encryption"; "RedDSA: re-randomisable Schnorr for Orchard"; *Ironwood Guide*, "The note commitment tree"; "Nullifiers"

Checkpoint

Why can't the nullifier simply be the hash of the note commitment?

Checkpoint

Why can't the nullifier simply be the hash of the note commitment?

- The commitment is public: it is the leaf. Anyone could hash every leaf and match the results against published nullifiers — each spend would point at its mint, and the transaction graph would return.
- Uniqueness would survive (distinct commitments, distinct hashes); unlinkability would not. The two properties are separate, and the second needs a secret.
- Deployed: $\mathsf{nf} = \mathsf{Extract}([\,F_{\mathsf{nk}}(\rho) + \psi\,]\,G^{\mathsf{nf}} + \mathsf{cm})$ — keyed by nk, so only its holder can compute it; the discrete-log structure keeps uniqueness provable and defends unlinkability twice over.

Ironwood Guide, "Nullifier uniqueness and unlinkability"

Part C

The data model

A note: the shielded unit of value

The private object: $\mathbf{n} := (d,\ \mathsf{pk_d},\ v,\ \rho,\ \psi,\ \mathsf{rcm})$.

field	job	in Bitcoin
$(d, \mathsf{pk_d})$	the recipient's diversified address: an 88-bit diversifier and a transmission key (Part D)	scriptPubKey
v	the value, $0 \le v < 2^{64}$ zatoshi	amount
ρ	a per-note uniqueness seed, fixed at creation (where it comes from: the nullifier, later in this part)	—
ψ	nullifier salt, derived from the note seed rseed	—
rcm	commitment trapdoor, derived from rseed; for an Ironwood note, last and from every other field (Part F)	—

Bitcoin publishes every field of an output in clear; every field of a note stays with its owner.

Ironwood Guide, "Notes"; "The note seed: Ironwood derivations (lead byte 0x03)"

A note: content and machinery

- Content: the address and the value. Machinery: ρ and ψ exist only to make the note's eventual nullifier unique and unpredictable; rcm makes the commitment hiding.
- Alice's note: $v = 2$ ZEC $= 2 \cdot 10^8$ zatoshi, at one of her own addresses, in the Ironwood pool — its plaintext carries lead byte `0x03` (Part F).
- Bitcoin publishes every field of an output; the note itself never touches the chain — what does is next.

Ironwood Guide, "Notes"; "The note seed: Ironwood derivations (lead byte 0x03)"

Vocabulary: one protocol, two pools

- The *Orchard protocol* is the shared cryptography: the curves, Sinsemilla, the Action circuit, notes, commitments, nullifiers, keys, note encryption.
- A *pool* is state: one note commitment tree, one nullifier set, one public balance row (the chain value pool balance).
- The *Ironwood pool* is current: all new value enters here, its note plaintexts carry lead byte `0x03` (Part F), and its tree began empty at activation.
- The *Orchard pool* is the legacy carrier, sealed to spend-only by three rules (next slide).
- One asset, ZEC; one chain; one transaction format carries both pools' components (Part H).
- Bitcoin analogue: an output type on the same chain — P2WPKH beside P2PKH — if Bitcoin also tracked the total value held in each type.
- The running example lives in the Ironwood tree.

Ironwood Guide, "The Orchard protocol and its two pools"; "Orchard pool, Ironwood pool"; "Chain value pool balances"

The seal: three rules

Three consensus rules seal the Orchard pool, binding every transaction regardless of its version:

1. **No issuance.** A coinbase transaction must carry an *empty* Orchard component.
2. **No circulation.** Every Orchard-pool Action has $\mathsf{enableCrossAddress} = 0$: a spend may pay only its own receiver. The rule lives in the Action-circuit verifying key, which consensus selects by branch, so no transaction version bypasses it.
3. **No entry.** Per transaction, $v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$: net value may leave the pool, never enter.

– Motive: legacy notes are unrecoverable under a quantum break (Part I); consensus stops their circulation and drains the pool through a public turnstile (Part H).
– Bitcoin analogue: a soft fork that narrows what an output type may do — still spendable, but only back to its own key or out of the type; no longer created by coinbase, no longer funded from outside (Part E).

Ironwood Guide, "Chain state and pool rules"

Sinsemilla: a hash built from curve additions

$$\mathsf{Acc}_0 := Q(D), \qquad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \mathbin{\dot{+}} P[m_{i+1}]) \mathbin{\dot{+}} \mathsf{Acc}_i, \qquad \mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell,$$
$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\|\text{-M}}(M) + [r]\, H_D$$

for M split into 10-bit chunks $m_1, \ldots, m_\ell$; $\dot{+}$ is incomplete curve addition, the plain formula, valid when its inputs are neither equal nor opposite; the short forms return Extract of the point.

- The generators $Q(D)$, $P[0], \ldots, P[2^{10} - 1]$ and H_D are GroupHash outputs: nothing up any sleeve, no known discrete-log relation among them.
- Collision resistance reduces to discrete log on Pallas — no assumption beyond what the curve already needs.

Ironwood Guide, "Sinsemilla: hash, commitment, and short forms"

Why Sinsemilla: in-circuit cost

- Cost: one table lookup and two additions per 10-bit chunk, native to a circuit with lookups.
- A bit-oriented hash must be re-expressed through field arithmetic and range checks, and dominates a circuit.
- Bitcoin can afford SHA-256: nobody recomputes it inside a proof; here the spender must recompute the commitment inside one.

Ironwood Guide, "Cost inside a proof"; *Crypto Guide*, "Arithmetisation-friendly hashing: Poseidon"

The note commitment: what the chain stores

$$\mathsf{cm} := \mathsf{NoteCommit}_{\mathsf{rcm}}(\mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi)) \in \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}),$$

$$\mathsf{cmx} := \mathsf{Extract}(\mathsf{cm}) \in \mathbb{F}_{p_{\mathsf{Pallas}}}$$

on chain	cmx, one field element (plus a ciphertext, Part F)
private	d, $\mathsf{pk_d}$, v, ρ, ψ, rcm — recipient, amount, seeds, trapdoor

– Scheme: the Sinsemilla commitment under its own domain, in the Pallas group $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$; $\star$ is the 32-byte point encoding; the tree consumes a field element, hence Extract.

Ironwood Guide, "Note commitments"; "Sinsemilla: hash, commitment, and short forms"

The note commitment: hiding, binding, and the CT contrast

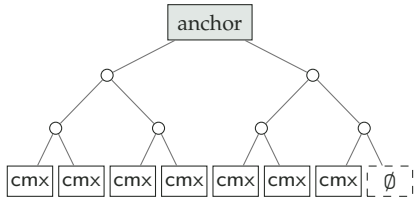
- Hiding: $[\mathsf{rcm}]H_D$ masks the hash — perfectly for a uniform rcm, computationally (PRF) for the seed-derived one. Binding: opening one cm as two notes yields a discrete-log relation.
- Confidential Transactions commit to the amount and publish a range proof beside it; this commitment covers recipient and amount, and is opened only inside the zero-knowledge proof (Part E).

Ironwood Guide, "Hiding and binding of note commitments"; "Sinsemilla: hash, commitment, and short forms"; *Crypto Guide*, "Hiding and binding"

The chain's shielded state: two structures that only grow

	set	operations
Bitcoin	the UTXO set	insert on output, delete on spend
each pool	the note commitment tree	append every cmx; a root per block
	the nullifier set	insert every nf; never delete

- The tree expresses existence of value, as the UTXO set does; stopping a double spend is not its job.
- The nullifier set is the "deleted" half; nothing maps a nullifier back to a leaf. Bitcoin's set forgets a spent coin; these two forget nothing.



public cmx leaves; one tree per pool

Ironwood Guide, "The note commitment tree"; "Nullifier sets"; protocol specification, "Note Commitment Trees"; "Nullifier Sets"

The tree by the numbers

- Depth 32: room for 2^{32} leaves, filled in creation order, one tree per pool for the pool's whole life.
- Nodes are layer-tagged Sinsemilla short hashes: a 10-bit layer tag and two 255-bit children, 520 bits, 52 chunks.
- Empty positions hold the sentinel 2, the x-coordinate of no Pallas point, so no leaf can collide with "empty".
- Bitcoin's txid tree is rebuilt per block with SHA-256d and its depth follows the transaction count; this tree is one, fixed in depth, and only ever extended.

Ironwood Guide, "The Merkle hash and the tree"; protocol specification, "Note Commitment Trees"

Minting: what an output writes to the chain

- An Action's output side appends $\mathsf{cmx}_{\text{new}}$ to the tree and attaches an ephemeral key epk and two ciphertexts: C_{enc} to the recipient, C_{out} to the sender's own outgoing key (Part F). Nothing readable about the new note is written.
- The value is hidden inside cm, bound to the recipient; the Action's net value $v_{\text{old}} - v_{\text{new}}$ is hidden a second time inside its value commitment

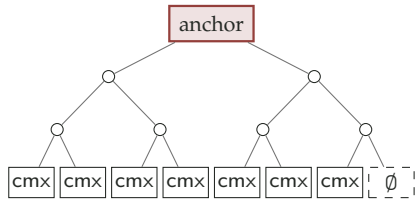
$$\mathsf{cv} := [\, v_{\text{old}} - v_{\text{new}} \,]\, V + [\mathsf{rcv}]\, R,$$

  whose homomorphism lets a validator check conservation without any amount (Part E′).
- Alice's Action 1 — the deck's label; wire order is shuffled (Part E) — mints Bob's note: $v = 1.5$ ZEC $= 1.5 \cdot 10^8$ zatoshi, ρ_{new} := the nullifier of Alice's note. On chain: cmx, cv, and the delivery fields epk, C_{enc}, C_{out} — opaque strings.
- Bitcoin: an output writes amount and scriptPubKey in clear, and the UTXO set indexes them.

Ironwood Guide, "A two-Action payment"; "Note encryption"; "Value commitments"

Resting: the frontier and the anchor

- Appending on the right alters no old root: an anchor names one fixed historical tree, for ever.
- A block's anchor is its tree's final root, one per main-chain block; a transaction's must name an *earlier* block's: no spend of an unmined note, no mempool chains.
- A wallet seeds its tree from the *frontier*: the newest leaf plus one sibling root per filled subtree along its path — enough to append and recompute the root; its own notes' paths it keeps as retained nodes and subtree roots (Part F).
- Consensus accepts the anchor of any main-chain block since the pool's activation: no window.



a Bitcoin input names one outpoint;

this spend names a tree

Ironwood Guide, "The note commitment tree"; "Anchors"; "Authentication paths"; *Wallet Guide*, "Frontiers as scan seeds"; protocol specification, "Action Transfers and their Descriptions"

Which anchor a wallet picks

- Consensus mandates no depth; the choice is wallet policy.
- Deployed default: the newest *checkpoint* at or below $H - 3$, with H the target height. A checkpoint is a tree state the wallet stored at a block boundary, one series per pool; the pick is made per pool and the oldest taken, so checkpoint gaps and that cross-pool minimum can make the anchor older.
- ZIP-315's stated rationale: input availability, and a fixed depth hides whether the spent input was trusted (untrusted notes wait 10 confirmations, trusted ones 3).
- The Guide's own argument: the anchor's tree is the candidate set, so the newest admissible anchor names the largest one, and an idiosyncratically deep anchor would fingerprint the spender.
- An old anchor is no soundness hole: membership in an older, smaller tree proves the note was committed even earlier. Stopping a second spend is the nullifier's job, not the anchor's.
- Bitcoin has no such choice: an input names its outpoint, whatever the tip.

Ironwood Guide, "Anchors"; *Wallet Guide*, "Target and anchor heights"

When the anchor vanishes: reorganisations

- A reorganisation that strikes the anchoring block from the main chain invalidates the anchor; deeper anchors make this rarer, no depth rules it out.
- Automatic rollback depth is node policy, not consensus. ZIP-218 recommends 600 blocks for NU7; a fork below a node's retained rollback window needs recovery outside that window.
- Deployed wallets truncate to a checkpoint below the fork, rescan, and rebuild the transaction. The rebuilt spend reveals the same nullifiers, because each is a fixed function of its note.
- Re-anchoring a signed transaction with its txid unchanged is specified — ZIP-229 makes v6 anchors authorizing data, the proof rather than the signature pins them — with wallet support pending.
- Bitcoin: a reorged transaction stays valid wherever its inputs survive; here the tree it named may no longer exist.

Ironwood Guide, "Reorganisation"; *Wallet Guide*, "Reorganisation detection and recovery"; "PCZT v2: deferred anchors and the Ironwood bundle"

Not a mixer

- The candidate set of Alice's spend is every leaf of the Ironwood tree under her anchor: everyone's notes since the pool's activation, when the tree began empty.
- No participants, no coordinator, no round, nobody to wait for: the other leaves' owners never take part.
- CoinJoin: the anonymity set is one transaction's co-signers, and the graph remains — every output is a child of the whole input set.
- What narrows the set here is not the proof but metadata: notes known to the observer, timing, fees, transparent components (Part H). Tree size alone does not measure it.

Ironwood Guide, "The note commitment tree"; "Anchor choice as public data"; "Orchard pool, Ironwood pool"

The nullifier: a keyed serial number

$$\mathsf{nf} := \mathsf{Extract}\big([\, F_{\mathsf{nk}}(\rho) + \psi \,]\, G^{\mathsf{nf}} + \mathsf{cm} \big) \in \mathbb{F}_{p_{\mathsf{Pallas}}},$$
$$G^{\mathsf{nf}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard}, \texttt{K})$$

- In words: keyed by nk, the owner's nullifier key; seeded by ρ; salted by ψ; bound to cm. The PRF F_{nk} is Poseidon keyed by nk, an algebraic hash the circuit evaluates natively.
- Where ρ comes from: $\rho_{\mathsf{new}} := \mathsf{nf}_{\mathsf{old}}$, the nullifier of the note spent in the same Action — every note is born from a spend and inherits its nullifier.
- A dummy spend — an Action with no real input — seeds ρ from Extract of a random Pallas point, and still publishes a nullifier.
- Bitcoin's spent marker is the outpoint, the coin's own name; this one names nothing.

Ironwood Guide, "Nullifiers"; *Crypto Guide*, "Arithmetisation-friendly hashing: Poseidon"; protocol specification, "Dummy Notes (Orchard)"

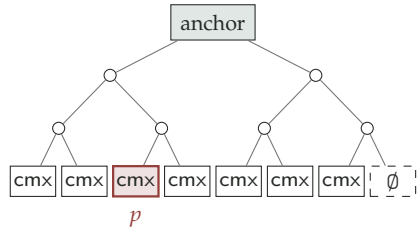
Uniqueness and unlinkability, separately

- The chain's rule on ρ closes the loop: duplicate nullifiers are rejected per pool, so accepted notes' ρ values are unique within the pool — forged notes cannot be made to share a nullifier.
- The leaf index never enters: position is a private witness of the membership check, not an input to `nf`.
- Recall from Part B, now with every input in view: *uniqueness* because each ingredient is fixed once the note exists and two notes sharing one `nf` would yield a discrete-log relation on Pallas; *unlinkability* because without `nk` the masking term looks random — under decisional Diffie–Hellman on Pallas, or independently under Poseidon as a PRF: either suffices, defence in depth.
- Bitcoin's marker is unique because it is the coin's name; this one is unique because two names would break discrete log.

Ironwood Guide, "Nullifier chaining"; "Nullifier uniqueness"

Alice's leaf: the position is a secret

- Alice's 2 ZEC note sits at position p, recorded by her wallet when trial decryption succeeded; p and its 32 siblings are private witness, only the anchor public.
- Her nullifier was fixed at mint — nk, ρ, ψ, cm are all set — and only a holder of nk can compute it.
- At spend she reveals nf; the node checks it is not in the set, then records it. Nothing links nf to the leaf at p.
- The SPV proof you know — a Merkle path to a public root — runs here inside the zero-knowledge proof.



position p and the path stay inside the proof

Ironwood Guide, "Authentication paths"; "Nullifiers"; "Verification of a transaction"

Checkpoint

Alice's restored wallet has found its old notes.
How does it learn which are spent?

Checkpoint

Alice's restored wallet has found its old notes. How does it learn which are spent?

- The chain never marks a leaf spent, so the tree cannot say.
- With `nk` the wallet computes each note's nullifier and discards those already in the pool's nullifier set — compact blocks carry every `nf` for this, and to supply ρ when detecting the note minted beside it (Part F).
- With the incoming keys alone it sees notes arrive but cannot tell which have left: spent status is a capability of `nk` (Part D).
- Bitcoin: a restored wallet asks the UTXO set.

Ironwood Guide, "Nullifier sets"; "Nullifiers"

Part D

Keys and addresses

BIP-32 becomes ZIP-32: one seed, two trees

$$(\mathsf{sk}_i, c_i) := \text{halves of } \mathsf{PRFexpand}_{c_{\text{par}}}(\texttt{0x81} \,\|\, \mathsf{sk}_{\text{par}} \,\|\, i), \qquad i \geq 2^{31}$$

- One seed (32–252 bytes, at least 256 bits of entropy) roots both trees, so one backup recovers every key. Transparent keys keep BIP-44: $m/44'/133'/\mathit{account}'/\mathit{change}/\mathit{index}$, the two lowest levels non-hardened, an xpub at the account.
- Orchard keys stop at $m_{\mathsf{Orchard}}/32'/133'/\mathit{account}'$: hardened only, every level by the formula above, the chain code discarded at the account. The xpub mode — a public key deriving child public keys — is undefined, so no Orchard extended full viewing key exists.
- No change level. BIP-44's change branch separates addresses on a public graph; a shielded change output is indistinguishable on chain from any other output, so a change subtree would separate nothing.

Wallet Guide, "Wallet seeds"; "Hardened-only derivation"; "The standard account path"; ZIP-32, "Key path levels"

Two scopes per account: external and internal

$$\mathsf{r_{ivk}^{int}} := \mathrm{ToScalar}(\mathrm{PRFexpand}_{\mathsf{r_{ivk}}}(\mathtt{0x83} \,\|\, \mathsf{ak} \,\|\, \mathsf{nk})), \qquad \mathsf{fvk^{int}} := (\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}^{int}})$$

- Only $\mathsf{r_{ivk}}$ changes (ToScalar reduces the 512-bit output modulo the Pallas scalar-field order). The same ask spends and the same nk nullifies in both scopes; ivk, dk, ovk are re-derived, so an internal address does not decrypt under the external ivk.
- The internal key $\mathsf{fvk^{int}}$ follows from fvk: a full viewing key sees both scopes; an external $(\mathsf{dk}, \mathsf{ivk})$ — a payment processor — sees the external scope only, never the change: neither the internal scope nor any ovk follows from it.

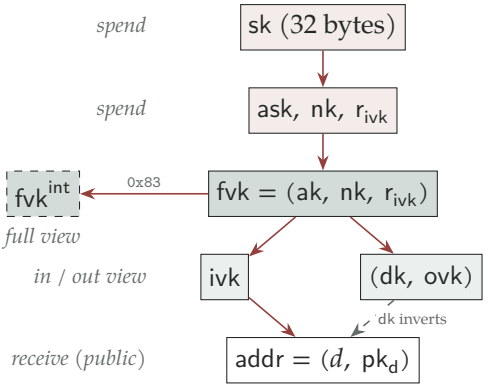
Wallet Guide, "Internal keys: change and auto-shielding"; ZIP-32, "Orchard internal key derivation"; ZIP-316, "Deriving Internal Keys"

The internal scope: change and auto-shielding

- What BIP-44 puts in the path sits here in the key: internal addresses take change and auto-shielded funds. *Shielding* is moving transparent funds into the pool; wallets do it automatically, to an internal address.
- The deployed wallet library (librustzcash) sends change to the internal address at index 0: Alice's 0.4999 ZEC goes there.

Wallet Guide, "Internal keys: change and auto-shielding"; *Ironwood Guide*, "The balancing value"

The ladder: one secret, graded capabilities



$\mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}} := \mathsf{PRFexpand_{sk}}(0x06/0x07/0x08)$

$\mathsf{ak} := [\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$; $\mathsf{nk}, \mathsf{r_{ivk}}$ carried

$\mathsf{ivk} := \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$
$(\mathsf{dk}, \mathsf{ovk}) :=$ halves of $\mathsf{PRFexpand_{r_{ivk}}}(0x82 \,\|\, \mathsf{ak} \,\|\, \mathsf{nk})$

$d := \mathsf{FF1\text{-}AES256_{dk}}(j)$, $\mathsf{g_d} := \mathsf{GroupHash}(d)$,
$\mathsf{pk_d} := [\mathsf{ivk}]\, \mathsf{g_d}$

Red arrows are one-way: a rung is handed down without the powers above it. BIP-32 has two rungs, xprv and xpub, and the public ledger supplies the rest; here every rung between is a separate secret, and $(\mathsf{dk}, \mathsf{ovk})$ come from fvk by a PRF, not from ivk. The circuit re-walks $\mathsf{fvk} \to \mathsf{ivk} \to \mathsf{pk_d}$ (A7) with $\mathsf{g_d}$ as a witness: the diversifier step never enters it.

Ironwood Guide, "Key capabilities and scope"; "Viewing keys"; "The Action statement"; protocol specification, "Orchard Key Components"

What holds the rungs apart

step	operation	why it is one-way
$\mathsf{sk} \to (\mathsf{ask}, \mathsf{nk}, \mathsf{r_{ivk}})$	$\mathsf{PRFexpand_{sk}}(\mathtt{0x06/0x07/0x08})$	PRF (BLAKE2b)
$\mathsf{ask} \to \mathsf{ak}$	$[\mathsf{ask}]\, G^{\mathsf{SpendAuth}}$	discrete log on Pallas
$\mathsf{fvk} \to \mathsf{ivk}$	$\mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$	hiding under $\mathsf{r_{ivk}}$
$\mathsf{fvk} \to (\mathsf{dk}, \mathsf{ovk})$	halves of $\mathsf{PRFexpand_{r_{ivk}}}(\mathtt{0x82} \,\|\|\, \mathsf{ak} \,\|\|\, \mathsf{nk})$	PRF
$\mathsf{fvk} \to \mathsf{fvk}^{\mathsf{int}}$	$\mathsf{PRFexpand_{r_{ivk}}}(\mathtt{0x83} \,\|\|\, \mathsf{ak} \,\|\|\, \mathsf{nk})$	PRF
$\mathsf{ivk} \to \mathsf{pk_d}$	$[\mathsf{ivk}]\, \mathsf{g_d}$	discrete log on Pallas
$j \to d$	$\mathsf{FF1\text{-}AES256_{dk}}(j)$	*not* one-way: dk inverts it

- The scalar ivk is a commitment, not a hash, because it must bind *and* hide (ak, nk): an ivk holder learns neither. Since $\mathsf{r_{ivk}}$ also keys the 0x82 PRF, hiding rests on a joint assumption once dk or ovk is exposed.
- Why a separate nk: spent status can be delegated without spend authority — the viewing key holds nk, never ask.

Ironwood Guide, "Key capabilities and scope"; *Wallet Guide*, "Diversifier indices"; protocol specification, "Orchard Key Components"

Graded capabilities: spend, full view, spent status

capability	holds	can	cannot
spend	ask (from sk)	authorise spends, both scopes	—
full view	fvk = (ak, nk, r_{ivk})	everything below, both scopes; every address and its index	spend: ask is a discrete log away
spent status	nk (inside fvk)	the nullifier of a note it holds	spend; find notes (needs ivk)

– Bitcoin keeps only the spend row secret (xprv); full view and spent status are what its public ledger shows to everyone.

Ironwood Guide, "Capability separation"; *Wallet Guide*, "Orchard key components of an account"

Graded capabilities: incoming, outgoing, receive

capability	holds	can	cannot
incoming view	$(\mathsf{dk}, \mathsf{ivk})$	detect and decrypt incoming; derive all 2^{88} addresses and invert them to indices	nullifiers, so spent status; outgoing; the internal scope
outgoing view	ovk	open C_{out} of what this account and scope sent (a sender may set $\mathsf{ovk} = \perp$: then no ovk opens it)	incoming; the internal ovk
receive	$(d, \mathsf{pk_d})$	be paid	anything else — the only public item

– Every row but the last is a distinct secret; Bitcoin's ledger collapses everything between xprv and the address into what anyone can read.

Ironwood Guide, "Key capabilities and scope"; "The outgoing ciphertext"; *Wallet Guide*, "Raw encodings"; ZIP-316, "Deriving Internal Keys"

Viewing keys: xpub is the bridge, not the analogue

- An xpub is a watch-only wallet: it enumerates the addresses, and the public ledger supplies incoming, outgoing and spent status — to anyone, xpub or not.
- The pair $(\mathsf{dk}, \mathsf{ivk})$ is the watch-only wallet for incoming: it enumerates the addresses *and* decrypts. Three differences:
  1. Bitcoin shows everything regardless; here nothing is visible without the key — the outputs are ciphertext.
  2. Spent status is a second secret: nk computes the nullifiers that the chain's set is matched against.
  3. Outgoing is a third: ovk opens C_{out}. The xpub gets both free only because Bitcoin's ledger is public.
- "Viewing" names a capability, not publicity: every viewing key is secret material disclosed deliberately — to an auditor, a processor — and validation needs none of them.

Ironwood Guide, "Key capabilities and scope"; "Verification of a transaction"; *Wallet Guide*, "Unified viewing keys"

Diversified addresses: one key, 2^{88} addresses

$$d := \mathsf{FF1\text{-}AES256}_{\mathsf{dk}}(j), \qquad \mathsf{g_d} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd},\ d),$$
$$\mathsf{pk_d} := [\mathsf{ivk}]\,\mathsf{g_d}, \qquad \mathsf{addr} := (d,\ \mathsf{pk_d})$$

- Every index j of the $2^{88} \approx 3.1 \times 10^{26}$ works: GroupHash is total (a fixed fallback covers the identity), so no rejection loop. A raw receiver is $11 + 32 = 43$ bytes; all of them are spent by one ask and found by one ivk, at no extra scanning cost.
- A keyed permutation, not a counter and not a hash: consecutive indices scatter over $\{0,1\}^{88}$, never collide (random 88-bit draws would, near 2^{44}), and dk inverts the map, so the wallet stores nothing per address.
- Wallet-side only, never in the circuit — which is why AES is fine here.

Ironwood Guide, "Diversified addresses"; *Wallet Guide*, "Diversifier indices"; "Raw encodings"; *Crypto Guide*, "Format-preserving encryption and FF1"; ZIP-32, "Orchard key path"

Unlinkable, even to the payer

$$(d, \mathsf{pk_d}) \quad \text{and} \quad (d', \mathsf{pk_d}') : \quad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d},\ \ \mathsf{pk_d}' = [\mathsf{ivk}]\,\mathsf{g_d}' \quad \text{— same ivk?}$$

- Deciding it is a DDH problem on Pallas with GroupHash as a random oracle: given two addresses of different owners and a third derived from one of them, no one can tell which. Alice, who paid one of Bob's addresses, learns nothing about the other.
- Only the holder of $(\mathsf{dk}, \mathsf{ivk})$ can check — by re-deriving the address at the decrypted index under that ivk.

Unlinkability: the wallet boundary

- One rule for implementers: never offer a chosen-diversifier oracle (an address for an attacker's d under your ivk): it breaks unlinkability for every address of that ivk.
- Under one xpub, addresses link on the public graph once spent together (Part A); a leaked xpub links all beneath it, spent or not. Here an address reveals only itself.

Wallet Guide, "Diversifier indices"; *Ironwood Guide*, "Diversified addresses"; protocol specification, "$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ and $\mathsf{DiversifyHash}^{\mathsf{Orchard}}$ Hash Functions"

Unified addresses (ZIP-316): one string, several receivers

- One string carries at most one receiver per type — transparent P2PKH or P2SH (`0x00/0x01`), Sapling (`0x02`), Orchard (`0x03`) — with at least one shielded. Items are encoded in ascending typecode, passed through F4Jumble — an unkeyed four-round Feistel permutation of the whole payload, HRP included — then Bech32m: a substituted receiver changes the whole string, not one region of it.
- What a sender does: use the most preferred receiver it supports — Orchard over Sapling over transparent; a wallet may refuse a type, never reorder.

Unified addresses: pools and viewing keys

- Which pool: an Orchard receiver names the protocol, not a pool, and all new value enters the Ironwood pool, so a payment to it lands in the Ironwood tree; the receiver has no string form of its own.
- The same container with viewing-key items is a unified full or incoming viewing key.
- Bitcoin: one address encodes one script type; here one address lets the sender's wallet choose the pool.

Wallet Guide, "Unified addresses"; "String encoding: F4Jumble and Bech32m"; "Unified viewing keys"; *Ironwood Guide*, "Orchard pool, Ironwood pool"; ZIP-316, "Encoding of Unified Addresses"

Bob's address in the example

Bob hands Alice a unified address; her wallet takes its Orchard receiver $\mathsf{addr}_B = (d_B, \mathsf{pk_d})$, 43 bytes, and derives

$$\mathsf{g_d} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash} : \texttt{Orchard-gd},\ d_B), \qquad \mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d},$$
$$\mathbf{n} := (\mathsf{addr}_B,\ 1.5\ \text{ZEC},\ \rho,\ \psi,\ \mathsf{rcm}).$$

- That is all it learns: no ivk, nk, ak, no index j, no link to Bob's other addresses. On chain the address appears only blinded: inside cm, inside C_{enc}, and as epk under a fresh esk.
- Minted in the Ironwood tree, lead byte 0x03; Bob's one ivk trial-decrypts both pools.
- Her change, 0.4999 ZEC, goes to her internal address at index 0: no address of hers appears either.
- Bitcoin displays the script beside the amount; here neither appears in the clear.

Ironwood Guide, "Encryption to the recipient"; "Notes"; "Trial decryption and note acceptance"; *Wallet Guide*, "Unified addresses"; "Internal keys: change and auto-shielding"

Checkpoint

Which key does an accountant need to audit a company's incoming *and* outgoing shielded payments — and what can they not do with it?

Checkpoint

Which key does an accountant need to audit a company's incoming *and* outgoing shielded payments — and what can they not do with it?

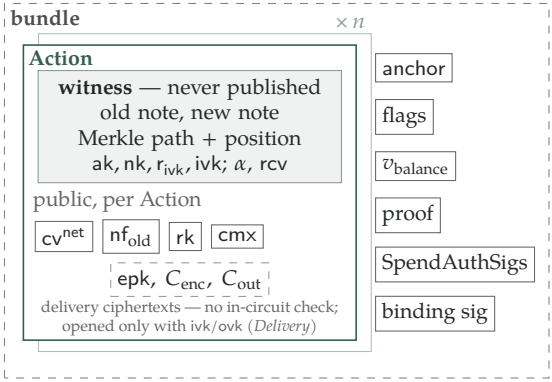
- One unified full viewing key per account. Inside it, fvk yields ivk (incoming), nk (which of those notes are spent) and ovk (outgoing) — for the external scope and, through the 0x83 step, the internal one, so change is covered too.
- Not spend: ask is a discrete logarithm away from ak. Not wander into a sibling account: only hardened derivation exists, and no Orchard extended full viewing key.
- Not read what was sent under some other ovk, or under none: the sender may set ovk $= \perp$, and then no ovk opens C_{out}. Nor anything received by an account they were not given.

Ironwood Guide, "Key capabilities and scope"; "The outgoing ciphertext"; *Wallet Guide*, "Orchard key components of an account"; "Internal keys: change and auto-shielding"; "Unified viewing keys"

Part E

The Action statement

Anatomy of an Action



- One Action spends one old note and creates one new one; the shape is fixed, so an Action never shows which it "really" is.
- Inside: the witness — both notes, the path, the keys, α, rcv — never published.
- On the rim, per Action: cv^{net}, nf_{old}, rk, cmx — the four fields the proof binds.
- Dashed: epk, C_{enc}, C_{out} — delivery ciphertexts, read by no clause; only ivk or ovk opens them (*Delivery and light clients*).

A Bitcoin input names its coin and shows its script; a Bitcoin output shows its amount and its script. Here every field is a commitment, a serial number, a disguised key, or a ciphertext.

Ironwood Guide, "Action description"; "The outgoing ciphertext"; "The Action statement"

The bundle around it

per bundle	once	role
anchor	one root, shared by every Action	the tree the spends prove membership in
flags	enableSpends, enableOutputs, enableCrossAddress	consensus gates (A8–A10)
v_{balance}	signed, cleartext	net value leaving the pool
proof	one aggregate Halo 2 proof	covers all n Actions at once
binding sig	one signature	proves the values balance
per Action	SpendAuthSig under rk	the owner authorised this spend

- Each Action is 820 public bytes, 660 of them ciphertext; Alice's bundle carries two Actions and one 7264-byte proof.
- The node checks the proof, the anchor, the nullifier set, and the two signatures — and never sees a note. Both signatures and v_{balance} return in *Balance and authority*.

Ironwood Guide, "Verification of a transaction"; "Ironwood component"

Why a spend and an output are fused

$$n_{\text{act}}(s, o) = \max(\max(s, o), 2) \qquad \text{Alice: } s = 1,\ o = 2 \ \Rightarrow\ n_{\text{act}} = 2$$

- **Uniformity.** Every Action looks like every other: one spend slot, one output slot, whatever the payment needed.
- **Padding.** The spend list and the output list are each extended to n_{act}; the floor is two. Alice's one spend and two outputs (Bob, change) pad with one dummy spend.
- **Scope.** This count, and the shuffle on the next slide, are the Ironwood component's (cross-address enabled). A sealed Orchard-pool bundle pads to $s + o$ and pairs each spend with an output to the spent note's own receiver.

A Bitcoin transaction shows its true input and output counts; here the count is n_{act}, and the fee is computed from that public count (*The transaction and the turnstile*).

Wallet Guide, "Padding, dummies, and shuffling"; *Ironwood Guide*, "Meaning of A10"

Dummies and the shuffle

- **Dummies.** A slot with nothing real in it holds a *dummy*. A dummy spend is a zero-value note under a fresh random spending key, its ρ the Extract of a random Pallas point; it still publishes a nullifier. A dummy output is a zero-value note to a fresh random address, its $\rho := \mathsf{nf}_{\mathrm{old}}$ like any output's.
- **Shuffling.** Spends and outputs are shuffled *independently*, then zipped: which spend shares an Action with which output is random.
- Alice's bundle: her 2 ZEC note and a dummy on the spend side, Bob's note and her change on the output side; which pair forms which Action is the shuffle's choice.

A Bitcoin input is an input and an output is an output; here a dummy spend looks exactly like a real one and, like a real one, publishes a nullifier the pool records.

Wallet Guide, "Padding, dummies, and shuffling"; *Ironwood Guide*, "Dummy notes"; protocol specification, "Dummy Notes (Orchard)"

The statement, in the validator's language

"Some leaf under this anchor is a note I own; this is its serial number; this commitment hides its value change; this signing key is a fresh disguise of the owner's — and I have shown you neither the leaf, the note, nor the owner's key."

- **Public inputs:** anchor, $\mathsf{cv}^{\mathsf{net}}$, $\mathsf{nf}_{\mathsf{old}}$, rk, cmx and the three flags — eight names, ten field elements (the two Pallas points enter by their coordinates). Here $\mathsf{cv}^{\mathsf{net}}$ and $\mathsf{nf}_{\mathsf{old}}$ are the data model's cv and nf, marked *net* (this Action's value change) and *old* (the note spent).
- **Witness:** both notes, the path and position, ak, nk, $\mathsf{r}_{\mathsf{ivk}}$, ivk, α, rcv.
- **Relation:** ten clauses, A1–A10, over those inputs. The node's verifying key fixes exactly these ten, selected by branch; a proof of nine is a proof for a different circuit, and it fails.

Bitcoin's validator runs *your* script. This validator runs one verifier keyed to one fixed circuit; the transaction supplies inputs, never logic.

Ironwood Guide, "The Action statement"; "The Action circuit and the Halo 2 proof"; protocol specification, "Action Statement (Orchard)"

The ten clauses, ordered by the four checks

Bitcoin's check	clause	what the proof establishes
coin exists (UTXO lookup)	A3	a Merkle path from cm_{old} to the anchor
not spent twice (entry deleted)	A5	nf_{old} is this note's serial, keyed by nk
conserved ($\sum$ in $\geq \sum$ out)	A4	cv^{net} hides $v_{\text{old}} - v_{\text{new}}$, both below 2^{64}
authorised (script + signature)	A6, A7	rk disguises ak; ak, nk sit behind the address
well-formed	A1, A2	both commitments open to notes; $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$
gated	A8, A9	spends and outputs switched on by the flags
sealed	A10	cross-address transfers switched off, or not

Four of Bitcoin's checks become seven clauses; three more are consensus gates the flags drive. The remaining check — "has this serial been seen?" — is global history and stays with the node.

Ironwood Guide, "The Action statement"; "Requirements on a shielded payment"; "Verification of a transaction"

Alice's two Actions against the ten clauses

clause	Action 1: 2 ZEC → Bob 1.5	Action 2: dummy → change 0.4999
A1	Alice's 2 ZEC note opens cm_{old}	a zero-value dummy opens cm_{old}
A2	Bob's note; $\rho_{\text{new}} := \mathsf{nf}$ of Alice's note	change note; $\rho_{\text{new}} := \mathsf{nf}$ of the dummy
A3	a 32-sibling path to the Ironwood anchor	gated off: $v_{\text{old}} = 0$
A4	cv^{net} commits to $+0.5$ ZEC	cv^{net} commits to -0.4999 ZEC
A5	nf of Alice's note, under her nk	nf of the dummy, under its random nk
A6	$\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$, Alice's ak	the same, the dummy's ak
A7	Alice's ak, nk sit behind her address	the dummy's key, behind its address
A8, A9	both flags set: both products vanish	the same
A10	$\delta = 0$ (Ironwood): nothing constrained	the same

The two net values sum to $+0.5 - 0.4999 = +0.0001$ ZEC $= 10\,000$ zatoshi: the fee, and the bundle's v_{balance} (*Balance and authority*).

Ironwood Guide, "The Action statement"; *Wallet Guide*, "Padding, dummies, and shuffling"; "The conventional fee"

What Alice's public fields carry

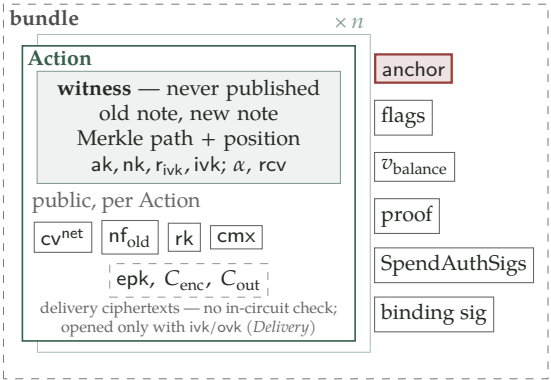
field	Action 1	Action 2
$\mathsf{cv}^{\mathsf{net}}$	+0.5 ZEC under a fresh rcv	−0.4999 ZEC under a fresh rcv
$\mathsf{nf}_{\mathsf{old}}$	Alice's note's serial: the note is retired	the dummy's serial, recorded all the same
rk	a fresh disguise of Alice's ak	a fresh disguise of the dummy's ak
cmx	Bob's 1.5 ZEC note	Alice's 0.4999 ZEC change note, at her internal address
epk, C_{enc}	to Bob's $\mathsf{pk_d}$; opened with his ivk	to Alice's internal $\mathsf{pk_d}$; found with $\mathsf{ivk}^{\mathsf{int}}$
C_{out}	under Alice's ovk: she can re-read it	under Alice's internal ovk (ZIP-316)
bundle	one Ironwood anchor; flags $(1, 1, 1)$; $v_{\mathsf{balance}} = +10\,000$ zatoshi; one proof; two SpendAuthSigs; one binding signature	

Which of the two is "Action 1" on the wire is decided by the shuffle. Two nullifiers, two commitments: the shape of every two-Action bundle.

Ironwood Guide, "The outgoing ciphertext"; *Wallet Guide*, "Padding, dummies, and shuffling"; "Internal keys: change and auto-shielding"; "Outgoing viewing keys"

Coin exists — A3, the path to the anchor

Fold Extract($\mathsf{cm}_{\mathrm{old}}$) up 32 witnessed siblings to root; then $v_{\mathrm{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$.

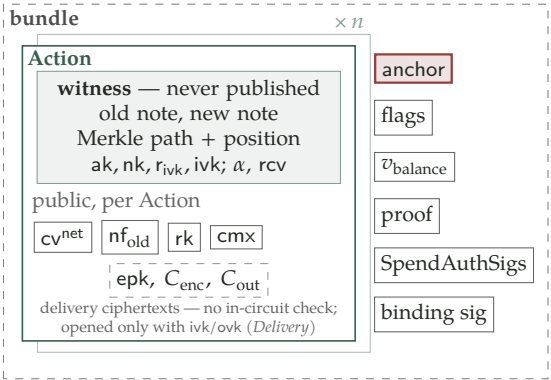


- Witness: the siblings and the position bits. Public: the anchor, one per bundle.
- Bitcoin looks the coin up in the UTXO set, and so names it. This clause says only "some leaf under this anchor": the candidate set is the Ironwood tree under the anchor.
- A zero-value dummy skips the check by arithmetic, not by a flag.

Which of the four checks?

Coin exists — A3, the path to the anchor

Fold Extract($\mathsf{cm}_{\mathrm{old}}$) up 32 witnessed siblings to root; then $v_{\mathrm{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$.



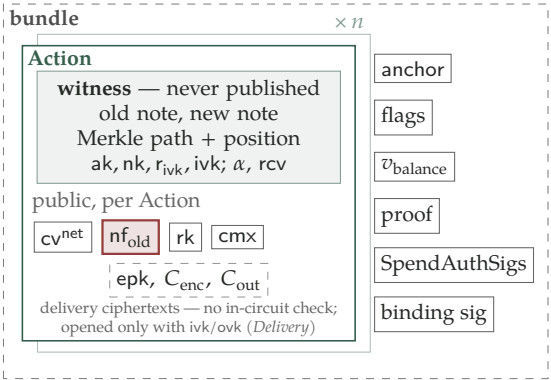
- Witness: the siblings and the position bits. Public: the anchor, one per bundle.
- Bitcoin looks the coin up in the UTXO set, and so names it. This clause says only "some leaf under this anchor": the candidate set is the Ironwood tree under the anchor.
- A zero-value dummy skips the check by arithmetic, not by a flag.

Which of the four checks? — coin exists.

Ironwood Guide, "The note commitment tree"; "Anchor choice as public data"; "The Action statement"

No double spend — A5, the nullifier

$$\mathsf{nf}_{\mathrm{old}} = \mathsf{Extract}\big([\,\mathsf{Poseidon}_{\mathsf{nk}}(\rho_{\mathrm{old}}) + \psi_{\mathrm{old}}\,]\,G^{\mathsf{nf}} + \mathsf{cm}_{\mathrm{old}}\big)$$

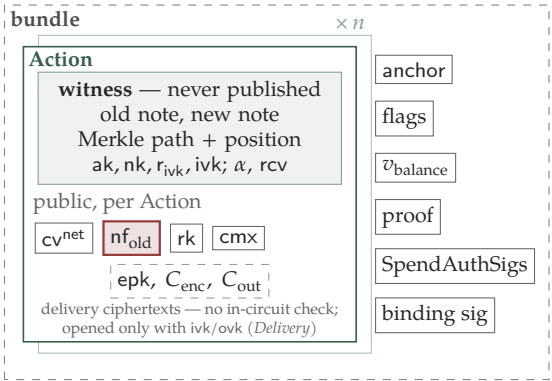


- Clause A5 binds the published serial to the spent note under the witnessed nk; otherwise a spender could invent a fresh serial per spend and the set would never repeat.
- "Seen before?" is global history: the node checks the set, the proof cannot.
- Bitcoin deletes the entry; here an unlinkable serial is added. The opening's three lines: membership (A3), serial (A5), balance (A4, *Balance and authority*).

Which check?

No double spend — A5, the nullifier

$$\mathsf{nf}_{\mathrm{old}} = \mathsf{Extract}\big([\,\mathsf{Poseidon}_{\mathsf{nk}}(\rho_{\mathrm{old}}) + \psi_{\mathrm{old}}\,]\,G^{\mathsf{nf}} + \mathsf{cm}_{\mathrm{old}}\big)$$



- Clause A5 binds the published serial to the spent note under the witnessed nk; otherwise a spender could invent a fresh serial per spend and the set would never repeat.
- "Seen before?" is global history: the node checks the set, the proof cannot.
- Bitcoin deletes the entry; here an unlinkable serial is added. The opening's three lines: membership (A3), serial (A5), balance (A4, *Balance and authority*).

Which check? — not spent twice.

Ironwood Guide, "Nullifiers"; "Verification of a transaction"

Authorised — the linkage problem first

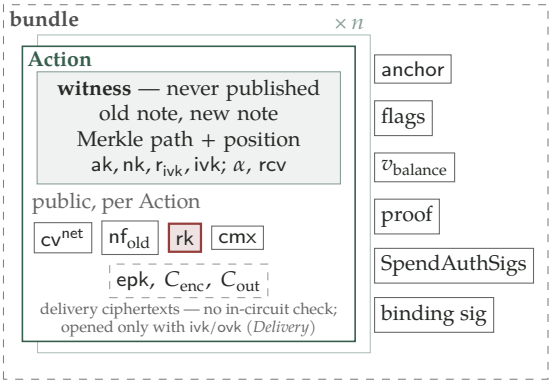
$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d} \quad \text{for every } d$$

- One spend key ak per wallet, not per coin: every diversified address shares it through ivk (*Keys and addresses*).
- The obvious design — publish ak and a signature under it — would stamp every spend the wallet ever makes with one key: every note it spends, from every address, linked.
- Bitcoin's version of this is address reuse, and it is optional. Here it would be structural.
- Needed: a key that is fresh for every spend and still proves the same secret.

Crypto Guide, "Key re-randomisation and unlinkability"; *Ironwood Guide*, "Randomised validating keys"

Authorised — A6, a disguised key

$$\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}, \qquad \mathsf{rsk} = \mathsf{ask} + \alpha$$

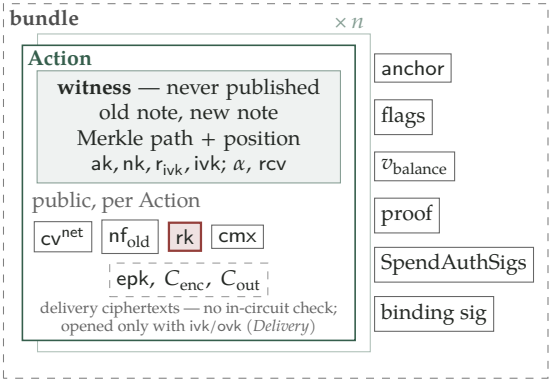


- Clause A6: the published key is ak shifted by a fresh, secret α. The circuit checks the shift; the node checks a signature under rk (*Balance and authority*).
- Uniform α makes rk uniform whatever ak is.
- Schnorr with a per-spend tweak: a non-hardened BIP-32 child whose tweak is uniform and never published.

Which check?

Authorised — A6, a disguised key

$$\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}, \qquad \mathsf{rsk} = \mathsf{ask} + \alpha$$



- Clause A6: the published key is ak shifted by a fresh, secret α. The circuit checks the shift; the node checks a signature under rk (*Balance and authority*).
- Uniform α makes rk uniform whatever ak is.
- Schnorr with a per-spend tweak: a non-hardened BIP-32 child whose tweak is uniform and never published.

Which check? — authorised.

Ironwood Guide, "Randomised validating keys"; *Crypto Guide*, "Key re-randomisation and unlinkability"

Authorised — A7, one identity behind the address

$$\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk}), \qquad \mathsf{pk_{d_{old}}} = [\mathsf{ivk}]\,\mathsf{g_{d_{old}}}$$

- Clause A6 alone proves a disguise of *some* ak; A7 ties that ak, and the nk of A5, to the spent note's address by the ladder of *Keys and addresses*.
- One identity signs (A6), nullifies (A5), and is paid (A7): the same secret behind all three, and none of them published.
- Authority is split in two: the circuit proves the key is the owner's; the signature under rk proves the spender holds it (*Balance and authority*).

In Bitcoin a P2PKH address *is* the key's hash, so "this key matches this coin" is one hash comparison in the clear. Here the address is a commitment to the key, and the match is proved.

Ironwood Guide, "The Action statement"; "Spend authorisation"

Well-formed — A1 and A2, both notes really are notes

$$\mathsf{cm}_{\mathrm{old}} = \mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{old}}}(\mathbf{n}_{\mathrm{old}}), \qquad \mathsf{Extract}(\mathsf{NoteCommit}_{\mathsf{rcm}_{\mathrm{new}}}(\mathbf{n}_{\mathrm{new}})) = \mathsf{cmx},$$
$$\text{with } \rho_{\mathrm{new}} := \mathsf{nf}_{\mathrm{old}}.$$

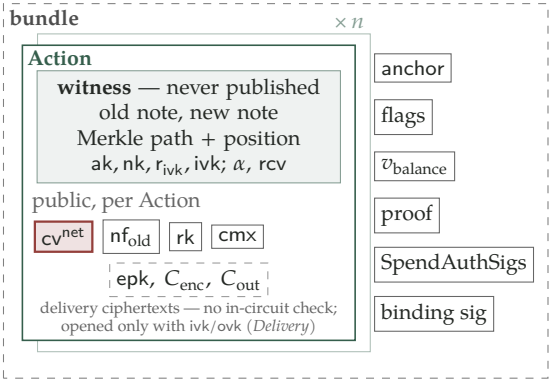
- Clause A1: the object being spent opens as a note — address, value, ρ, ψ — under the witnessed rcm; the commitment is binding, so one leaf cannot be reopened as a second note.
- Clause A2: the created note is committed the same way, and the chain stores only cmx.
- The chain $\rho_{\mathrm{new}} := \mathsf{nf}_{\mathrm{old}}$ is enforced *here*: the new note's seed is the serial this Action reveals, so accepted seeds are unique within the pool and two notes cannot share a nullifier.

A Bitcoin output is plaintext: well-formed means it parses. Here the note is hidden, so "this commitment opens to a note" must be proved.

Ironwood Guide, "Note commitments"; "Nullifier chaining"; "The Action statement"

Conserved — A4, the value commitment

$$\mathsf{cv}^{\mathsf{net}} = [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R, \qquad v_{\text{old}}, v_{\text{new}} \in [0, 2^{64}), \quad \text{sign} \in \{-1, +1\}$$

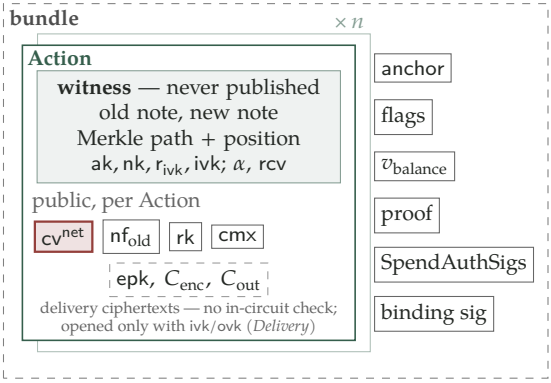


- Clause A4 commits to this Action's value *change* under a fresh trapdoor rcv; the bundle sums these (*Balance and authority*).
- The range is the point: the sum lives in a group of order about 2^{254}, where a value near the order is a small negative one. Unbounded, the books would "balance" while minting.
- Elements' range proof exists for the same reason; here it lives inside the Action proof, with the opening.

Which check?

Conserved — A4, the value commitment

$$\mathsf{cv}^{\mathsf{net}} = [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R, \qquad v_{\text{old}}, v_{\text{new}} \in [0, 2^{64}), \quad \text{sign} \in \{-1, +1\}$$



- Clause A4 commits to this Action's value *change* under a fresh trapdoor rcv; the bundle sums these (*Balance and authority*).
- The range is the point: the sum lives in a group of order about 2^{254}, where a value near the order is a small negative one. Unbounded, the books would "balance" while minting.
- Elements' range proof exists for the same reason; here it lives inside the Action proof, with the opening.

Which check? — conserved.

Ironwood Guide, "Value commitments"; "Value conservation"

Gated — A8 and A9, the flags

$$v_{\text{old}} \cdot (1 - \mathsf{enableSpends}) = 0, \qquad v_{\text{new}} \cdot (1 - \mathsf{enableOutputs}) = 0$$

- The flags are bundle-level bits the node supplies as public inputs; the circuit only multiplies.
- With a flag clear, only a zero value passes: a cleared enableSpends admits dummies and nothing else.
- A coinbase must clear enableSpends: its Actions spend dummies and create the block's shielded reward. Ordinary bundles set both flags, as Alice's does.

Bitcoin's coinbase has one null input by format. Here the same rule is arithmetic on a hidden value — the only way to gate what the node cannot read.

Ironwood Guide, "The Action statement"; "Coinbase transaction"; "Bundle flags"

Sealed — A10, the cross-address gate

$$\delta \cdot (x(\mathsf{g}_{\mathsf{d}_{\mathrm{old}}}) - x(\mathsf{g}_{\mathsf{d}_{\mathrm{new}}})) = 0, \quad \delta \cdot (y(\mathsf{g}_{\mathsf{d}_{\mathrm{old}}}) - y(\mathsf{g}_{\mathsf{d}_{\mathrm{new}}})) = 0,$$
$$\delta \cdot (x(\mathsf{pk}_{\mathsf{d}_{\mathrm{old}}}) - x(\mathsf{pk}_{\mathsf{d}_{\mathrm{new}}})) = 0, \quad \delta \cdot (y(\mathsf{pk}_{\mathsf{d}_{\mathrm{old}}}) - y(\mathsf{pk}_{\mathsf{d}_{\mathrm{new}}})) = 0$$

- The flag $\delta := \mathsf{disableCrossAddress}$ is one more public input, its position unspecified. On the wire it travels negated, as $\mathsf{enableCrossAddress}$, bit 2 of the flags byte.
- With $\delta = 1$ the four products force the new note to pay the spent note's own receiver, as full points. With $\delta = 0$ they vanish.
- The Ironwood component sets the bit (`0b111`): Alice's bundle has $\delta = 0$. An Orchard-pool component must clear it (`0b011`): $\delta = 1$.

The shape is A3's product gate again, on four more rows. A Bitcoin output type cannot be told "pay only yourself"; this one can.

Ironwood Guide, "Meaning of A10"; "Bundle flags"

The seal: three rules, and two checks that are not clauses

- **No issuance.** A coinbase carries an empty Orchard component.
- **No circulation.** Every Orchard-pool Action has $\mathsf{enableCrossAddress} = 0$, so A10 confines its output to the spent note's own receiver. Enforced by the verifying key consensus selects by branch, so it binds whatever the transaction version.
- **No entry.** Per transaction, $v_{\text{balance}}^{\mathsf{Orchard}} \geq 0$: value may leave the pool, never enter.

An owner of a sealed-pool note pays anyone by *leaving*: the value exits through a positive v_{balance}, and the output slot holds a zero-value note to the spent note's own receiver.

Not clauses. Before the proof, the node parses: every field element and point canonically encoded, and rk not the identity. Consensus rules, not relation.

A Bitcoin soft fork narrows what a script may do, in the clear; this seal narrows what a hidden note may do, by a public flag the circuit reads.

Ironwood Guide, "Chain state and pool rules"; "Meaning of A10"; "Verification of a transaction"

Checkpoint

Which clause would you delete to counterfeit, and what stops you?

Checkpoint
Which clause would you delete to counterfeit, and what stops you?

- Delete A3 and spend a note nobody minted; delete A4's range and wrap a subtraction into new value; delete A1 and reopen one leaf as two notes; delete A5 and spend one note under a fresh serial each time.
- What stops you: the circuit is fixed, and consensus selects its verifying key by branch. A proof of a weaker statement is a proof for a different circuit, and no node holds that key.

Ironwood Guide, "Knowledge soundness of the Action proof"; "The Action circuit and the Halo 2 proof"

Intermission

Break

Which clause would you delete to counterfeit, and what stops you?

Part E′

Balance and authority

Conservation: the envelopes add

The commitment of Part B, $\mathsf{Commit}(v; r)$, written $C(v; r)$ from here on: $C(v; r) := [v]\,V + [r]\,R$, with V and R hashed to Pallas so that nobody knows $\log_V R$. Toy stand-in: the additive group $\mathbb{Z}_q$, $q = 65\,537$, $V = 7$, $R = 11$; one unit is 10^4 zatoshi, the fee. Logs are free here; on Pallas they are not.

note	v (units)	r	$C(v; r)$
Alice's, 2 ZEC	20 000	4 321	56 457
Bob's, 1.5 ZEC	15 000	1 234	53 037
change, 0.4999 ZEC	4 999	9 876	12 555

$$C_A - C_B - C_c - [1]\,V = 56\,395 = [\,4\,321 - 1\,234 - 9\,876\,]\,R = [58\,748]\,R.$$

– The V-components cancel because the values do; a commitment to zero under the net randomness survives, and only the author who drew the three r knows that net randomness.

Ironwood Guide, "Value commitments"; *Crypto Guide*, "The Pedersen commitment"

Conservation: per Action, not per output

- Bitcoin subtracts three amounts it reads.
- Confidential Transactions: one commitment per input and per output, and the two sums must differ by the fee.
- Here: one commitment per Action, already the net of its spend and its output,

$$\mathsf{cv} := [v_{\text{old}} - v_{\text{new}}]\, V + [\mathsf{rcv}]\, R,$$

  so Alice's bundle publishes two envelopes for three notes and declares the net flow v_{balance} beside them.

Ironwood Guide, "Net value commitment"; *Crypto Guide*, "The Pedersen commitment"

Cancellation is modular: why A4 range-checks

The sum cancels modulo the group order, and a value at the order commits like zero: $C(q+3;\,5) = C(3;\,5) = 76$, where q units is 6.5537 ZEC.

- A counterfeit the toy accepts: change of $4\,999 + q = 70\,536$ units, 7.0536 ZEC, from a 2 ZEC note. The commitment is unchanged, $C(70\,536;\,9\,876) = 12\,555 = C(4\,999;\,9\,876)$, and $20\,000 - 15\,000 - 70\,536 - 1 = -65\,537 \equiv 0 \pmod{q}$.
- A4 closes the gap inside the Action proof: $v_{\text{old}}, v_{\text{new}} \in [0, 2^{64})$ and sign $\in \{-1, +1\}$, so every net value is a small signed integer; the Pallas order has 255 bits, and a bundle's bounded sum of bounded values cannot reach it.
- Elements attaches a range proof to each output. Here the range check is one clause of the statement the Action proof already carries. Bitcoin reads each amount and bounds it.

Ironwood Guide, "The Action statement"; "Modular balance is integer balance"; *Crypto Guide*, "The Pedersen commitment"

The binding signature: a key only a balanced author knows

Per Action, A4 fixes $\mathsf{cv} = [v_{\text{old}} - v_{\text{new}}]\,V + [\mathsf{rcv}]\,R$. Summing over the bundle and subtracting the declared net flow v_{balance}:

$$\sum_i \mathsf{cv}_i = \left[\textstyle\sum_i (v_{\text{old}} - v_{\text{new}})_i\right] V + [\mathsf{bsk}]\,R, \qquad \mathsf{bsk} := \textstyle\sum_i \mathsf{rcv}_i,$$

$$\mathsf{bvk} := \sum_i \mathsf{cv}_i - [v_{\text{balance}}]\,V, \qquad \text{balanced} \iff \mathsf{bvk} = [\mathsf{bsk}]\,R.$$

- The key bvk is public: every node recomputes it from the wire. Its secret bsk is a sum of trapdoors only the author drew, and a discrete log of bvk only when the books balance; otherwise finding one means finding $\log_R V$.
- The binding signature is RedPallas under bvk on base R: a Schnorr proof of knowledge of bsk, hence of balance, revealing no amount and not bsk itself.
- Bitcoin's check is arithmetic on amounts it reads; this one is a signature check under a key derived from commitments.

Ironwood Guide, "The binding signature"; "Balance and key knowledge"; *Crypto Guide*, "Binding signatures as a proof of knowledge of a discrete logarithm"

The one public amount: v_{balance}

The field v_{balance} is cleartext and signed, one per pool; a positive value is value leaving the pool.

- Fully shielded, the only value leaving is the fee, so $\sum_i (v_{\text{old}} - v_{\text{new}})_i = \text{fee}$ and $v_{\text{balance}} = \text{fee}$: Bitcoin's implicit $\sum \text{in} - \sum \text{out} = \text{fee}$ becomes a public, signed number. Alice's reads +10 000 zatoshi.
- The transparent leg enters through the same field. Transparent outputs are paid from a positive v_{balance}; shielding funds a negative one, its Actions spending dummies and creating real notes. The crossing amount is public; the receiving note is not.
- Bitcoin's fee is reconstructed by looking up every input. Fully shielded, as Alice's is, the balancing field is the one amount the transaction states, and it is the fee; with a transparent leg the fee is again a difference of public amounts, as in Bitcoin.

Ironwood Guide, "Value commitments and the binding signature"; "The balancing value"; *Wallet Guide*, "The conventional fee"

Authorisation on the wire: two signatures, one scheme

RedPallas is Schnorr on Pallas with the base point P a parameter: key $X = [x]\,P$; a signature (N, s), $32 + 32 = 64$ bytes, verifies iff $[s]\,P = N + [c]\,X$ with $c = H(N \,\|\, X \,\|\, m)$. The nonce commitment N is Part B's R, renamed so that R can stay the commitment base.

	per	base P and key X	knowing x means
SpendAuthSig	Action	$G^{\mathsf{SpendAuth}}$, $\mathsf{rk} = \mathsf{ak} + [\alpha]\,G^{\mathsf{SpendAuth}}$	authority: $\mathsf{rsk} = \mathsf{ask} + \alpha$
binding signature	pool	R, $\mathsf{bvk} = \sum \mathsf{cv} - [v_{\text{balance}}]\,V$	balance: $\mathsf{bsk} = \sum \mathsf{rcv}$

Ironwood Guide, "Spend-authorisation and binding-signature instances"; "The transaction format"; *Crypto Guide*, "RedDSA: re-randomisable Schnorr for Orchard"

Authorisation on the wire: who checks what

- Division of labour. The circuit checks the key's pedigree, which must stay hidden: rk is a shift of the ak welded into the spent note's address (A6, A7). The node checks the signature, which is public: the holder of that key approved this transaction. Signing needs ask + α — both, or nothing.
- Bitcoin: the coin's own key, in the script or hashed there, revealed at its one spend; reuse links only if the address was reused. Here one ak stands behind every address of the wallet, so an un-randomised spend would publish it at every note the wallet ever spends; hence a fresh, uniformly distributed rk per spend, unlinkable to ak and to every other rk.

Ironwood Guide, "Randomised validating keys"; *Crypto Guide*, "Key re-randomisation and unlinkability"

What the signatures sign

Every shielded signature in the transaction signs one message: the ZIP-244 signature digest, a tree of personalised BLAKE2b hashes, one node per transaction component.

- It commits to all effecting data — everything under the txid: each Action's nf, cmx, cv, rk and ciphertexts, the flags, v_{balance}, and the branch identifier of the network upgrade. No Action can be altered or transplanted into another transaction.
- For the shielded signatures the sighash type is fixed at ALL: nothing like ANYONECANPAY, SINGLE or NONE. One message, all of it, for the spend-authorisation and binding signatures alike.

What the signatures sign: the anchor and the txid

- One field the digest used to cover no longer is: from v6 the anchor joins the proofs and signatures in authorizing data. The proof pins it as a public input (A3), and the block commits to authorizing data (Part H); a transaction can be re-anchored without changing its txid: specified (ZIP-229), wallet support pending (Part H).
- Without transparent inputs, the signature digest is the txid. Bitcoin's sighash byte chooses coverage per input; so do Zcash's transparent inputs, but every shielded signature covers all of it.

Ironwood Guide, "Transaction digests and signatures"; *Wallet Guide*, "Authorisation of a transaction"

Alice's two Actions

	Action 1	Action 2
spent note, v_{old}	Alice's, 2 ZEC	dummy, 0: A3 gated off
created note, v_{new}	Bob's, 1.5 ZEC	change, 0.4999 ZEC, internal address
$v_{\text{old}} - v_{\text{new}}$	+0.5 ZEC	−0.4999 ZEC
nf	Alice's note's	the dummy's
rk, SpendAuthSig	$\mathsf{ak} + [\alpha_1]\, G^{\mathsf{SpendAuth}}$, Alice signs	a throwaway key's, it signs
toy cv	$C(+5\,000;\, 3\,141) = 4\,014$	$C(-4\,999;\, 2\,718) = 60\,442$

- Two outputs force $n_{\text{act}} = \max(\max(1, 2), 2) = 2$, so one dummy spend pads the spend list: a zero-value note under a fresh random spending key, its ρ the Extract of a random point; it still publishes a nullifier and still signs.
- Fee $= 5\,000 \cdot \max(2, 2) = 10\,000$ zatoshi $= v_{\text{balance}}$. The change goes to Alice's internal address, derived from her own keys and never published.

Wallet Guide, "Padding, dummies, and shuffling"; "The conventional fee"; "Internal keys: change and auto-shielding"; protocol specification, "Dummy Notes (Orchard)"

The bundle balances, however it is shuffled

Toy check on the pairing shown:

$$\sum \mathsf{cv} = 4\,014 + 60\,442 = 64\,456, \qquad \mathsf{bvk} = 64\,456 - [1]\,V = 64\,449 = [5\,859]\,R,$$
$$\mathsf{bsk} = 3\,141 + 2\,718 = 5\,859.$$

- The other pairing, Alice's note with the change and the dummy with Bob: $\mathsf{cv} = 8\,484$ and $55\,972$, the same $\sum \mathsf{cv} = 64\,456$, the same bvk and bsk. Balance is a property of the bundle, not of the pairing.
- Spends and outputs are padded, shuffled independently, and zipped; each Action draws its own rcv and α, the dummy included, so nothing on the wire says which Action is the dummy.
- Bitcoin: one input naming the 2-coin output, two outputs, and the change tell — one round amount and one not. Here: two Actions of one shape and a fee.

Wallet Guide, "Padding, dummies, and shuffling"; "Authorisation of a transaction"; *Ironwood Guide*, "The binding signature"

Checkpoint

What does the validator learn from Alice’s bundle?

Checkpoint

What does the validator learn from Alice's bundle?

- That the four checks pass. The anchor is a main-chain root and the proof verifies: exists. Two nullifiers are absent from the set: not twice. Two SpendAuthSigs verify under $\mathsf{rk}_1, \mathsf{rk}_2$: authorised. The binding signature verifies under the recomputed bvk: conserved.
- The metadata: two Actions; $v_{\text{balance}} = +10\,000$ zatoshi, hence the fee; the anchor; two nullifiers, two cmx, two ciphertext pairs; 1 640 bytes of Actions, a 7 264-byte proof, three 64-byte signatures; when it arrived.
- Not: who, to whom, the amounts, which leaf, or which Action is the dummy — or whether either is.

Ironwood Guide, "Verification of a transaction"; "A two-Action payment"; "The transaction format"

Part F

The proof system: from a statement to a few kilobytes

A zk-SNARK, precisely enough

A *relation* R pairs a public statement x with a private witness w; the prover holds both, the verifier only x and a short string:

$$\mathrm{Prove}(x, w) \to \pi, \qquad \mathrm{Verify}(x, \pi) \in \{0, 1\}.$$

- *Knowledge sound*: from any prover that convinces the verifier, an extractor pulls out a w with $(x, w) \in R$. Plain soundness would allow "a spendable note exists"; other people's notes exist. The prover must *hold* one.
- *Zero knowledge*: a simulator given only x reproduces π: the proof adds nothing beyond the statement (next slide).

A Schnorr signature is a proof of knowledge for the one relation "I know $\log_G X$"; a SNARK is the same guarantee for any relation — here the ten clauses of the Action statement.

Crypto Guide, "SNARKs: succinct non-interactive arguments of knowledge"; "Knowledge soundness and extractors"; "The simulation paradigm and zero knowledge"

Zero knowledge: what π hides, and what it does not

- The proof reveals nothing beyond the public inputs: whatever a verifier learns from π, a simulator produces from x alone.
- The public inputs (nf, rk, cv, $\mathsf{cm_x}$) are *published*; that they cannot be linked to the note and key rests on other assumptions — commitment hiding, the keyed nullifier PRF, key re-randomisation (Parts C and E) — not on π.
- Zero knowledge is necessary, not sufficient: a perfect proof beside a nullifier equal to $\mathsf{cm_x}$ would hide nothing.

Bitcoin's witness is the secret's public shadow (a signature, a script preimage); here the witness never leaves the prover, and the shadow is the public-input tuple, unlinkable for reasons the proof does not supply.

Crypto Guide, "The simulation paradigm and zero knowledge"; *Ironwood Guide*, "Zero knowledge of the Action proof"

Succinct, and non-interactive

- *Succinct*: the proof is logarithmic in the circuit (commitments and openings, never the polynomials). Halo 2's verifier still pays one linear multi-scalar multiplication (MSM), shared across a batch; it is a SNARK in the proof-size sense.
- *Non-interactive*: wherever a live verifier would toss a coin, the prover computes

  $$\text{challenge} := \text{Hash}(\text{everything sent so far}),$$

  statement and every commitment included — Fiat–Shamir. A commitment is an input of the hash, so it is fixed before the challenge that tests it exists. The proof becomes one static object; verification recomputes the hashes and runs the same checks.

A full node that has never met Alice verifies her Action long after it was made, from the proof string alone. Schnorr is Fiat–Shamir applied to the log relation: *proof of knowledge + Fiat–Shamir = signature*; a SNARK is the same recipe, kept short.

Crypto Guide, "SNARKs: succinct non-interactive arguments of knowledge"; "The Fiat–Shamir transform: from interactive to non-interactive"; *Halo 2 Guide*, "Removing the verifier: the Fiat–Shamir transform"

Arithmetisation: the claim becomes a table

Toy claim: "I know x with $x^3 + x + 5 = 35$" over $\mathbb{F}_{97}$ ($x = 3$) — the Action statement in miniature. One operation per row; one gate equation, evaluated on every row:

$$q_M\, a\, b + q_L\, a + q_R\, b + q_O\, c + q_C + \mathrm{PI} = 0$$

	advice			fixed					instance	
row	a	b	c	q_M	q_L	q_R	q_O	q_C	PI	gate
0	3	3	9	1	0	0	−1	0	0	$x \cdot x = x^2$
1	9	3	27	1	0	0	−1	0	0	$x^2 \cdot x = x^3$
2	27	3	30	0	1	1	−1	0	0	$x^3 + x = s$
3	30	0	0	0	1	0	0	5	−35	$s + 5 = 35$

Advice is the prover's secret trace; fixed columns are the circuit; the instance column carries the public 35. Copy constraints wire the rows ($a_0 = b_0 = b_1 = b_2$, $c_0 = a_1$, $c_1 = a_2$, $c_2 = a_3$). A range check is not algebra: it is a *lookup* into a table. Bitcoin Script is executed by every validator; a circuit is never executed, only checked.

Halo 2 Guide, "From computation to a grid of constraints"; "Proving a value sits in a table: lookups"; "PLONKish arithmetisation"

Rows become polynomials

Name the rows by the fourth roots of unity in $\mathbb{F}_{97}$: $\omega = 22$, $\omega^2 = 96 = -1$, $\omega^4 = 1$, so row i lives at $\omega^i \in H = \{1, 22, 96, 75\}$. *Interpolate* each column: the advice column a is the unique polynomial of degree < 4 through its four values,

$$a(1) = 3,\ a(22) = 9,\ a(96) = 27,\ a(75) = 30 \quad \Longrightarrow \quad a(X) = 90 + 61X + 22X^2 + 24X^3.$$

Four numbers in, four coefficients out: the column *is* the polynomial, sampled at the rows. Likewise b, c and the public q_M, q_L, q_R, q_O, q_C, PI. Substitute into the gate equation:

$$G(X) := q_M ab + q_L a + q_R b + q_O c + q_C + \mathrm{PI}, \qquad \deg G \le 9,$$

and $G(\omega^i)$ is exactly row i's gate equation. So

$$\text{every gate holds} \iff G(\omega^i) = 0 \text{ for all } i \iff G \text{ vanishes on } H.$$

A Merkle root also folds many rows into one object, but it opens one leaf at a time; a polynomial can be probed anywhere.

Halo 2 Guide, "From the grid to polynomials"

All rows at once: one identity, or none

The polynomial that is zero exactly on the rows is $Z_H(X) := \prod_{h \in H}(X - h) = X^4 - 1$, and by the factor theorem, applied four times,

$$G \text{ vanishes on } H \iff Z_H \mid G \iff G = t \cdot Z_H \text{ for some polynomial } t.$$

Honest trace: $\deg G = 9$, $\deg t = 5$, remainder 0. Now let the prover claim $x^2 = 10$ and write $c_0 = 10$: row 0's gate reads $3 \cdot 3 - 10 = -1 \neq 0$, so $Z_H \nmid G^*$, and for the Euclidean quotient t^*

$$D(X) := G^*(X) - t^*(X)\, Z_H(X) = 24\,(1 + X + X^2 + X^3) \neq 0.$$

A correct computation *is* a polynomial identity; a wrong one cannot be. The copy constraints get the same treatment (a running product that equals 1 when the wiring holds and, at random challenges, almost never otherwise), and so do lookups. Bitcoin's validator checks each input's script on its own; here every row collapses into one identity that holds everywhere or fails almost everywhere.

Halo 2 Guide, "From vanishing to a single identity"; "Enforcing the wiring: the permutation argument"

One random point: the protocol

1. The prover *commits* to a, b, c and t (two slides on); the verifier already knows $q_M, \ldots, q_C$ and PI.
2. The transcript hash draws $z \in \mathbb{F}$ — Fiat–Shamir; BLAKE2b in the deployed transcript.
3. The prover opens $a(z), b(z), c(z), t(z)$ with short proofs that these are the committed polynomials' values.
4. The verifier forms $G(z)$ from the openings, computes $Z_H(z) = z^4 - 1$, and checks one field equation:

$$G(z) \stackrel{?}{=} t(z)\, Z_H(z).$$

Honest trace, $z = 20$: $Z_H(20) = 46$, $t(20) = 67$, $G(20) = 75 = 67 \cdot 46$. ✓ Every honest prover passes at every z, because $G = t\, Z_H$ holds as polynomials.

Bitcoin's validator re-executes the script; this one evaluates one equation at one point.

Halo 2 Guide, "Why checking one random point is a proof"; "Removing the verifier: the Fiat–Shamir transform"; "The complete Halo 2 protocol"

One random point: why a lie is caught

Cheat: $D := G^* - t^* Z_H$ is nonzero and *frozen* before z is drawn — the commitments pin G^* and t^*. The check passes exactly when $D(z) = 0$, and a nonzero polynomial has at most $\deg D$ roots (Schwartz–Zippel), so

$$\Pr_z[\text{cheat accepted}] = \Pr_z[D(z) = 0] \le \frac{\deg D}{|\mathbb{F}|}.$$

- For the $c_0 = 10$ cheat, $D = 24(1 + X + X^2 + X^3)$ has roots $\{22, 75, 96\}$ — the three rows where the tampered gate still held: $3/97$ for that t^*, $11/97$ for any admissible t^*.
- At $z = 20$: $G^*(20) = 20$ but $t^*(20)\, Z_H(20) = 64$. Caught.
- Over $\mathbb{F}_{p_{\text{Pallas}}}$ with $\deg D \approx 2^{14}$ the term is about 2^{-240} — one Schwartz–Zippel term, not the system's security level; the discrete-log layer gives about 126 bits.

A false statement is too rigid to hide: it disagrees with the truth almost everywhere, and one probe lands where it disagrees. Bitcoin never needs this; its validator sees the data.

Halo 2 Guide, "Why checking one random point is a proof"; *Math Guide*, "The Schwartz–Zippel lemma"

Committing to a polynomial: the problem

The random-point argument assumed the polynomials were fixed *before* the random point. Needed: a short commitment P to f, and a short proof that $f(x) = v$ at a drawn point x for the committed f.

Commit to the coefficient vector $\vec{a}$ with a Pedersen vector commitment, generators $\vec{G}$ and H hashed into existence:

$$P = \mathsf{Commit}(f; r) = \langle \vec{a}, \vec{G} \rangle + [r]\, H \in \mathbb{G}$$

— one Vesta point for 2^{11} coefficients; binding under discrete log, hiding by $[r]H$, no secret behind $\vec{G}$. A Merkle root also commits to a vector, and opens one entry in $\log_2 d$ hashes; the next slide opens a linear combination of *all* entries in $2 \log_2 d$ points.

Halo 2 Guide, "Why the prover cannot wriggle: binding commitments"; *Crypto Guide*, "Polynomial commitment schemes"; "Hashing to a curve point"

Committing to a polynomial: the opening claim

Evaluation is an inner product with a public vector:

$$f(x) = \langle \vec{a}, \vec{b}(x) \rangle, \qquad \vec{b}(x) := (1, x, x^2, \dots, x^{d-1}).$$

Fold the claimed value in with a second hashed generator U: the opening claim becomes

$$P + [v]\,U = \langle \vec{a}, \vec{G} \rangle + [r]\,H + [\langle \vec{a}, \vec{b} \rangle]\,U.$$

Sending $\vec{a}$ would cost d elements. The *inner-product argument* — the Bulletproofs family you know from range proofs — proves the inner product in $2 \log_2 d$ points: 22 for $d = 2^{11}$. A Merkle proof names one leaf; an IPA opening is one number computed from every coefficient, at a point chosen after they were fixed.

Halo 2 Guide, "The inner-product polynomial commitment"; *Crypto Guide*, "Polynomial commitment schemes"

The inner-product argument: one fold

Split every vector in half, $\vec{a} = (\vec{a}_{\mathrm{lo}}, \vec{a}_{\mathrm{hi}})$, likewise $\vec{G}$ and $\vec{b}$, and publish the two cross terms

$$L := \langle \vec{a}_{\mathrm{lo}}, \vec{G}_{\mathrm{hi}} \rangle + [\ell]\, H + [\langle \vec{a}_{\mathrm{lo}}, \vec{b}_{\mathrm{hi}} \rangle]\, U,$$
$$R := \langle \vec{a}_{\mathrm{hi}}, \vec{G}_{\mathrm{lo}} \rangle + [r_R]\, H + [\langle \vec{a}_{\mathrm{hi}}, \vec{b}_{\mathrm{lo}} \rangle]\, U.$$

Draw a challenge u and fold, both parties in step:

$$\vec{a}' := u\,\vec{a}_{\mathrm{lo}} + u^{-1}\vec{a}_{\mathrm{hi}}, \quad \vec{G}' := u^{-1}\vec{G}_{\mathrm{lo}} + u\,\vec{G}_{\mathrm{hi}}, \quad \vec{b}' := u^{-1}\vec{b}_{\mathrm{lo}} + u\,\vec{b}_{\mathrm{hi}}.$$

Multiply out: $\langle \vec{a}', \vec{G}' \rangle = \langle \vec{a}, \vec{G} \rangle + u^2 \langle \vec{a}_{\mathrm{lo}}, \vec{G}_{\mathrm{hi}} \rangle + u^{-2} \langle \vec{a}_{\mathrm{hi}}, \vec{G}_{\mathrm{lo}} \rangle$, and the same for $\langle \vec{a}', \vec{b}' \rangle$. Hence

$$P + [v]\, U + [u^2]\, L + [u^{-2}]\, R = \langle \vec{a}', \vec{G}' \rangle + [r + u^2\ell + u^{-2} r_R]\, H + [\langle \vec{a}', \vec{b}' \rangle]\, U :$$

the same claim at half the length. The verifier updates $\vec{G}'$ and $\vec{b}'$ by public arithmetic and never sees $\vec{a}$. This is the textbook (Bulletproofs) form; a Bulletproofs range proof folds its vectors by exactly this move.

Halo 2 Guide, "The inner-product polynomial commitment", Construction "IPA-based polynomial commitment"

The inner-product argument: the recursion

After $k = \log_2 d$ rounds every vector has length 1. The proof is $2k$ points (L_j, R_j), the final scalar a, and the aggregated blinder $r^* = r + \sum_j (u_j^2 \ell_j + u_j^{-2} r_j)$. The verifier accepts iff

$$P + [v]\,U + \sum_{j=1}^{k} ([u_j^2]\,L_j + [u_j^{-2}]\,R_j) \stackrel{?}{=} [a]\,G^{(0)} + [r^*]\,H + [a \cdot b^{(0)}]\,U.$$

The inner-product argument: verification and binding

- The scalar $b^{(0)} = \prod_\ell (u_\ell^{-1} + u_\ell \, x^{2^{\ell-1}})$ costs $O(\log d)$ field operations.
- The point $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ is a multi-scalar multiplication of length d — the one linear step. The node shares it across a batch; recursion would defer it into an accumulator (not deployed).
- *Binding*: two openings of one commitment would exhibit a non-trivial discrete-log relation among $\vec{G}, H$ on Vesta; an extractor rewinds three challenges per round.

A Bitcoin signature check costs a fixed handful of group operations; an IPA opening is logarithmic in size but ends in a circuit-sized wall of group arithmetic.

Halo 2 Guide, "The inner-product polynomial commitment"; "Accumulation and the Halo trick"; *Crypto Guide*, "Polynomial commitment schemes"

What the fold hides, and the deployed normalisation

- Folding alone is *not* zero knowledge: the final scalar a is a challenge-dependent linear form in the coefficients, and blinding P and the round points hides those points, not a.
- The deployed opening first masks f by a random polynomial vanishing at the opening point, commits to it as S, draws two further challenges, and only then folds.
- The deployed fold is rescaled: weights $(1, u_j^{-1})$ on the coefficients, $(1, u_j)$ on $\vec{b}$ and $\vec{G}$, the verifier accumulating $[u_j^{-1}]\,L_j + [u_j]\,R_j$; the two conventions are related by per-round rescaling.

The slides' $[u^2]L + [u^{-2}]R$ form is the textbook one; the deployed proof carries the same $2k$ points plus S and two final scalars. A Bitcoin signature exposes its public key by design; here even the last scalar of an opening must be masked.

Halo 2 Guide, "The inner-product polynomial commitment", Remarks "Folding alone is not zero knowledge", "The deployed IPA normalisation"

The fold by hand: two rounds

A $d = 4$ fold with numbers: the toy $p(X) = 5 + X + X^3$ opened at $x = 3$, so $v = 35$. Stand-in group $\mathbb{Z}_{97}$ written additively — a "point" is a residue and $[k]G = kG \bmod 97$; it has no discrete-log hardness, only the fold's algebra.

$$\vec{a} = (5,1,0,1), \qquad \vec{b} = (1,3,9,27), \qquad \vec{G} = (2,3,5,7),$$
$$H = 11, \qquad U = 13, \qquad r = 17, \qquad P = \langle \vec{a}, \vec{G} \rangle + [r]H = 13.$$

round	L_j	R_j	u_j	$\vec{a}'$	$\vec{b}'$	$\vec{G}'$	fold check
$j = 2$ $(\ell_2, r_2 = 19, 23)$	13	4	3	$(15, 68)$	$(92, 82)$	$(48, 22)$	$25 = 25$
$j = 1$ $(\ell_1, r_1 = 29, 31)$	52	58	5	(11)	(21)	(42)	$12 = 12$

Each row's fold check is the one-fold identity: the folded claim $\langle \vec{a}', \vec{G}' \rangle + [r^*]H + [\langle \vec{a}', \vec{b}' \rangle]U$ equals the accumulated $P + [v]U + [u^2]L + [u^{-2}]R$.

The fold by hand: the final check

The prover sends $a = 11$ and $r^* = 30$. From the challenges alone the verifier computes

$$\vec{s} = (u_1^{-1}u_2^{-1},\ u_1u_2^{-1},\ u_1^{-1}u_2,\ u_1u_2) = (13, 34, 20, 15), \qquad G^{(0)} = \langle \vec{s},\ \vec{G} \rangle = 42,$$
$$b^{(0)} = g(x; u_1, u_2) = (u_1^{-1} + u_1 x)(u_2^{-1} + u_2 x^2) = 21,$$

and compares the two sides:

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 = 12,$$
$$[a]\,G^{(0)} + [r^*]\,H + [a \cdot b^{(0)}]\,U = 12.$$

Equal: accepted. Four coefficients were never sent; four points and two scalars were. In $\mathbb{Z}_{97}$ anyone can divide, so nothing here is hidden or binding; on Vesta the same equations are, and $G^{(0)}$ is a length-2^{11} multi-scalar multiplication.

Fields and curves in the deployed circuit

- The circuit's native field is $\mathbb{F}_{p_{\mathsf{Pallas}}}$, the *base* field of Pallas: the points the statement handles — cm, rk, cv, ak, $\mathsf{pk_d}$ — have coordinates there, so the group law is native arithmetic, not emulated.
- The column polynomials are committed on Vesta, whose *scalar* field is exactly $\mathbb{F}_{p_{\mathsf{Pallas}}}$: the coefficients are the circuit's own field elements.

Bitcoin's secp256k1 is one curve doing one job: its validator verifies signatures, never circuits. And its base field, as the two-adicity slide shows, cannot even host the prover's evaluation domain.

Ironwood Guide, "The Action circuit and the Halo 2 proof"; *Math Guide*, "Base fields, scalar fields, and the Pasta cycle"

The Pasta cycle, once

Pallas over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ has order p_{Vesta}; Vesta over $\mathbb{F}_{p_{\mathsf{Vesta}}}$ has order p_{Pallas}: each curve's scalar field is the other's base field.

- A Vesta-committed proof is Vesta points with coordinates in $\mathbb{F}_{p_{\mathsf{Vesta}}}$, Pallas's scalar field; the circuit that verifies it natively is one over $\mathbb{F}_{p_{\mathsf{Vesta}}}$, committed on Pallas.
- That circuit's proof is Pallas points with coordinates in $\mathbb{F}_{p_{\mathsf{Pallas}}}$, verified natively by a circuit over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, committed on Vesta — proofs alternate between the two curves, each verifier a circuit, so one proof can attest to the previous one.
- Not deployed: the node verifies each bundle's proof directly and batches the deferred multi-scalar multiplications.

Bitcoin needs no such pair; a signature is verified by arithmetic, not by another proof.

Halo 2 Guide, "A cycle of curves for efficient native recursion"; "Accumulation and the Halo trick"; *Math Guide*, "Base fields, scalar fields, and the Pasta cycle"

Why not Bitcoin's curve: two-adicity

Interpolating a column over $n = 2^{11}$ rows needs a primitive 2^{11}-th root of unity, hence $2^{11} \mid p - 1$; the prover's quotient FFTs run on $8n = 2^{14}$ points at the deployed gate degree. The largest k with $2^k \mid p - 1$ is the field's *two-adicity*.

field	bits	two-adicity of $p - 1$	largest domain
$\mathbb{F}_{p_{\text{Pallas}}}$ (Pallas base)	255	32	2^{32}
$\mathbb{F}_{p_{\text{Vesta}}}$ (Vesta base)	255	32	2^{32}
secp256k1 base field	256	1	2
secp256k1 scalar field	256	6	64

The Pasta primes were designed with two-adicity exactly 32 (*Math Guide*; their values: protocol specification, "Pallas and Vesta"): domains of every size up to 2^{32}. Bitcoin's curve buys its implementation speed from $a = 0$ (*Crypto Guide*); its prime $p = 2^{256} - 2^{32} - 977$ was not chosen for evaluation domains: $p - 1$ is twice an odd number, a domain of two points. The secp256k1 rows are this deck's own computation, not a volume figure.

Math Guide, "Pallas and Vesta assembled", Remark "Two-adicity and FFT-friendly primes"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"; *Halo 2 Guide*, "The complete Halo 2 protocol"

Trust: what a ceremony is, and why there is none

Pairing SNARKs (Sapling's Groth16) bake a secret into the parameters; the pairing-based polynomial commitment (KZG, after Kate, Zaverucha and Goldberg) shows the shape:

$$pp = ([\tau^0]G, [\tau^1]G, \dots, [\tau^d]G, \dots), \qquad \tau \text{ destroyed.}$$

Whoever keeps τ forges: for any C, z, y the element $\pi := [(\tau - z)^{-1}]\,(C - [y]G)$ passes the check with no polynomial behind it — the *toxic waste*. Groth16's secrets are per circuit. A multiparty ceremony keeps the waste unknown provided *one* participant destroyed their share; the Sapling pool still rests on that.

The bound: under ZIP-209 each pool's chain value balance is public and may not go negative, so a forged Sapling proof creates value only inside the Sapling pool and can withdraw at most what that pool publicly holds — the turnstile (Part H).

Orchard and Ironwood: the IPA generators are hashed from a public domain tag, nothing to leak, nothing to destroy; binding rests on discrete log alone. Bitcoin never had a setup; Sapling has one; Orchard returns to none.

Crypto Guide, "Polynomial commitment schemes"; "SNARKs: succinct non-interactive arguments of knowledge"; *Halo 2 Guide*, "The 'Halo' in Halo 2: deferring the expensive check"; *Ironwood Guide*, "Chain value pool balances"

Three generations

	Sprout	Sapling	Orchard, Ironwood
SNARK	BCTV14 (original)	Groth16	Halo 2
curve	BN254	BLS12-381, Jubjub in-circuit	Pallas, Vesta
setup	six-party MPC	large MPC	none
hash in circuit	SHA-256	Pedersen	Sinsemilla
proof	296 B	192 B	$2720 + 2272a$ B
verification	constant	three pairings	$O(\log n)$ + shared MSM

A soundness flaw in BCTV14, found after deployment, would have allowed undetected counterfeiting; Sprout was re-proved with Groth16 and deprecated. Sapling: Groth16 over BLS12-381, Jubjub inside the circuit; a far faster prover, a per-circuit ceremony, no curve cycle. Orchard's proof grew because transparency costs $2 \log_2 n$ points. Bitcoin changed its signature scheme once, by opt-in; Zcash has carried shielded payments through three proof systems, each with its own pool — the third with two, Orchard and Ironwood, on one circuit.

Ironwood Guide, "The Action circuit and the Halo 2 proof"; protocol specification, "Zero-Knowledge Proving Systems"

The Orchard circuit by the numbers

rows	$2^{11} = 2048$; one fixed circuit, every Action an instance
columns	ten advice, fixed columns, a Sinsemilla lookup table
proof per bundle	$2720 + 2272\,a$ B: 4992 for one Action, 7264 for Alice's two
of which IPA	22 pair points and one blinding commitment, 32 B each
proving	a few seconds on commodity hardware
verifying	$O(\log n)$ per proof, plus one MSM shared across a batch

One aggregate proof covers all of a bundle's Actions; the per-Action term is that Action's column commitments and evaluations, the constant term the shared quotient, multipoint, and IPA data. A spend-auth signature is 64 bytes; a one-Action proof is 78 signatures' worth. Bitcoin's witness cost is per input and grows with the script it satisfies; here it is per bundle, logarithmic in the circuit, and independent of the anonymity set.

Ironwood Guide, "The Action circuit and the Halo 2 proof"; "Ironwood component"; *Halo 2 Guide*, "The complete Halo 2 protocol"

Five gadget families, three circuit versions

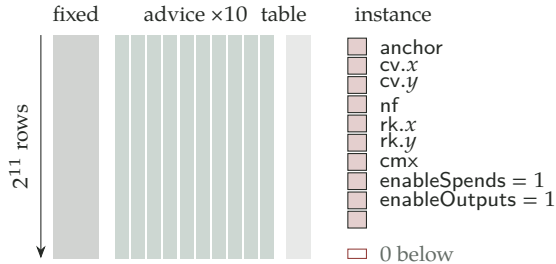
- *ECC*: the Pallas group law — $\mathsf{rk} = \mathsf{ak} + [\alpha]\, G^{\mathsf{SpendAuth}}$ (A6), $\mathsf{pk_d} = [\mathsf{ivk}]\, \mathsf{g_d}$ (A7), cv (A4), the nullifier's point (A5).
- *Sinsemilla*: the generator table as a lookup; each 10-bit chunk costs one lookup and two incomplete additions; carries NoteCommit (A1, A2) and $\mathsf{Commit}^{\mathsf{ivk}}$ (A7).
- *Merkle path*: 32 layer-tagged hashes, a conditional swap per layer by one bit of the position, root = anchor gated by v_{old} (A3).
- *Poseidon*: the nullifier PRF; its only non-linearity is $x \mapsto x^5$ (A5).
- *Decomposition*: range checks $v \in [0, 2^{64})$ and the sign bit (A4); canonicity of witnessed point and field-element encodings — distinct from the node's parsing of public encodings (Part E).

Three circuit versions, one selected verifying key

Three Action-circuit versions exist — the original, a correction, the Ironwood circuit with the cross-address input — and consensus selects the verifying key by branch, so every Action in either pool now verifies against the Ironwood key. Bitcoin's validator runs whatever script an output declares; this one runs a verifier keyed to one fixed circuit.

Ironwood Guide, "The Action circuit and the Halo 2 proof"; "Chain state and pool rules"

Alice's Action as an instance



Instance (Action 1: 2 ZEC → Bob 1.5): eight public inputs, ten field elements; the two points enter by coordinates; the flags as the Ironwood component sets them.

Advice: both notes; 32 siblings and the position; ak, nk, r_{ivk}, ivk; α; rcv; $(\text{magnitude}, \text{sign}) = (50\,000\,000, +1)$.

Copy constraints tie each instance cell to the advice cell holding the same value; the verifier computes the instance commitment itself, so a proof for other values fails the opening.

Bitcoin's witness (scriptSig or witness field) is in the clear; here the witness fills advice cells nobody reads.

Ironwood Guide, "The Action statement"; *Halo 2 Guide*, "Instance columns and binding the public statement"

Checkpoint

What could a holder of the Sapling ceremony's toxic waste do, what stops the analogue in Orchard, and what bounds the damage?

Checkpoint

What could a holder of the Sapling ceremony's toxic waste do, what stops the analogue in Orchard, and what bounds the damage?

- Forge accepting Sapling proofs of false statements: counterfeit inside the Sapling pool, invisible to every verifier. Not decrypt notes — soundness and zero knowledge are separate guarantees.
- In Orchard there is no trapdoor to hold: the generators are hashed, and binding reduces to discrete log.
- Under ZIP-209 each pool's chain value balance is public, and a block that would drive it negative is invalid — counterfeit is capped at the pool's exit.

Crypto Guide, "SNARKs: succinct non-interactive arguments of knowledge"; *Ironwood Guide*, "Chain value pool balances"

Part G

Delivery and light clients

The note plaintext: what travels to Bob

The chain stores cmx; the note itself must still reach Bob. What Alice sends him is the *note plaintext*

$$\mathsf{np} := (\mathtt{0x03},\ d,\ v,\ \mathsf{rseed},\ \mathsf{memo}), \qquad 1 + 11 + 8 + 32 + 512 = 564 \text{ bytes.}$$

- The lead byte names the format: 0x03 is the Ironwood note, the only value the Ironwood pool admits.
- The 32-byte seed rseed is the note's whole randomness. Three PRFexpand calls keyed by it, each under its own domain byte, yield esk $(\mathtt{0x04} \,\|\, \rho)$, ψ $(\mathtt{0x09} \,\|\, \rho)$, and rcm (next slide); with $\rho := \mathsf{nf}_{\mathrm{old}}$, the seed is bound to its Action.
- The memo is 512 bytes of sender-chosen data, encrypted with the rest; it never appears in the clear.
- *Bitcoin contrast:* an output shows its amount and script to everyone, and an OP_RETURN note is public.

Ironwood Guide, "The note plaintext"; "The note seed: Ironwood derivations (lead byte 0x03)"; protocol specification, "Note Plaintexts and Memo Fields"

The Ironwood trapdoor: one hash over every note field

The one note-level difference between the pools: how rcm leaves the seed.

$$\begin{aligned}
\text{Ironwood pool } (\texttt{0x03}): \quad & \mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathsf{pre}_{\mathsf{rcm}})), \\
& \mathsf{pre}_{\mathsf{rcm}} := \texttt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi), \\
\text{Orchard pool } (\texttt{0x02}): \quad & \mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\texttt{0x05} \,\|\, \mathrm{LE}_{256}(\rho))).
\end{aligned}$$

- A $1 + 32 + 32 + 8 + 32 + 32 = 137$-byte preimage against 33: every note field enters the trapdoor, so rcm is derived last, after ψ. The commitment itself is unchanged: same NoteCommit, same tree construction — each pool its own instance (ZIP-229).
- Both pools' notes carry esk and ψ byte-identically; only the trapdoor differs, and the lead byte says which derivation applies.
- *Bitcoin contrast:* a Bitcoin output has no trapdoor at all — there is nothing hidden to bind.

Ironwood Guide, "The note seed: Ironwood derivations (lead byte 0x03)"; "Note seed and lead byte"

Why the detour through BLAKE2b: post-quantum binding

- The commitment's binding is Sinsemilla collision resistance — a discrete-log assumption, which a large quantum computer breaks (Part I). Once it falls, NoteCommit alone binds nothing.
- Model PRFexpand as a random oracle. Every field enters pre_{rcm}, so any change to a committed note re-randomises rcm; landing two honestly derived note–trapdoor pairs on one commitment is a collision event in the oracle, with no group structure for Shor.
- The composite of derivation and NoteCommit is therefore post-quantum binding — *provided a verifier checks the derivation*. No deployed Action clause does; the claim is conditional on that future check.
- This is what makes an Ironwood note *recoverable*, and no Orchard- or Sapling-pool note. Parts H and I return to it.
- *Bitcoin contrast:* nothing in a Bitcoin output is hidden, so nothing must stay bound after a break; here the hidden note must.

Ironwood Guide, "The note seed: Ironwood derivations (lead byte `0x03`)"; "Purpose of the hashed trapdoor"

Sealing the note: one-shot Diffie–Hellman, then an AEAD

Alice holds Bob's address $(d, \mathsf{pk_d})$, where $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and $\mathsf{g_d} = \mathsf{GroupHash}(d)$. With esk drawn from rseed she runs a key agreement to the diversified base:

$$\mathsf{epk} := [\mathsf{esk}]\,\mathsf{g_d}, \qquad \mathsf{s} := [\mathsf{esk}]\,\mathsf{pk_d}, \qquad K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s},\ \mathsf{epk}),$$
$$C_{\mathrm{enc}} := \mathsf{Enc}_{K_{\mathrm{enc}}}(\mathsf{np}) \qquad (564 + 16\text{-byte tag} = 580\ \text{bytes}).$$

- The KDF is BLAKE2b-256 over the encodings of s and epk; the AEAD is ChaCha20-Poly1305 under a fixed all-zero nonce and empty associated data — a fresh key for every note.
- Why Bob: $[\mathsf{ivk}]\,\mathsf{epk} = [\mathsf{ivk}][\mathsf{esk}]\,\mathsf{g_d} = [\mathsf{esk}]\,\mathsf{pk_d} = \mathsf{s}$, so the holder of ivk — and the sender, who knows esk — recovers K_{enc} from the published pair $(\mathsf{epk}, C_{\mathrm{enc}})$.
- *Bitcoin contrast:* BIP-352 runs the same ECDH with the input keys standing in for esk — no epk is published — to derive an output key the chain then shows in the clear; here the shared secret encrypts the note, amount and memo included.

Ironwood Guide, "Encryption to the recipient"; protocol specification, "In-band secret distribution (Sapling and Orchard)"

The second ciphertext: the sender's own copy

One ciphertext serves Bob; a second can serve Alice's books. Given an outgoing viewing key, the pair $(\mathsf{pk_d}, \mathsf{esk})$ is sealed under the *outgoing cipher key*:

$$\mathsf{ock} := \text{BLAKE2b-256}(\mathsf{ovk} \,\|\, \mathsf{cv} \,\|\, \mathsf{cmx} \,\|\, \mathsf{epk}), \qquad C_{\text{out}} := \mathsf{Enc}_{\mathsf{ock}}(\mathsf{pk_d},\ \mathsf{esk}).$$

- From $(\mathsf{pk_d}, \mathsf{esk})$ the holder of ovk re-derives $\mathsf{s} = [\mathsf{esk}]\,\mathsf{pk_d}$ and reads the note as Bob does: $64 + 16 = 80$ bytes buy a sent history.
- Binding cv, cmx, epk into the key pins C_{out} to its own Action; with $\mathsf{ovk} = \perp$ the sender seals random bytes under a random key instead, and no outgoing key ever opens it.
- The Action publishes $(\mathsf{epk}, C_{\text{enc}}, C_{\text{out}})$ beside cv, nf, rk, cmx; the node never opens the three, and no clause of the Action statement mentions them.
- *Bitcoin contrast:* a Bitcoin sender needs no copy — the chain is the copy; here a wallet's sent history exists only if it kept a key that reads it.

Ironwood Guide, "The outgoing ciphertext"; "What the statement does not check"; protocol specification, "Decryption using an Outgoing Viewing Key (Sapling and Orchard)"

Finding your money: detection is successful decryption

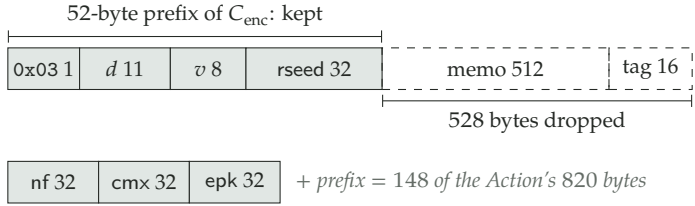
Nothing on chain names a recipient, so Bob's wallet trial-decrypts *every* Orchard-protocol output with ivk:

$$\mathsf{s} := [\mathsf{ivk}]\,\mathsf{epk}, \qquad K_{\mathrm{enc}} := \mathsf{KDF}(\mathsf{s},\ \mathsf{epk}), \qquad \mathsf{np} := \mathsf{Dec}_{K_{\mathrm{enc}}}(C_{\mathrm{enc}}).$$

- The AEAD tag rejects a wrong key; an open is a *candidate*, not yet a note.
- Two re-derivations close the obvious forgeries: esk from rseed must give $[\mathsf{esk}]\,\mathsf{g_d} = \mathsf{epk}$, and the note rebuilt from (d, v, rseed) and the Action's own $\rho := \mathsf{nf}_{\mathrm{old}}$ must give $\mathsf{Extract}(\mathsf{NoteCommit}_{\mathsf{rcm}}(\ldots)) = \mathsf{cmx}$.
- The cost: one key agreement per output for each ivk, plus the bandwidth of every output. One ivk scans both pools.
- *Bitcoin contrast:* a Bitcoin wallet asks for its addresses' outputs, a lookup over public scripts; here no field on chain names the recipient, so there is nothing to ask for.

Ironwood Guide, "Trial decryption and note acceptance"; *Wallet Guide*, "Compact blocks (ZIP-307)"; protocol specification, "Decryption using an Incoming Viewing Key (Sapling and Orchard)"

Compact blocks (ZIP-307): what a light client downloads



- A compact block keeps only what three tasks need: detect a payment, detect a spend of one's own notes, update witnesses.
- Per Action: nf, cmx, epk, and the prefix. Dropped: memo, tag, C_{out}, cv, rk, anchor, proof, signatures. Each block also states its end-of-block tree size per pool.
- *Bitcoin contrast:* an SPV client downloads headers and asks for what matches; this client downloads every compact block in range and asks for no particular output.

Wallet Guide, "Compact blocks (ZIP-307)"; "The compact block and the compact transaction"; "Compact outputs, spends, and Actions"; *Ironwood Guide*, "Trial decryption and note acceptance"

Accepting a note without its tag

Truncation at 52 bytes discards the Poly1305 tag, so a compact decryption cannot be authenticated the AEAD way:

$$\mathsf{np} := \mathsf{ciphertext}[0..52) \;\oplus\; \mathsf{ChaCha20}_{K_{\mathrm{enc}}}(\mathrm{nonce} = 0)[64..116)$$

— the bare keystream, past block 0, which the full AEAD reserves for the tag key. The wallet accepts iff

1. the 52 bytes parse as a compact note plaintext;
2. the recomputed commitment equals the served cmx — Bob holds neither cv nor the binding signature, so commitment binding is what he relies on;
3. the esk re-derived from rseed reproduces epk.

The commitment stands in for the tag: under binding, no sender can make Bob accept a note the chain did not commit to.

– *Bitcoin contrast:* BIP-37 hands each peer a Bloom filter of the wallet's scripts; BIP-157 matches a per-block filter locally, then fetches the matching blocks. Here the "filter" is ivk, applied to every block, and no match is ever requested.

Wallet Guide, "Compact trial decryption"; *Ironwood Guide*, "Acceptance of an Ironwood-pool output"

Leaks, in both directions

- **At the scan stage, less.** Every compact block in a range is fetched whether or not anything inside opens; the server sees range bounds and timing, never a per-block match. A BIP-157 client reveals each block it fetches after a filter hit.
- **After detection, more.** The memo and C_{out} live in the full transaction, which compact blocks omit; fetching it names an exact txid, where a BIP-157 client names a whole block.
- ZIP-307 attaches three requirements to that fetch: obscure interest with uninteresting transactions (SHOULD), fetch every transaction of any block touched (SHOULD), tell the user (MUST). The deployed client and server libraries ship no decoy policy.
- Structural, not accidental: a client that asks for ranges reveals where it stopped; one that fetches what it detected reveals what it detected.

Wallet Guide, "Leakage of transaction queries"; "Leakage of range queries"

Placing the note: a position from public data

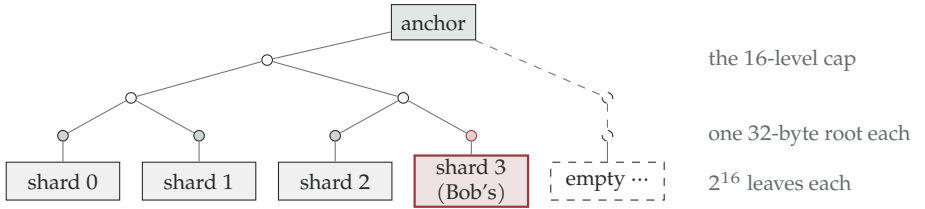
Trial decryption tells Bob *that* an output is his; the tree needs to know *where*.

$$\text{pos}(\text{output } i \text{ of } tx) \;=\; \text{start-of-block size} \;+\; \text{offset}(tx) \;+\; i$$

- Arithmetic on public data, never a replay: start size = the stated end-of-block size − the block's own outputs; offset counts the outputs of the block's earlier transactions.
- The wallet appends *every* cmx — Bob's and everyone else's — and marks its own; a note without a position has no witness, so the size is load-bearing, not a convenience.
- A spend of Bob's note is detected the same way: every compact nullifier is compared against the wallet's own unspent nullifiers, locally.
- *Bitcoin contrast:* an SPV client's branch arrives because its filter (BIP-37), its block fetch (BIP-157), or — on Electrum — the txid itself named its interest. Bob names nothing; his position is his own arithmetic.

Wallet Guide, "Note positions and tree sizes"; *Ironwood Guide*, "Authentication paths from public data"

Shards and the cap: a witness without the whole tree



- Cut the depth-32 tree at half depth: shards of 2^{16} leaves, at most 2^{16} of them, under a 16-level cap of shard roots.
- Bob hashes his own shard from its leaves and takes every completed shard as one root from the server; the unfinished tip shard, like his own, is scanned from leaves. Low 16 siblings from leaves, high 16 from roots; an unbegun shard is a per-level constant.
- *Bitcoin contrast:* an SPV client is handed one branch by its peer. Bob folds his own from public roots and asks for nothing — asking for one note's path would say which note is his. Only the anchor is ever public.

Ironwood Guide, "Authentication paths"; *Wallet Guide*, "Subtree roots and the cap"

Checkpoints: the tree as of a block

- A *checkpoint* is the wallet's record of its tree's state at a block boundary — the position of the last leaf as of that block, plus the marks dropped since the previous one — identified by the block's height.
- Two uses: fold a path against that block's root, so a witness can target any checkpointed block; and truncate the tree back to it when a reorganisation strikes.
- *Shared* means present in every pool's tree: when one tree gains a checkpoint at a height, the others gain one too, so a single height names a state of every tree.
- A bounded recent set is kept, plus durable ones exempt from pruning; a block that appends nothing need not create one.
- *Bitcoin contrast:* Bitcoin Core's checkpoints are hard-coded block hashes inside the node; these are wallet-local tree snapshots that consensus never sees.

Ironwood Guide, "Anchors"; *Wallet Guide*, "The information a wallet retains"; "Reorganisation detection and recovery"

The frontier as the scan seed

A *frontier* is the rightmost path of a tree of size n: the last leaf plus one sibling hash per filled subtree on its way to the root — enough to keep appending, and enough to seed a shard tree.

- A wallet is born from the frontier of the block *before* its birthday: that frontier summarises every earlier commitment, so no block below the birthday is ever replayed.
- Every scanned batch is seeded with the chain state of the block before it, checked against the first block's own end-of-block size before any tree is touched; a contradiction rejects the batch as mis-sequenced. A size the block's own contents contradict is the continuity error — the reorganisation path.
- A found note is tree-ready once its own shard's block extent and the tip range are scanned — not the whole history.
- *Bitcoin contrast:* a restored Bitcoin wallet also rescans from its birthday, but needs no tree state: the UTXO set is public, and a peer answers a filter, or an Electrum server answers by address.

Wallet Guide, "Frontiers as scan seeds"; "The account birthday"; "Witness readiness"

What the server learns: the schedule

Keys never travel; requests do. ZIP-307's stated goal is payment-detection privacy against an honest-but-curious proxy; spending is outside its scope.

observable	reveals	mitigation status
peer IP, connection timing	endpoint, activity windows	Tor capability; deployment unknown
lowest range start	approximate account birthday	none
range boundaries	scan state and progress	bucketing proposed, unimplemented

Bitcoin contrast: a BIP-157 client shows a peer the same kind of schedule — which filter ranges it pulls, and when — and then every block it fetches on top. Here the content holds and the schedule leaks.

Wallet Guide, "Payment-detection privacy"; "Key independence of queries"; "Leakage of range queries"; "Summary of server observables"

What the server learns: the content channels

Detection stays on the device; three kinds of request carry content anyway, each sent only after the wallet itself decided to.

observable	reveals	mitigation status
fetch after detection	transactions of interest	decoys a SHOULD; no library policy
submission via the server	the whole transaction, linked to origin	circuit isolation; deployment unknown
transparent-address calls	addresses outright	jitter (one-use addresses only)

Bitcoin contrast: a BIP-37 client leaks its addresses up to the false-positive rate, a BIP-157 client its matched blocks; here content leaks only through what the wallet sends after detecting.

Wallet Guide, "Key independence of queries"; "Leakage of transaction queries"; "The submission channel"; "Summary of server observables"

Mitigations that ship, and the structural reading

- The baseline holds: across the whole service surface no request carries key material — only heights, hashes, txids, transparent addresses, and raw transactions. Trial decryption and nullifier matching run on the device.
- Transport: a Tor client with circuit isolation exists in the wallet library, so submission need not share a circuit with scanning; whether any deployed wallet routes through it by default is not determinable from the sources.
- Timing: exponentially distributed intervals in a shuffled order — only for the checks on *ephemeral* addresses, the one-use transparent addresses of ZIP-320 payments (Part H). No jitter exists for the shielded fetch-on-detect.
- Bucketing — rounding a range start down to a multiple of N — is ZIP-307's own proposal, unimplemented.
- The reading: none of this is a property of compact-block contents, and none of it is inevitable; Tor, bucketing, jitter, and cover traffic can each change it.

Wallet Guide, "Key independence of queries"; "Leakage under bucketing"; "Summary of server observables"

Spending from a light client: the witness and the anchor

- Witness: Bob's 32 siblings are folded on the device, from his shard's leaves and the other shards' roots, against a checkpointed block; the path is the private input of the membership clause.
- Anchor, recalled from Part C: any main-chain block's root since the pool's activation is admissible, no window, so proving is decoupled from broadcasting; appending alters no old root; an older anchor is no soundness hole, and a second spend is the nullifier's problem.
- *Bitcoin contrast:* a Bitcoin input names one txid and index, and the validator looks it up. The anchor names a tree; the path into it never leaves the device.

Ironwood Guide, "Old anchors"; "Authentication paths"

Which anchor: the wallet's policy, and whose reasons

- Anchor: the newest shared checkpoint at or below $H - 3$, with H the target height, one above the tip — wallet policy, not consensus; ZIP-315's stated reasons and the Guide's anonymity-set argument are Part C's.
- Spendable under the same policy: 3 confirmations for trusted notes, 10 for untrusted, counted from the mined block; the note's shard fully scanned; and its block at or below the anchor.
- *Bitcoin contrast:* a Bitcoin wallet also waits for confirmations, but which block it "anchors" to is nobody's business, because the input is named either way.

Ironwood Guide, "Recommended anchor depth"; *Wallet Guide*, "The confirmation policy"; "Spendable notes"

Expiry and broadcast

- Expiry (ZIP-203): every transaction carries an expiry height, zero meaning none; a block that includes it above that height is invalid. ZIP-218 recommends target +120; a uniform delta, like a uniform fee, is not a fingerprint.
- Store, then broadcast: the wallet persists the transaction and locks its inputs before the network sees it, then submits it in one call. Once expired unmined, it releases its inputs: the notes are spendable again.
- *Bitcoin contrast:* a stuck transaction has no consensus expiry — mempools drop it by policy after a timeout, or RBF or CPFP bumps it. Here it expires by rule, and its inputs come free.

Wallet Guide, "Expiry (ZIP-203)"; "Submission and the transaction identifier"; "The transaction record and locked notes"

The reorganisation case

- A reorganisation past the anchoring block invalidates the anchor: the wallet truncates to a checkpoint, rescans, and rebuilds.
- The rebuilt transaction re-reveals the same nullifiers, because their derivation is deterministic: the notes are the same, so the serial numbers are.
- Re-anchoring with the txid unchanged is specified (ZIP-229, anchors in authorizing data); wallet support is pending.
- *Bitcoin contrast:* a reorganised Bitcoin transaction is simply mined again, since it names its inputs and no anchor. Here a struck anchor means a rebuild.

Wallet Guide, "Reorganisation detection and recovery"; "PCZT v2: deferred anchors and the Ironwood bundle"

Three openings on Alice's transaction

ciphertext	opened by	with	yields
Action 1, C_{enc}	Bob	ivk	$(d,$ 1.5 ZEC, rseed$)$; the memo, once fetched
Action 2, C_{enc}	Alice	ivk^{int}	change 0.4999 ZEC, at her internal address
Action 1, C_{out}	Alice	ovk	$(\text{pk}_\text{d}, \text{esk})$, hence Bob's note as he reads it
Action 2, C_{out}	Alice	internal ovk	$(\text{pk}_\text{d}, \text{esk})$, hence her change (ZIP-316)

- Bob: cmx and epk check, position recorded; with nk a nullifier absent from the set. From a stranger: untrusted, 10 confirmations.
- Alice finds her change by trial decryption under the internal scope, never through an outgoing key; internal notes are trusted: 3 confirmations.
- Alice re-reads what she sent Bob through C_{out} — her sent history, and an auditor's, given ovk.
- *Bitcoin contrast:* Bob's payment and Alice's change are the two outputs everyone reads, and a memo would be an OP_RETURN.

Ironwood Guide, "Trial decryption and note acceptance"; "The outgoing ciphertext"; *Wallet Guide*, "Internal keys: change and auto-shielding"; "Outgoing viewing keys"; "The confirmation policy"

Checkpoint

Bob's light client holds ivk and the 52-byte prefixes: what can the server still infer, and what would a Bitcoin SPV client have given away instead?

Checkpoint

Bob's light client holds ivk and the 52-byte prefixes: what can the server still infer, and what would a Bitcoin SPV client have given away instead?

- Still inferred: the endpoint and when it is active; the earliest range start, about his birthday; the range bounds, his progress; the exact txid, if he fetches the full transaction for the memo; the whole transaction before broadcast, if he submits through the same server.
- Never: which outputs opened, their values, or ivk itself.
- A BIP-37 client hands each peer a Bloom filter of its scripts: addresses, balance, and graph, up to the false-positive rate. A BIP-157 client narrows that to the blocks it fetches after a match — still by asking.

Wallet Guide, "Key independence of queries"; "Summary of server observables"; "The submission channel"

Part H

The transaction and the turnstile

A v6 transaction beside a Bitcoin transaction

	Bitcoin	Zcash v6
header	version, lock time	+ version group, branch id, expiry height: 20 B in all
inputs	outpoint, scriptSig, sequence	the same bytes; ECDSA over secp256k1, no Taproot; the scriptSig outside the txid
outputs	value, scriptPubKey	the same bytes
Sapling bundle	—	spends, outputs, Groth16 proofs
Orchard bundle	—	Actions of the sealed pool
Ironwood component	—	a Actions at 820 B; flags, $v^{\text{Ironwood}}_{\text{balance}}$, anchor; one proof of $2720 + 2272a$ B; $a + 1$ signatures at 64 B

One ledger, one transaction type, one asset: the shielded parts are components beside the transparent ones. Alice's payment fills the last component alone: $a = 2$ Actions, 7264 proof bytes, 9166 bytes in all.

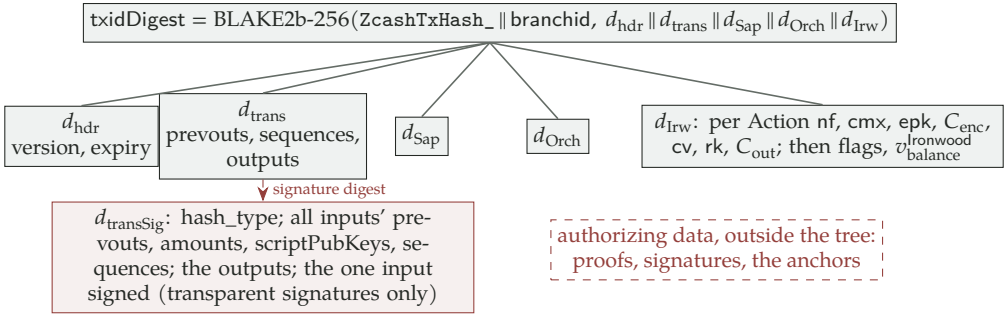
Ironwood Guide, "Version-6 transaction"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Effecting data, authorizing data: the SegWit shape

- **Effecting data** is everything under the txid: header, transparent outpoints, sequences and outputs (each scriptSig is authorizing data: SegWit's move, applied to the legacy input), every Action's public fields, the flags, the balancing fields. **Authorizing data** is what attests to it: proofs and signatures — and, from v6, the three anchors.
- Bitcoin's txid is SHA-256d of the bytes, and SegWit moved the witness out of them, re-committed through the coinbase. A v6 txid is a BLAKE2b-256 tree over the effecting data alone, and ZIP-244 re-commits the authorizing data through `hashAuthDataRoot`, folded into the header's `hashBlockCommitments`. Recomputing a proof or a signature changes nothing the identifier names.
- Because the anchor is authorizing data, a v6 transaction can be *re-anchored* — its membership proof rebased onto a later root — with its txid unchanged: specified (ZIP-229), wallet support pending.
- NU7 permits v5 and v6, not v4 (draft ZIP-259). The Orchard-pool seal applies independently of the version.

Ironwood Guide, "Transaction digests and signatures"; "The transaction format"; *Crypto Guide*, "The transparent layer's hashes"; *FlyClient Guide*, "The authorising-data root"; *Wallet Guide*, "PCZT v2: deferred anchors and the Ironwood bundle"

The sighash (ZIP-244): a tree of digests



- Every node is one BLAKE2b-256 under its own personalisation; the root's carries the branch id, so the same bytes have a different identity on a parallel chain.
- The signature digest is this tree with d_{trans} swapped for $d_{transSig}$. A transaction with no transparent inputs has nothing to swap: its signature digest *is* its txid.

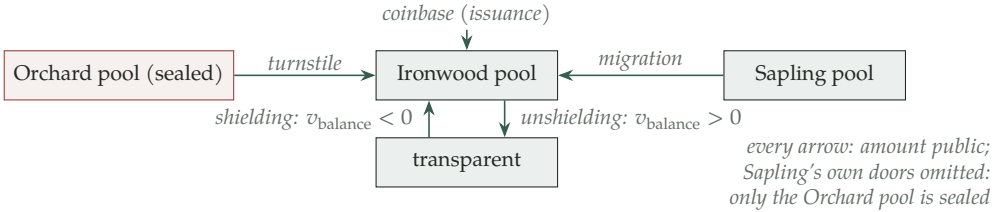
Ironwood Guide, "Transaction digests and signatures"; *Wallet Guide*, "Submission and the transaction identifier"

What the shielded signatures sign

- Each Action's spend-authorisation signature under rk and the bundle's binding signature under bvk sign the signature digest: every Action's nf, cmx, cv, rk and ciphertexts, the flags, v_{balance}. No Action can be transplanted into another transaction, no field altered.
- Bitcoin lets each signer pick a `hash_type`: ALL, NONE, SINGLE, each with or without ANYONECANPAY — six meaningful values (Taproot adds SIGHASH_DEFAULT as a seventh spelling of ALL, which ZIP-244 does not define) — and a transparent Zcash input is restricted by consensus to exactly those six. A shielded signature has one: SIGHASH_ALL, always. There is no shielded ANYONECANPAY.
- The anchor is not under the signature from v6. It does not need to be: the proof carries it as a public input (A3), so the signature attests to the spend and the proof pins the tree.
- Alice's transaction has no transparent inputs: her two spend-authorisation signatures and her binding signature all sign her txid.

Ironwood Guide, "Transaction digests and signatures"; "Bundle binding"; *Wallet Guide*, "Signing"; "Proving"

Pools and the doors



- **Shielding.** Transparent inputs fund a negative $v_{\text{balance}}^{\text{Ironwood}}$; the Actions spend dummies and create real notes. Alice shields 10 ZEC: $v_{\text{balance}}^{\text{Ironwood}} = -9.99985$ ZEC is printed, her new note is not. Wallets shield automatically, to an internal address.
- **Coinbase.** Issuance enters a shielded pool directly, but every shielded coinbase output must decrypt under the all-zero ovk: anyone can read what issuance entered which pool.
- **Migration.** Sapling or the sealed Orchard pool to Ironwood, the latter through the turnstile; both balancing fields are cleartext. Bitcoin has one pool and needs no doors.

Ironwood Guide, "The balancing value"; "Chain state and pool rules"

The turnstile (ZIP-209): audited at the boundary

$$\mathsf{balance}_P \;:=\; -\sum_{\text{tx}} v^P_{\text{balance}}(\text{tx}) \;\geq\; 0 \quad \text{in every block, for } P \in \{\mathsf{Sap}, \mathsf{Orch}, \mathsf{Irw}\}$$

- Every pool has a public balance under this rule: Sprout's is $\sum$ `vpub_old` $-\sum$ `vpub_new`, the transparent pool's the UTXO sum, the lockbox's its accrued subsidy. A block driving any negative, or the total past MAX_MONEY, is invalid.
- *Inside* a pool a counterfeit is undetectable: no check sums the notes. Sprout's original proof system had exactly such a flaw (Part G); a forged proof mints silently.
- *At the exit* the damage is capped: a forger can withdraw at most what the pool publicly holds. The Sapling pool still rests on its Groth16 ceremony; the turnstile bounds it.
- Had the Ironwood pool held only Alice's deposit (9.99985 ZEC), Bob's +9.99985 ZEC exit would empty it exactly; one zatoshi more and his block is invalid. Bitcoin audits by summing every coin; Zcash audits each pool at its door.

Ironwood Guide, "Chain value pool balances"; "Outflow bound"; ZIP-209; protocol specification, "Chain Value Pool Balances"; "BCTV14"

Why the Orchard pool is sealed

- The seal is the three rules of Parts C and E — no issuance, no circulation ($\mathsf{enableCrossAddress} = 0$), no entry ($v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$) — binding whatever the transaction version.
- The reason is Part F's one note-level difference: a legacy note (lead byte `0x02`) is not quantum-recoverable; an Ironwood note (lead byte `0x03`, hashed `rcm`) is. So consensus stops the old notes circulating and funnels their value out through the turnstile, while owners can still spend what they hold.
- The turnstile is the seal's accounting shape: the Orchard pool's balance can only fall, every exit metered in cleartext by $v^{\mathsf{Orchard}}_{\mathsf{balance}}$, so the legacy pool drains through a public gate.
- Bitcoin never retires an output type; a P2PK coin spends forever. Here a pool can be sealed by three rules and drained through a metered gate.

Ironwood Guide, "Chain state and pool rules"; "The Action circuit and the Halo 2 proof"; "Purpose of the hashed trapdoor"; "Outflow bound"

Fees (ZIP-317): a function of the public shape

$$fee = 5000 \cdot \max(2,\ logical_actions) \quad \text{zatoshi}$$

$$logical_actions = \max\left(\left\lceil \frac{\text{in bytes}}{150} \right\rceil, \left\lceil \frac{\text{out bytes}}{34} \right\rceil\right) + 2\, nJoinSplit_{\text{Sprout}} + \max(spends, outputs)_{\text{Sap}} + nActions_{\text{Orch}} + nActions_{\text{Irw}}$$

- The divisors are the standard P2PKH input (150 B) and output (34 B): one typical transparent input or output weighs one action, and a fatter script pays in proportion.
- An Action spends one note *and* creates one, so it is counted once — charging inputs and outputs separately would tax the padding.
- Alice: two Actions, $5000 \cdot \max(2, 2) = 10\,000$ zatoshi. A one-Action payment padded to two pays the same. A shielding transaction with one P2PKH input: $1 + 2 = 3$ actions, 15 000.

Wallet Guide, "The conventional fee"; "The fee on padded counts"

Fees (ZIP-317): why a convention, and who enforces it

- **Computable from public fields alone.** A fee that depended on which notes were spent or what change came back would print that data in the amount paid.
- **Uniform across wallets.** Every wallet is expected to pay exactly this fee, so the fee is not a fingerprint. Padding to two Actions is free by design: the grace window is two.
- **A convention, not consensus.** Nodes relay transactions paying at least it; the recommended block template picks among those by fee ratio, capped at 4×, and admits at most 50 unpaid actions per block. Overpaying beyond 4× buys no further priority.
- **Bitcoin** runs a market: the bid per virtual byte, and the coin selection behind it, vary by wallet and moment — and both are read as fingerprints. Zcash flattens the market on purpose.

Wallet Guide, "The conventional fee"; "Relay, mempool, and block template"

The public face of Alice's transaction

public field	Alice pays Bob 1.5 ZEC from a 2 ZEC note
version	v6; transparent, Sapling and Orchard parts empty
Ironwood Actions	2 (one real spend, one dummy; shuffled)
nullifiers, commitments	2 and 2
$v_{\text{balance}}^{\text{Ironwood}}$	+10 000 zatoshi = the fee
anchor	the Ironwood root at the newest shared checkpoint at or below $H - 3$
expiry	target height + 120 blocks, about 50 minutes (ZIP-218)
signatures	2 spend-authorisation + 1 binding
size	9166 bytes, 7264 of them proof

Bitcoin prints the same payment as the input coin, both amounts, both scripts and, by subtraction, the fee. Here the fee alone is printed, as $v_{\text{balance}}^{\text{Ironwood}}$; the 2 ZEC, the 1.5 ZEC, the 0.4999 ZEC change and both addresses stay inside the commitments.

Ironwood Guide, "The transaction format"; "Recommended anchor depth"; "Disclosure by the balancing value"; *Wallet Guide*, "Padding, dummies, and shuffling"; "Expiry (ZIP-203)"

The privacy sliders: pool, address type, timing

- **Pool.** A transparent leg prints its amount as v_{balance}; fully shielded prints the fee and nothing else. The slider is the sender's, set by which funds and which receiver she uses.
- **Address type.** A unified address bundles receivers; the sender's wallet MUST use the most preferred one it supports: Orchard-protocol (into the Ironwood pool) over Sapling over transparent. A TEX address demands transparent funds, so the wallet builds two transactions through an ephemeral transparent output — both public.
- **Timing.** ZIP-218 recommends expiry at target + 120 blocks: an idiosyncratic delta fingerprints a transaction as an idiosyncratic fee would. Migration wallets split value into canonical denominations $m \cdot 10^k$ ZEC, $m \in \{1, 2, 5\}$, and draw anchors and expiries from shared grids for the same reason.
- Bitcoin has the second and third sliders; it has no pool slider, because it has one plaintext ledger. Uniformity is the strategy: the defaults protect only while everyone keeps them.

Wallet Guide, "Unified addresses"; "Input selection"; "Expiry (ZIP-203)"; ZIP-318, "Amount selection, canonical quantization"; "Anchor-height bucketing and cohorts"

PSBT → PCZT (ZIP-374): the unfinished transaction

- Bitcoin passes an unfinished transaction between roles — PSBT (BIP-174, plus BIP-370's Constructor): Creator, Constructor, Updater, Signer, Combiner, Input Finalizer, Transaction Extractor — and hardware wallets and multisig live on it.
- A PSBT cannot carry a shielded transaction: no representation for shielded spends and outputs, no notion of a proof distinct from a signature, and it tolerates unknown fields — unacceptable for a signer that must understand everything it authorises.
- A PCZT keeps the PSBT role structure and adds Zcash's own — among them an **IO Finalizer** (declares the inputs and outputs complete, derives the binding-signature key), a **Redactor** (strips what later parties need not see), and a **Prover** — ten roles in all.
- The Prover has no PSBT counterpart because a hardware signer cannot compute a Halo 2 proof, and a proof needs no spending key: proving is delegated, signing is not. Signers can be fully offline; the format carries everything they need.

Wallet Guide, "Partially created transactions (PCZT)"; "Roles and the modification state"

Signer versus Prover

	Signer	Prover
holds	ask — spend authority	fvk — no spending key
reads from the PCZT	α, rk; the spend's public fields and, unless redacted, its note fields	the note $(\mathsf{g_d}, \mathsf{pk_d}, v, \rho, \mathsf{rseed})$, the witness, α, rcv, the anchor
computes	the sighash from the effecting data; $\mathsf{rsk} = \mathsf{ask} + \alpha$, checks it matches rk, signs	one Halo 2 proof over all Actions; anchor and flags as public inputs
verifies first	nf, cv, cmx against the note fields: exactly what it authorises	fvk owns each note; $\rho_{\text{new}} = \mathsf{nf}_{\text{old}}$
learns	what it signs	the notes' private data, not authority

Either role can run without the other; the Prover never sees ask; the Signer may see the note data, or have it redacted.

Wallet Guide, "Signing"; "Proving"

PCZT: where it stands

- From v6 the Signer and the Prover are independent even of the anchor: signatures commit to no anchor, so a wallet can sign today and anchor, witness and prove at broadcast.
- Delegating the Prover reveals the notes' private data to the delegate, never spend authority; a Redactor strips what a later party need not see.
- Bins: the ZIP is Draft; the format is released as a library; the deployed wallet pipeline drives the roles end to end — it builds the PCZT, hands it out for proving and signing, and extracts and stores the result (Ironwood sent-output records not yet stored: a backend gap, not a format limit).
- With FROST threshold signing (Part I) this closes the objection "no script, so no multisig and no hardware wallets": the roles carry both.

Wallet Guide, "Partially created transactions (PCZT)"; "Combining, redaction, and privacy"; "Roles and the modification state"; "PCZT v2: deferred anchors and the Ironwood bundle"

Checkpoint

Alice shields 10 ZEC; the next day Bob unshields 9.9997 ZEC.
What can a chain analyst conclude, and what can they not?

Checkpoint

Alice shields 10 ZEC; the next day Bob unshields 9.9997 ZEC.
What can a chain analyst conclude, and what can they not?

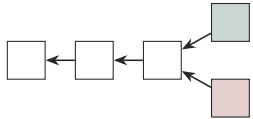
- **Conclude.** Two cleartext balancing fields, exact to the zatoshi: −9.99985 ZEC entered, +9.99985 ZEC left — 10 ZEC less two conventional fees of 15 000 zatoshi; the heights; the transparent addresses on each side; that the amounts match exactly and the times are close.
- **Not conclude.** That Bob's coins were Alice's. Her commitment hides her note; his nullifier is keyed by his `nk`; his candidate set is every leaf under his anchor. Her note may still be resting.
- Bitcoin would have handed over the path; Zcash hands over a correlation. Canonical denominations and shared timing grids exist because that correlation is evidence.

Ironwood Guide, "The balancing value"; "Anchor choice as public data"; "Nullifier unlinkability"; ZIP-318, "Amount selection, canonical quantization"; "Anchor-height bucketing and cohorts"

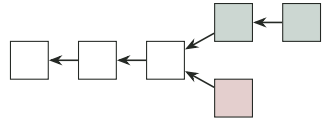
Part H′

Consensus: why one fork wins, and what the chain must remember

Bitcoin's chain rule, weighed in work



(a) equal work: nodes disagree



(b) one more block: the upper branch is heavier

- Every block names its parent, so blocks form a tree; each branch is one consistent history, and sibling branches may conflict.
- The work of a block is $\lfloor 2^{256} / (\texttt{ToTarget(nBits)} + 1) \rfloor$; the best valid chain has the greatest total work; ties go to the block received first.
- Bitcoin: the same rule. "Longest" equals "most work" only at constant difficulty, and nothing marks a chain as final — a heavier branch wins however deep the fork.

Consensus Guide, "Work, and the best chain"; protocol specification, "The Block Chain"; "Definition of Work"

Two double-spends; consensus answers one

- **Same chain.** Two spends of one note reveal the same nullifier, and a repeated nullifier is invalid by rule (Part C, and later in this part). No race, no probability.
- **Chain level.** Pay the merchant on the chain everyone sees, then replace that chain with one in which the payment never happened. No proof rules this out; only the proof-of-work race bounds it.

Bitcoin: the same two attacks; the first is a UTXO lookup there and a nullifier lookup here.

Consensus Guide, "Scope, sources, and the two double-spends"

The attack

As Meni Rosenfeld's *Analysis of hashrate-based double-spending* — the exact counterpart of the whitepaper's §11, followed throughout this part — lays it out:

1. broadcast the payment to the merchant;
2. mine a secret branch from the last pre-payment block carrying a conflicting payment to yourself;
3. wait until the merchant ships at n confirmations;
4. extend the secret branch until it carries more work, then release it — a well-formed chain, merely a different one.

Nothing in step 4 breaks a validity rule; the only question is probabilistic: does the secret branch ever get ahead?

Bitcoin: the attack the whitepaper's §11 analyses approximately; from here on its odds are computed exactly.

The model: memoryless clocks

- An exponential clock of rate λ has $\Pr[X > t] = e^{-\lambda t}$, and it is memoryless: $\Pr[X > s + t \mid X > s] = \Pr[X > t]$.
- Two clocks race: $\Pr[X < Y] = \lambda/(\lambda + \mu)$, and the winner is independent of the winning time.
- Honest network at rate p/T_0, attacker at q/T_0, $p + q = 1$, with T_0 the network's mean block time; both restart at every block. The next block is the attacker's with probability q, independently of everything before.

block attribution = an i.i.d. p/q coin sequence; T_0 drops out of every answer.

Bitcoin: the whitepaper's race in Rosenfeld's model, under the two assumptions he makes explicit — constant hashrates and constant difficulty. How well Equihash fits it is a deployment question (the rails, later in this part).

Consensus Guide, "Memoryless clocks and the attribution sequence"; "The catch-up probability"

The negative binomial law

The number m of attacker blocks found strictly before the honest network's n-th block:

$$P(m) \;=\; \binom{m+n-1}{m} p^n q^m, \qquad m = 0, 1, 2, \ldots$$

The $(m+n)$-th trial is the n-th honest success; the m attacker blocks sit among the first $m+n-1$ trials in any order.

First-to-n race. The honest side reaches n first (H_n) or the attacker does (A_n); counts advance one at a time, so there is no tie and $\Pr[H_n] + \Pr[A_n] = 1$.

At $q = 1/5$, $n = 6$: $P(0) = 0.262144$, $P(1) = 0.314573$, $P(2) = 0.220201$, …, $P(5) = 0.021139$; $\Pr[m \geq 6] = 0.011654$.

Bitcoin: the whitepaper approximates this law by a Poisson (the frame 'Satoshi's approximation', after the worked race).

The catch-up theorem (Rosenfeld §3)

Let z be the honest lead in blocks. Each block moves z up by 1 with probability p, down by 1 with probability q. The attack succeeds the moment z reaches -1: strictly ahead.

$$a_z \;=\; \Pr[\text{walk from } z \text{ ever reaches } -1] \;=\; \begin{cases} 1 & z < 0 \text{ or } q \geq p, \\ (q/p)^{\,z+1} & z \geq 0 \text{ and } q < p. \end{cases}$$

- Each block of deficit multiplies the chance by q/p: two forks cannot compete for long, and one quickly wins.
- At $q \geq p$ the attacker overtakes with probability 1 from any finite deficit; no confirmation count defends.
- Bitcoin: the whitepaper writes $(q/p)^z$ — it scores success at breakeven and banks no pre-mined block; the two conventions cancel term for term (shown with the confirmations).

Consensus Guide, "The catch-up probability"

Proof idea: a walk between two barriers

Absorb the walk at -1 and at $N > z$; let $h_N(z)$ be the chance of reaching -1 first. Conditioning on the first step,

$$h_N(z) = p\,h_N(z+1) + q\,h_N(z-1), \qquad h_N(-1) = 1, \quad h_N(N) = 0.$$

The trial solution t^z works exactly when t is a root of

$$p\,t^2 - t + q \;=\; p\,(t-1)\,(t - q/p),$$

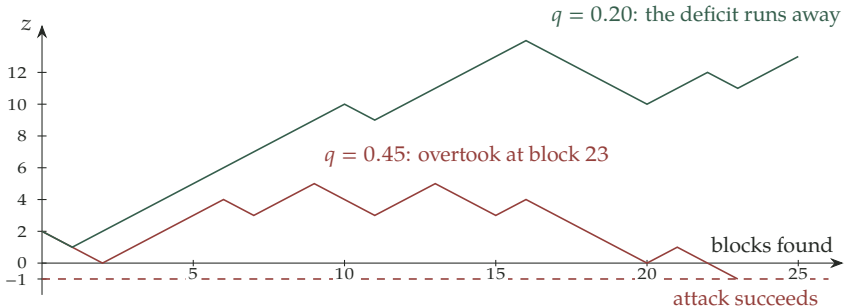
so for $q \neq p$

$$h_N(z) \;=\; \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N} \;\xrightarrow{N\to\infty}\; \begin{cases} (q/p)^{z+1} & q < p, \\ 1 & q > p, \end{cases}$$

and for $q = p$ the double root gives $h_N(z) = (N - z)/(N + 1) \to 1$. The limit is legitimate because the events "reach -1 before N" increase to "ever reach -1".

Consensus Guide, "The catch-up probability"

The deficit walk



Two seeded runs from $z = 2$. At $q = 0.45$ this run reaches -1 after 23 blocks; at $q = 0.20$ the deficit runs to 13, from which the chance of ever returning is $(1/4)^{14} \approx 3.7 \times 10^{-9}$. Decay anchors: at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$.

Consensus Guide, "The catch-up probability"

Waiting for confirmations: the pre-mined block

The merchant ships after n confirmations. Rosenfeld banks one attacker block before the race is scored, so with m further attacker blocks the walk starts at

$$z = n - m - 1.$$

Average the catch-up probability over the negative binomial m:

$$r(q,n) = \underbrace{\sum_{m=0}^{n-1} P(m)\,(q/p)^{\,n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m \geq n} P(m)}_{\text{already ahead}},$$

and $r = 1$ when $q \geq p$, since every catch-up factor is then 1. The convention is a modelling choice: with no banked block the walk starts at $n - m$ and every probability shrinks.

Bitcoin: this is the whitepaper's question, answered exactly rather than approximately (next slides).

Rosenfeld eq. (1): the closed forms

Each catch-up term transforms exactly,

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} = \binom{m+n-1}{m} p^m q^n,$$

the probability that the attacker's n-th block arrives while the honest side holds m; summed over $m \le n-1$ this is $\Pr[A_n]$, and so is the already-ahead tail. Hence

$$\begin{aligned} r(q,n) &= 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) \\ &= 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n = 2 \Pr[A_n]. \end{aligned}$$

Rosenfeld eq. (1): the race doubling

The race doubling. Under the one-pre-mined-block convention the catch-up half equals the already-ahead half exactly; the double-spend probability is twice the chance of winning a first-to-n race, and at $q = p$ each half is $\frac{1}{2}$, giving $r = 1$ continuously.

A full race, exactly: $q = 1/5$, $n = 6$

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(1/5, 6)$			0.023308

Every entry is an exact rational, printed to six decimals; the deficit starts at $z = 5 - m$ and the catch-up factor is $4^{-(6-m)}$.

The two halves agree to every digit, as the race doubling says they must; the closed form gives the same 0.023308.

A merchant who ships after six confirmations against a possible twenty-percent adversary carries a 2.33% strict-overtake risk.

Bitcoin: six confirmations is Bitcoin's folk number; here is what it buys, exactly.

Satoshi's approximation

The whitepaper computes the same quantity with one approximation and two offsetting conventions:

- the attacker's progress is taken as Poisson with mean $\lambda = nq/p$, $\Pr[k] = e^{-\lambda}\lambda^k/k!$;
- no block is pre-mined, but success is scored at breakeven rather than strictly ahead — the catch-up factor is again $(q/p)^{n-m}$, term for term; the conventions cancel and the Poisson step is the entire gap.

Satoshi's approximation: comparison with the exact risk

q	n	whitepaper	exact
10%	6	0.0243%	0.0591%
30%	6	13.211%	15.645%
10%	10	0.0001%	0.0008%

In each pair the approximation understates the risk; the direction is not uniform over all (q, n). The reason to prefer the negative binomial is that it is exact under the stated assumptions.

Blocks, not time

The catch-up formula contains neither T_0 nor any other time scale: a deficit's security is a function of block *counts* alone. Two chains with the same q offer identical security per confirmation — and therefore different security per minute.

- NU7 targets 25-second blocks; Bitcoin 600. Thirty minutes buys 3 Bitcoin confirmations or 72 Zcash ones in nominal expected time; under a 10% withholding attack either wait takes about 33.3 minutes, since public blocks arrive at rate p.
- Conditional on those counts, at $q = 10\%$: $r(0.1, 3) = 1.712\%$ against $r(0.1, 72) \approx 9.35 \times 10^{-34}$ under the model's fixed-hash-share assumption.
- The caveat: the model has no network. Faster blocks make honest blocks race one another and self-orphan, which effectively raises q; the comparison holds q fixed.

Consensus Guide, "The catch-up probability"; "Confirmation times at 25-second spacing"

What the numbers say: $r(q, n)$ in percent

q	n=1	n=3	n=6	n=10
2%	4.000	0.016	$5.4 \cdot 10^{-6}$	$1.6 \cdot 10^{-10}$
5%	10.000	0.232	0.0012	$1.2 \cdot 10^{-6}$
10%	20.000	1.712	0.059	0.0008
20%	40.000	11.584	2.331	0.316
30%	60.000	32.616	15.645	6.511
40%	80.000	63.488	49.300	37.218
45%	90.000	81.375	73.375	65.793
50%		100 for every n		

The columns $n = 3$ and $n = 10$ are the deployed wallet defaults: three confirmations for trusted outputs, ten for untrusted ones.

Three confirmations assign a 10% adversary 1.712%; ten assign the same adversary 0.0008%, and a 20% adversary 0.316%.

Every fixed count is a (q, ε) policy — an assumed adversary and an accepted risk — not a law of nature.

Bitcoin: "nothing is special about six" — it assumes an attacker below 10% and a tolerable risk below 0.1%; both figures are arbitrary.

Consensus Guide, "Waiting for confirmations"; "Confirmation requirements"; "Wallet confirmation policy"

Pick the adversary and the risk; read off n

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
5%	2	3	4
10%	2	4	6
20%	4	8	13
30%	9	20	32
40%	34	82	133
45%	135	331	539

The least n pushing $r(q, n)$ below each target: logarithmic in the inverse risk, explosive in q. A 10% attacker is answered by 6 confirmations, a 45% attacker by 539 — about 3.74 hours of 25-second blocks.

No finite count is final: $r(q, n) \geq 2q^n > 0$, since the attacker may simply find the first n blocks.

Divergence at the majority line: the required n tends to infinity as $q \uparrow 1/2$, because $r(1/2, n) = 1$ for every n.

Bitcoin: the same table, the same shape; only the wall-clock price of a row differs.

Consensus Guide, "Confirmation requirements"

The economics: when the attack pays

The attacker targets k merchants at v each; the goods are worth αv to him; he gives up after o vain blocks; each of his blocks would have earned the block income B. Expected profit over not attacking:

$$k\alpha v - (1-r)(kv + oB) \;=\; kv(\alpha + r - 1) - (1-r)\,oB,$$

non-positive whenever

$$v \;\le\; \frac{o\,(1-r)\,B}{k\,(\alpha + r - 1)} \;\overset{\alpha=1}{=}\; \frac{o}{k} \cdot \frac{B\,(1-r)}{r}.$$

- The cost unit B is the miner's block income after the applicable subsidy and fee allocation. Exposure scales linearly with B; measure the payment in units of this income.
- Bitcoin: Rosenfeld's $B = 25$ BTC with $o = 20$, $k = 5$, $\alpha = 1$ gives $v \le 100\,(1/r - 1)$ BTC; the same choices here give $\lfloor 4B(1-r)/r \rfloor$.

Consensus Guide, "The economics of the attack"; "Reward-normalised exposure"; protocol specification, "Calculating Block Subsidy, Funding Streams, Lockbox Disbursement, and Founders' Reward"; "ZIP 214 Funding Streams"; ZIP-1016

The largest payment the modelled attack cannot profit from

q	n=1	2	4	6	8	10
5%	36	271	10,327	344,743	10,931,506	336,737,801
10%	16	67	729	6,759	59,475	508,917
20%	6	15	55	167	467	1,262
30%	2	5	11	21	35	57
40%	1	1	2	4	5	6

Whole block-income units: $\lfloor 4(1-r)/r \rfloor$ — heuristic, as in Rosenfeld's Table 2: the unlimited-horizon $r(q, n)$ stands in for the $o = 20$ stopping rule. Against a 30% attacker, six confirmations admit 21 whole units of B under this heuristic. Patience buys safety exponentially; the required count grows only logarithmically in the value at stake.

Bitcoin: the same table with $B = 25$ BTC is Rosenfeld's Table 2.

Majority is a different problem

- At $q = p$ strict overtake is certain, but the walk has zero drift; that alone gives no lasting monopoly.
- Past $q = p$ the attacker double-spends at no cost beyond normal mining, rejects every other miner's blocks to take the whole issuance, and excludes transactions at will — an attempt to destroy the currency, motivated from outside it.
- Confirmation policy is not the defence there; the defence is the cost of assembling the majority, which lies outside this model.

Bitcoin: Rosenfeld's remark is about Bitcoin; nothing in it depends on the chain.

NU7 rails

Rail	Value	Standing
Reorg window	600 blocks (≈ 4.17 h)	ZIP-218 node-policy recommendation
Coinbase maturity	100 blocks (≈ 41.7 min)	consensus (transparent)
Difficulty	every block; window 102, damping 4, clamp 16%/32%	consensus
Proof of work	Equihash $(200, 9)$, SHA-256d filter	consensus
Wallet waits	3 trusted / 10 untrusted; anchor $\leq H - 3$	wallet policy

For transparent coinbase outputs, $n = 100$: $r(0.3, 100) \approx 3.7 \times 10^{-9}$, $r(0.4, 100) \approx 0.43\%$, yet $r(0.45, 100) \approx 15.7\%$ — porous near the majority line, as every fixed count must be. The window caps automatic rollback, not the theory: a fork rooted below it needs operator intervention.

Bitcoin: maturity and the memoryless race are inherited; per-block retargeting and the block spacing are not.

Consensus Guide, "Difficulty adjustment"; "Rollback limits, coinbase maturity, and finality"; "Wallet confirmation policy"; protocol specification, "Transaction Consensus Rules"; "Difficulty adjustment"; "Proof of Work"; draft ZIP-315

NU7: time and state

Draft ZIPs 218 and 259; Testnet activation height 4 465 026, Mainnet unassigned (6 October 2026).

$$\text{target spacing}: \quad 25\,\text{s}$$

$$\text{difficulty window}: \quad 102 \cdot 25 = 2550\,\text{s}$$

$$\text{expiry recommendation}: \quad 120 \cdot 25 = 3000\,\text{s}$$

No new transaction format: v5 and v6 remain; v4 is rejected. ZIP-237 adds delayed reissuance of funds removed from circulation to the halving-based subsidy.

ZIP-259; ZIP-218; ZIP-237

Where the deployed rule departs from the model

- **The tie-break.** The specification and Rosenfeld break equal-work ties first-seen, as Zebra does from 6.4.0, comparing tip hashes only where it cannot order arrivals. Earlier versions preferred the larger tip hash.
- **What that did to r.** Before 6.4.0, at equal work an attacker whose private tip hash won could reveal at once; the strict-overtake boundary $z = -1$ does not count that success. So $r(q, n)$ is exact for the strict-overtake model and a lower bound if only the tie-break is replaced by the hash rule — not a bound on the full deployed process, where difficulty moves and rollback is capped.
- **Equihash and memorylessness.** Each candidate solution faces a fresh SHA-256d lottery, so headers are the Bernoulli trials of the model; what Equihash changes is granularity — candidates arrive per solver run, and the exponential clock is good only while a run is short against the NU7 target spacing of 25 seconds.

Bitcoin: its nodes keep the first-seen rule; its hash trials are the model's ideal.

Consensus Guide, "Chain selection and the tie-break"; "Equihash and the memoryless model"

What consensus adds for shielded state: the nullifier set

The specification's rule, verbatim: "A nullifier MUST NOT repeat either within a transaction, or across transactions in a valid block chain." Each pool's nullifiers are "considered disjoint, even if they have the same bit pattern".

- The check is *stateful and ordered*: transactions apply in block order; each nf_{old} is tested against the set as it stands, then recorded, then each $\mathsf{cmx}_{\text{new}}$ is appended. The next spender proves membership against the new root; a second spend of the same note fails.
- It is the one check that cannot sit inside a proof, because it concerns the global history, not one transaction.
- Sets are per pool (ZIP-229): uniqueness of $\rho_{\text{new}} := \mathsf{nf}_{\text{old}}$ is pool-local, not global.

Bitcoin: the UTXO set is the same kind of ordered stateful check; here the set records serial numbers that name no output.

protocol specification, "Nullifier Sets"; *Ironwood Guide*, "State update"; "Nullifier sets"

Anchors, balances, and no mempool chains

- **Anchors.** The bundle's anchor "MUST refer to some earlier block's final" treestate for its pool — never a mid-block state, and any main-chain block since the pool's activation qualifies; the depth $H - 3$ is wallet policy.
- **No spend of an unmined note.** An Action cannot spend an output of its own transaction, and a note mined in this block is under no earlier block's final root. Bitcoin lets a mempool chain spend unconfirmed outputs; a shielded note cannot be spent until it is mined (the transparent layer keeps Bitcoin's rule).
- **The specified route.** ZIP-374 makes v6 anchors deferrable: sign a spend of a not-yet-mined note now, fill the anchor after the note is mined; signatures commit to neither anchor nor witness.
- **Balances.** Each pool's chain value balance is public and a block that would drive one negative is invalid (ZIP-209); every crossing writes a public row.

protocol specification, "Transactions and Treestates"; "Action Transfers and their Descriptions"; *Ironwood Guide*, "Anchors"; "Chain value pool balances"; *Wallet Guide*, "PCZT v2: deferred anchors and the Ironwood bundle"

Reorganisation of shielded state

- Trees, nullifier sets, and pool balances are chain state; a reorganisation rolls all three back together with the blocks.
- A reorganisation reaching the anchoring block strikes the anchor and the transaction with it. The rebuild spends the same notes, and nullifier derivation is deterministic, so it re-reveals the same nullifiers (the rebuild, truncate-and-rescan, and ZIP-374's re-anchoring: Part C).

Bitcoin: a transaction struck by a reorganisation is simply re-broadcast unchanged; here, once the fork reaches the anchoring block, the proof must be rebuilt against a surviving root.

Ironwood Guide, "Reorganisation"; *Wallet Guide*, "Reorganisation detection and recovery"; protocol specification, "Transactions and Treestates"

Checkpoint

A merchant sees Alice’s shielded payment at one confirmation. Which of the two double-spends is she exposed to, and what does the catch-up theorem say about waiting?

Checkpoint

A merchant sees Alice's shielded payment at one confirmation. Which of the two double-spends is she exposed to, and what does the catch-up theorem say about waiting?

- Not the same-chain one: a second spend of Alice's note reveals the same nullifier, and the set rejects it by rule, at any depth.
- The chain-level one: a secret branch may replace the block she saw. At $n = 1$ a 20% adversary succeeds in 40% of runs; each added confirmation drops the risk by a roughly constant factor, and $r(0.2, 10) = 0.316\%$.
- Waiting works in blocks, not minutes; it never reaches zero, and it stops working at $q = 1/2$.

Consensus Guide, "The catch-up probability"; "Confirmation requirements"; protocol specification, "Nullifier Sets"

Part I

Limits, frontier, takeaways

Limits: what the theorem leaves public

The end-to-end theorem hides the note contents and the spend-to-note link. It does not claim to hide:

- the public balance, the component and Action counts, any transparent leg, timing, address reuse, voluntary disclosures, wallet and network metadata;
- on Alice's bundle: two Actions, one anchor, two nullifiers, two commitments, and valueBalance = +10 000 zatoshi — the fee, in the clear.

The candidate set is the tree under the anchor, but publicly known dummy or spent leaves and other transaction metadata reduce the effective anonymity set; tree size alone does not measure it. An idiosyncratically old anchor fingerprints the spender.

Bitcoin: the graph itself is published. Here the graph is gone and the envelope stays: counts, fee, anchor, timing.

Ironwood Guide, "The end-to-end theorem"; "Non-claims"; "Anchor choice as public data"

Limits: the light client's server

The stated goal (ZIP-307): an honest-but-curious server must not learn which transactions are a wallet's. The keys never travel; the requests do — the full inventory is *Delivery and light clients*, "What the server learns". Two leaks survive the shipped mitigations:

- **Fetch-on-detect names a txid.** A scan is issued with exact range bounds and no per-block match; the full transaction fetched after a hit (the memo is not in the compact block) names exactly one transaction. The decoy fetches ZIP-307 asks for are a SHOULD the reference library does not supply.
- **The submission channel links the transaction to its origin.** A wallet that broadcasts through the same server hands it the whole transaction before the network sees it, tied to the session that scanned; circuit isolation exists in the client library, its deployment unknown.

Bitcoin: the scan leaks less than a BIP-157 client, which names each block it fetches after a filter hit; the fetch after a hit leaks more — one txid, not a block ("Leaks, in both directions").

Wallet Guide, "Payment-detection privacy"; "Leakage of transaction queries"; "The submission channel"; "Summary of server observables"

Limits: custody is a key

A note is $(\mathsf{addr}, v, \rho, \psi, \mathsf{rcm})$: it carries no script. Its spending condition is knowledge of ask, proved through rk (A6, A7) — whoever holds ask spends, and nothing below it on the ladder recovers it.

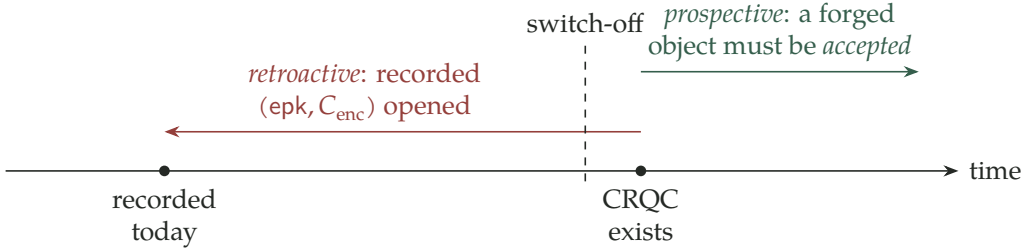
- Every key descends from one seed S: 32 to 252 bytes, at least 256 bits of entropy (ZIP-32). A weak seed is a weak link for all of them at once, and a brute-force search over weak seeds runs against many wallets simultaneously.
- What script gave Bitcoin moves into the wallet layer: threshold authority is FROST (t-of-n over ask; the verifier is untouched); hardware and offline signers use PCZT (a signer holds ask but cannot compute a Halo 2 proof).

Bitcoin: spending conditions live in script and every validator runs them — multisig, timelocks, hash locks. Here the condition is fixed by the circuit and discharged by one signature.

Ironwood Guide, "Notes"; "Key capabilities and scope"; *Wallet Guide*, "Wallet seeds"; "Partially created transactions (PCZT)"; *FROST Guide*, "Scope"

Quantum: two kinds of break

A CRQC — a quantum computer running Shor's algorithm at scale — takes discrete logarithms in polynomial time; every group assumption falls.



Retroactive: recorded data is exploitable the day the adversary exists. Prospective: a validator must *accept* a forgery, so switching off first removes acceptance. Orchard, both pools: one retroactive break.

Bitcoin: no ciphertext to open, but a silent-payment scan key has the same retroactive shape (next frame); an exposed public key is the prospective case.

ZIP-2005, "Motivation"; *Crypto Guide*, "Quantum adversaries: Shor's algorithm"

Retroactive: harvest now, decrypt later

Each Action publishes $(\mathsf{epk}, C_{\mathsf{enc}})$; one discrete-log link protects it, the Diffie–Hellman between esk and $\mathsf{pk_d}$ — the KDF and AEAD above it are symmetric.

$$\mathsf{ivk} = \log_{\mathsf{g_d}} \mathsf{pk_d} \quad \Longrightarrow \quad K = \mathrm{KDF}([\mathsf{ivk}]\,\mathsf{epk}, \ldots)$$

One logarithm for the lifetime of the ivk scope, then the recipient's own trial-decryption loop over the recorded chain: every $(d, v, \mathsf{rseed}, \mathsf{memo})$ ever sent to any diversified address sharing that ivk, past and future.

Bob's address went to Alice; whoever else holds it can one day read the 1.5 ZEC and the memo.

Ironwood Guide, "Confidentiality and key privacy of note encryption"

Retroactive: two limits, and the bin

- ivk views and never spends: no spend authority is conferred, and ask is not exposed — its exposure is the prospective case.
- The attack is keyed by the address: without $(d, \mathsf{pk_d})$ there is neither the base $\mathsf{g_d}$ nor the target $\mathsf{pk_d}$, and no known efficient generic route exists from the ciphertext alone.
- Bin: *open problem*. Nothing deployed or specified protects even future ciphertexts (ZIP-2005 excludes privacy by an explicit non-requirement); the recorded ones are unrepairable.

Bitcoin: no ciphertext to open, but silent payments carry the same shape: one logarithm of the published scan key replays the recipient's scan over recorded history and links everything that address ever received. No analogue for the amount and the memo — Bitcoin never hid them.

Ironwood Guide, "Key capabilities and scope"; ZIP-2005, "Non-requirements"

Prospective: the discrete-log integrity stack

Shor invalidates the reductions; each construction gets its own concrete consequence.

construction	with the generators' logarithms known
Sinsemilla binding (NoteCommit, $\mathsf{Commit}^{\mathsf{ivk}}$)	a second note under an on-chain cmx — balance breaks silently, turnstile-bounded; a second $(\mathsf{ak}, \mathsf{nk})$ for the same ivk — a spendability forgery
value commitment, binding signature	any cv reopens to any value; a solved bvk forges the signature — inflation, turnstile-bounded
spend authorisation (RedPallas)	one logarithm of ak or of any published rk forges spend authority; theft still needs a witness or a proof forgery
proof soundness (IPA on Vesta)	commitment equivocation; the knowledge-soundness reduction no longer holds

Recorded cv values stay perfectly hiding; α masks rk; transcripts are simulatable. Without the address, no generic route pulls these secrets from the recorded fields.

ZIP-2005, "Effects of discrete-logarithm-breaking attacks before the switch to the Recovery Protocol"

Why the integrity breaks are prospective

Each attack must place a forged object — a proof, a signature, a commitment opening — before a validator and have it *accepted*, at current height under current rules.

- Disabling the discrete-log pools before a CRQC exists removes acceptance of new transactions under those rules; settled history stays anchored by chain work and upgrade policy.
- Residue one: switch-off must precede the *first* forgery — an attack landed inside the window is not repaired afterwards.
- Residue two: honest funds inside a disabled pool need another way out. That is the problem ZIP-2005 exists to solve.

Bitcoin: the same shape. ECDSA and Schnorr fall to one logarithm of an exposed public key, and the forged spend must still be mined.

ZIP-2005, "Attacks addressed by the Recovery Protocol"; *Crypto Guide*, "Quantum adversaries: Shor's algorithm"

The symmetric backbone survives

Two primitive families carry no discrete-log structure.

- BLAKE2b: PRFexpand, hence every key and note-randomness derivation, the KDF, the outgoing cipher key, the RedPallas challenge, the ZIP-244 digests; the ZIP-32 Orchard tree is hardened-only and built entirely from it.
- Poseidon: the nullifier PRF, keyed by nk — a cryptanalytic heuristic, classically and quantumly alike.

Generic speedups only: Grover halves preimage security,

$$2^{256} \text{ classical evaluations } \longrightarrow \approx 2^{128} \text{ quantum ones,}$$

and the best implementable generic collision attack remains the classical parallel one. The load-bearing consequence: a seed holder can still prove, by re-executing the hardened derivations, that their keys descend from their seed. Ownership stays *provable* after the assumption that made it *enforceable* has fallen.

Bitcoin: SHA-256 and hardened BIP-32 derivation stand on the same ground.

Wallet Guide, "Key recoverability"; *Crypto Guide*, "Quantum adversaries: Shor's algorithm"

ZIP-2005: the hashed trapdoor

Recall the one note-level difference between the pools ("The Ironwood trapdoor"):

$$\mathsf{rcm} := \mathrm{ToScalar}(\mathsf{PRFexpand}_{\mathsf{rseed}}(\mathtt{0x0B} \,\|\, \mathsf{g_d}^{\star} \,\|\, \mathsf{pk_d}^{\star} \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{256}(\rho) \,\|\, \mathrm{LE}_{256}(\psi))),$$

a 137-byte preimage: every note field traverses BLAKE2b into the trapdoor.

- Under a CRQC, NoteCommit alone is not binding — Sinsemilla's collision resistance is the first thing to fall.
- Model PRFexpand as a random oracle: opening one commitment to two honestly derived notes is a collision-type event, with no group structure for Shor. Bound $O(q^3/N)$ in the QROM, about 2^{-74} at $q = 2^{60}$.

The hashed trapdoor: ownership after a quantum break

- The composite is post-quantum binding *provided a verifier checks the derivation*. No deployed clause does — that check is the Recovery Statement's job.
- Legacy `0x02` notes hash $\texttt{0x05} \| \rho$ only: unrecoverable, hence the seal ("Why the Orchard pool is sealed").

Bitcoin: ownership is a public key, exposed at spend; the hardened derivation behind it is symmetric too, but no rule reads it — the exposed key alone decides a spend.

Ironwood Guide, "The note seed: Ironwood derivations (lead byte `0x03`)"; "Purpose of the hashed trapdoor"; "Chain state and pool rules"

Recoverability, not resistance

The Recovery Statement, proved one day in a post-quantum proof system over a re-hashed tree: the key derivations from sk, $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}$, $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$, the ψ and rcm derivations, the commitment, a membership path, the nullifier, the α re-randomisation. Cost: 7 BLAKE2b-512 compressions, 1 BLAKE3 compression, 2 Sinsemilla commitments, 2 Pallas scalar multiplications, a Merkle path.

Binding only, privacy unchanged: the statement proves, with no group assumption, that one seed derives a committed note, its nullifier and its rk; it opens no ciphertext.

Ironwood Guide, "Purpose of the hashed trapdoor"; ZIP-2005, "Proposed Recovery Statement"; "Cost"

ZIP-2005: deployed, specified, designed, open

- *Specified and deployed*: the `0x03` derivation and the three seal rules — every Ironwood output is recoverable.
- *Specified, undeployed*: the quantum-key path (`qsk`, `qk`); no consensus rule.
- *Designed, unspecified*: the Recovery Protocol — proof system, collapsing tree hash, linking commitments all unfixed.
- *Open*: post-quantum proofs and signatures for ongoing spends; the switch-off schedule.

Sprout, Sapling and Orchard-pool notes are unrecoverable, so ZIP-2005 tells wallets to sweep everything into Ironwood as an ongoing process; deployed today, change is routed there. The one cost the strategy cannot answer: the harvested corpus accrues now.

Ironwood Guide, "Purpose of the hashed trapdoor"; ZIP-2005, "Proactive movement of funds to recoverable notes"; *Wallet Guide*, "Change"

Beyond the core: what each one adds

	what it adds	Bitcoin
FROST	*t*-of-*n* custody of ask; the aggregate is an ordinary spend-auth signature under rk	a Taproot FROST suite
FlyClient	logarithmic chain proofs: difficulty-weighted sampling over a Merkle mountain range	SPV fetches every header
ZSA	issued assets: per-asset notes and value commitments, issuance, burn	Elements' issued assets
Crosslink	proof of work kept, plus a trailing BFT finality layer cross-linked with it	confirmations, never finality

None of the four adds privacy: they add custody, chain verification, assets, and settlement to a core whose hiding is unchanged.

FROST Guide, "Scope"; *FlyClient Guide*, "Scope and status"; "Header-only verification"; *ZSA Guide*, "The OrchardZSA protocol"; *Crosslink Guide*, "Scope and status"

Beyond the core: the bins

	bin, as the volume states it
FROST	*specified*: RFC 9591 and ZIP-312 (Draft); shipped as a library; no consensus rule names it; key generation in no ZIP
FlyClient	commitment *deployed and consensus-critical* (ZIP-221, its root committed in every header since Heartwood); proofs *unbuilt*: no server serves one, no client asks; the Zcash instantiation designed, not specified
ZSA	core *specified* in Draft ZIPs 226 and 227; the carrier — v6 wire format, digests, lead byte, fees — *unspecified* since the ZIPs that held it were withdrawn; outside ZIP-259's NU7 scope
Crosslink	*design stage*: no ZIP, nothing activated on mainnet or testnet; a feature net runs prototype code; the construction rules *designed but unspecified*

Three bins throughout the volumes: *specified*, *designed but unspecified*, *open problem*; deployed behaviour cites the implementation and the specification instead.

FROST Guide, "Sources and their standing"; *FlyClient Guide*, "Scope and status"; "The specified commitment and its consumers"; *ZSA Guide*, "Scope and status"; "Epistemic classes"; *Crosslink Guide*, "Epistemic classes"; "Scope and status"

Where each one sits

block header	FlyClient — reads a root already committed
chain selection	Crosslink — a finality layer beside PoW
Action statement	ZSA — the only one inside the circuit
spend-auth signature	FROST — the same signature, more signers

Only ZSA changes the statement: an Asset Base witnessed once, A1 and A2 on a per-asset commitment, A4 by variable-base multiplication, the A3 dummy skip kept for the native asset alone, split notes. The other three leave the ten clauses as they are.

ZSA Guide, "The OrchardZSA Action statement"; *FROST Guide*, "Scope"; *FlyClient Guide*, "Scope and status"; *Crosslink Guide*, "Scope and status"

Takeaways: four checks, four mechanisms

check	Bitcoin	Zcash
coin exists	UTXO lookup	a commitment in an append-only tree; membership proven under an anchor, the position a private witness (A3)
authorised	script and signature	the circuit ties rk to ak (A6) and ak, nk to the note's address (A7); the node's signature check under rk proves $\mathsf{rsk} = \mathsf{ask} + \alpha$
not spent twice	the UTXO is deleted	the nullifier, keyed by nk and seeded by ρ: revealed once, rejected on repeat, unlinkable to cm (A5)
conserved	$\sum$ in $\geq \sum$ out in the clear	homomorphic value commitments (A4); the binding signature proves knowledge of $\sum$ rcv; fully shielded, valueBalance is the fee

A stranger still performs all four; the note is never seen.

Ironwood Guide, "Requirements on a shielded payment"; "Verification of a transaction"; "The Action statement"

Where to read more: the Zcash Arboretum

- **Foundations** — the *Math Guide* (fields, curves, hardness), the *Crypto Guide* (the toolbox's primitives), the *Halo 2 Guide* (the proof system, in full).
- **Deployed protocol** — the *Consensus Guide* (the consensus part), the *Ironwood Guide* (the spine of this workshop), the *Wallet Guide* (keys, fees, the pipeline, and what the server learns).
- **Frontier** — the *FlyClient*, *ZSA*, *Crosslink* and *FROST Guides*, each with its bins visible.

All non-normative: the protocol specification and the ZIPs are authoritative, and every volume cites the section it draws on.

Appendix

Backup

Backup: sources — the volumes, the specification, the ZIPs

- Volumes drawn on: the *Ironwood Guide* (the shielded critical path); the *Halo 2*, *Wallet*, *Crypto*, *Consensus* and *Math* Guides; one row each from the *FROST*, *FlyClient*, *ZSA* and *Crosslink* Guides. The volumes are non-normative.
- Normative: the protocol specification, and the ZIPs. Final or Active: 32 (keys), 203 (expiry), 209 (pool balances), 213 (shielded coinbase), 221 (history tree), 244 (digests), 316 (unified addresses), 317 (fees).
- Draft: 229 (v6 format, the two pools), 258 (the Ironwood upgrade), 307 (compact blocks), 312 (FROST), 315 (wallet practice), 318 (migration; a Reserved stub at the pin `69610984`, Draft at the recheck `dd1667ff`), 326 (wallets after the seal), 374 (PCZT). Proposed: ZIP-2005 (quantum recoverability). Reserved stub: ZIP-2006 (the cross-address restriction).

Backup: the standing of sources

- A deployed claim on a slide cites a specification section or a ZIP; a design-stage claim carries its bin: *specified*, *designed but unspecified*, *open problem*.
- Bitcoin has BIPs and no protocol specification; the reference client's behaviour is the definition. Here the specification is a document, and a node implements it.

Ironwood Guide, "Epistemic classes"; *Consensus Guide*, "Sources and their standing"

Backup: implementation snapshots, as the volumes cite them

Volume	Snapshot
Ironwood	orchard 0.15.0 (the librustzcash pin); orchard `bef8a27` (circuit, cross-address); librustzcash `e30517e4`; zips 69610984, rechecked at `dd1667ff`
Halo 2	halo2_proofs 0.3.2; halo2_gadgets 0.5.0; orchard 0.15.3
Wallet	librustzcash `e30517e4`, later snapshot `91f448b5`; zips `c6b7035` librustzcash `e30517e4`; lightwalletd `61fee32e`; Zaino `0e057e22`; zips 6961098
Consensus	zips 69610984; zebra `fa7657f1c`; librustzcash `e30517e43`; rechecked at zips `e753a6a3`

Ironwood Guide, "Scope and status"; *Halo 2 Guide*, "From statement to circuit: arithmetisation in practice"; *Wallet Guide*, "Addresses and viewing keys (ZIP 316)"; "The compact block and the compact transaction"; *Consensus Guide*, "Sources and their standing"

Backup: implementation snapshots, continued

Volume	Snapshot
FROST	frost `0966bd1`; reddsa `63c6ac0`; zips `e753a6a3`
FlyClient	`zcash_history` 0.5.0; zips `e753a6a3`
ZSA	zips `69610984`, rechecked at `e753a6a3`
Crosslink	tfl-book `fe6e1d6`; zips `e753a6a`
Crypto	no baseline pin; one design-stage cite, the orchard fork `cf801a5` (ZSA issuance)
Math	no implementation pin; instances cite the specification

FROST Guide, "Sources and their standing"; *FlyClient Guide*, "Sources and conventions"; *ZSA Guide*, "Scope and status"; *Crosslink Guide*, "Sources and conventions"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Backup: scope

- A wallet default describes the snapshot the volume cites, not every wallet; a node rail describes Zebra, named only for the reorg window and the tie-break. A Bitcoin Core release number does the same job: a claim about a default is a claim about one version.
- The three toys — the prime-order group for balance, the $\mathbb{F}_{97}$ circuit for the proof system, and the additive $\mathbb{Z}_{97}$ stand-in for the IPA fold (no discrete-log hardness) — isolate one mechanism each and estimate no soundness.
- A design-stage claim carries one of three bins, as the volumes assign them: *specified* (a published artefact pins it down), *designed but unspecified* (an informal source describes it), *open problem* (no artefact exists).
- Bitcoin's equivalent of the bins is a BIP's status field; the difference is that here a deployed rule may still carry a Draft ZIP, so the bin and the deployment are tracked separately.

Ironwood Guide, "Epistemic classes"; *Consensus Guide*, "Sources and their standing"

Backup: the Pasta field discipline — where each object lives

lives in	elements	role
$\mathbb{F}_{p_{\mathsf{Vesta}}}$ (Pallas *scalar* field)	$\mathsf{ask}, \mathsf{r_{ivk}}, \mathsf{rcm}, \mathsf{rcv}, \alpha, \mathsf{rsk}, \mathsf{esk}, \mathsf{bsk}$	multiply points
$\mathbb{F}_{p_{\mathsf{Pallas}}}$ (Pallas *base* field)	$\mathsf{nk}, \rho, \psi, \mathsf{ivk}, \mathsf{cmx}, \mathsf{nf}$	the circuit's native field
$\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ (Pallas points)	$\mathsf{ak}, \mathsf{rk}, \mathsf{g_d}, \mathsf{pk_d}, \mathsf{epk}, \mathsf{cm}, \mathsf{cv}$	group elements
bit strings	$\mathsf{sk}, \mathsf{rseed}, \mathsf{dk}, \mathsf{ovk}$ (256); d (88)	keys, seeds, the diversifier

- One rule sorts every symbol: what multiplies a point is a scalar modulo the group order p_{Vesta}; what the circuit hashes or compares is a base-field element; what BLAKE2b consumes is a byte string.
- Two base-field elements also act as scalars — ivk in $\mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ and the nullifier's $F_{\mathsf{nk}}(\rho) + \psi$ — valid only because $p_{\mathsf{Pallas}} < p_{\mathsf{Vesta}}$ (both 255 bits).
- Bitcoin's secp256k1 has the same two fields, coordinates modulo p and scalars modulo n, and ECDSA makes the same move once — $r := x(R) \bmod n$; nobody notices, because nothing recomputes it inside a proof.

Ironwood Guide, "The spending key and the spend-side secrets"; "Viewing keys"; "Notes"; "The nullifier"; "Randomised validating keys"; *Crypto Guide*, "The transparent layer's schemes: ECDSA and BIP-340 over secp256k1"

Backup: the Pasta field discipline — the two primes

$$p_{\mathsf{Pallas}} = 2^{254} + \texttt{0x224698fc094cf91b992d30ed00000001},$$
$$p_{\mathsf{Vesta}} = 2^{254} + \texttt{0x224698fc0994a8dd8c46eb2100000001}$$

- The cycle: $\#\mathcal{E}_{\mathsf{Pallas}}(\mathbb{F}_{p_{\mathsf{Pallas}}}) = p_{\mathsf{Vesta}}$ and $\#\mathcal{E}_{\mathsf{Vesta}}(\mathbb{F}_{p_{\mathsf{Vesta}}}) = p_{\mathsf{Pallas}}$, both prime, cofactor 1: each curve's scalar field is the other's base field.
- Both $p - 1$ carry exactly 2^{32}: power-of-two evaluation domains up to 2^{32} (the Action circuit uses 2^{11} rows). Both primes are 1 mod 3: an order-3 endomorphism speeds scalar multiplication.
- Deployed: the circuit's native field is $\mathbb{F}_{p_{\mathsf{Pallas}}}$ (Pallas points are native); its commitments live on Vesta, whose scalar field is that same $\mathbb{F}_{p_{\mathsf{Pallas}}}$. One half of the cycle — recursion is not deployed, and verification amortises by batching instead.
- Bitcoin's secp256k1: $p - 1$ has two-adicity 1 (its scalar field, 6): the largest power-of-two domain has 2 (or 64) points — no room for a 2^{11}-row circuit, and no need for one (the deck's own computation, not a volume figure).

Math Guide, "Pallas and Vesta assembled"; "Base fields, scalar fields, and the Pasta cycle"; *Halo 2 Guide*, "A cycle of curves for efficient native recursion"; *Ironwood Guide*, "The Action circuit and the Halo 2 proof"; protocol specification, "Pallas and Vesta"

Backup: Sinsemilla — the hash

$$Q(D) := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaQ}, D),$$

$$P[m] := \mathsf{GroupHash}^{\mathbb{G}}(\texttt{z.cash:SinsemillaS}, \mathrm{LE}_{32}(m)), \qquad 0 \le m < 2^{10}$$

$$\mathsf{Sinsemilla}_D(M) := \mathsf{Acc}_\ell, \qquad \mathsf{Acc}_0 := Q(D), \qquad \mathsf{Acc}_{i+1} := (\mathsf{Acc}_i \dot{+} P[m_{i+1}]) \dot{+} \mathsf{Acc}_i$$

- Chunk width $k = 10$, at most $c = 253$ chunks (2530 bits); the message is zero-padded on the right to a multiple of 10 bits. The 1024-point table is shared by every domain, $Q(D)$ is per domain: 1025 nothing-up-my-sleeve generators, all GroupHash outputs.
- Each chunk costs one table lookup and two incomplete additions ($\dot{+}$); a 510-bit message costs 51 rounds, not 510 bit operations. The table is a lookup argument in the circuit.
- Bitcoin's SHA-256 is tuned for a CPU's word operations; inside a circuit every one of its bits is a constraint. Sinsemilla is tuned for the circuit that must recompute it — and would be a poor CPU hash.

Ironwood Guide, "Sinsemilla: hash, commitment, and short forms"; "Cost inside a proof"; protocol specification, "Sinsemilla Hash Function"

Backup: Sinsemilla — the commitment and its short forms

$$\mathsf{SinsemillaCommit}_r(D, M) := \mathsf{Sinsemilla}_{D\|\texttt{-M}}(M) + [r]\,H_D,$$

$$\mathsf{ShortCommit}_r(D, M) := \mathsf{Extract}(\mathsf{SinsemillaCommit}_r(D, M)),$$

$$\mathsf{ShortHash}_D(M) := \mathsf{Extract}(\mathsf{Sinsemilla}_D(M)), \qquad H_D := \mathsf{GroupHash}^{\mathbb{G}}(D \,\|\, \texttt{-r},\ \varepsilon)$$

- Hiding needs no assumption: for uniform r the term $[r]\,H_D$ is uniform on the group. Binding rests on H_D sitting in its own sub-domain, unrelated to Q and the table as far as anyone knows — Pedersen's split of message base from blinding base.
- Instances: NoteCommit is this commitment in its own domain (a point; the chain stores cmx, its x); $\mathsf{Commit}^{\mathsf{ivk}}$ is ShortCommit over $\mathrm{LE}_{255}(\mathsf{Extract}(\mathsf{ak})) \,\|\, \mathrm{LE}_{255}(\mathsf{nk})$, 510 bits, 51 rounds; the tree's node hash is ShortHash over 520 bits, 52 rounds.
- Confidential Transactions' $[v]V + [r]R$ is the one-slot Pedersen ancestor of this shape — next frame.

Ironwood Guide, "Sinsemilla: hash, commitment, and short forms"; "Note commitments"; "The Merkle hash and the tree"; protocol specification, "Sinsemilla commitments"

Backup: Sinsemilla beside Confidential Transactions

$$\text{CT: } [v]\,V + [r]\,R; \quad \text{Pedersen hash: } \textstyle\sum_j [m_j]\,P_j;$$
$$\text{Sinsemilla: } [2^\ell]\,Q(D) + \textstyle\sum_i [2^{\ell-i}]\,P[m_i].$$

- The bridge: Confidential Transactions' $[v]V + [r]R$ is the one-slot Pedersen commitment; the Pedersen *hash* extends it to a message with one independent generator per chunk — a table that grows with the message length.
- The limit: Sinsemilla keeps the discrete-log collision reduction but swaps the roles: a chunk value *selects* a point from one shared 1024-entry table, and the chunk's position supplies a fixed power-of-two weight. One table serves every domain and every length up to 253 chunks.
- So a one-chunk Sinsemilla is $[2]\,Q(D) + P[m_1]$, a lookup, not $[m_1]\,V$: Pedersen's coefficient has become Sinsemilla's index.
- In Elements the opening of $[v]V + [r]R$ is range-proved by a Bulletproof; here NoteCommit's opening is recomputed inside the Action proof, which is what the lookup table is tuned for.

Crypto Guide, "Sinsemilla: an algebraic hash-based commitment"; *Ironwood Guide*, "Collision resistance of the Sinsemilla instances"; "Cost inside a proof"

Backup: Sinsemilla — why a collision is a discrete logarithm

$$\mathsf{Sinsemilla}_D(M) = [2^\ell]\,Q(D) + \sum_{i=1}^{\ell} [2^{\ell-i}]\,P[m_i], \qquad \chi_j(m) := \sum_{i=1}^{\ell} 2^{\ell-i}\,[m_i = j],$$

$$\mathsf{Sinsemilla}_D(M) = \mathsf{Sinsemilla}_D(M') \implies \sum_{j=0}^{2^{10}-1} [\chi_j(m) - \chi_j(m')]\,P[j] = O.$$

- Unfold the accumulator (no addition exception): every chunk lands on a table generator with a power-of-two weight; a repeated chunk value collects its weights into χ_j, taken modulo p_{Vesta}. Take $M \neq M'$ of equal length.
- The vector $\chi(m)$ is injective: the binary digits of χ_j mark where chunk value j occurs, and $\ell \leq 253$ keeps $2^{253} < p_{\mathsf{Vesta}}$, so nothing wraps. A collision is a non-trivial relation among independently sampled random-oracle generators, found only by discrete logarithms.
- A SHA-256 collision would falsify a heuristic; a Sinsemilla collision would compute a discrete logarithm on Pallas: the same kind of assumption that guards every Bitcoin key, on a different curve.

Ironwood Guide, "Collision resistance of the Sinsemilla instances"; *Crypto Guide*, "Sinsemilla: an algebraic hash-based commitment"

Backup: Sinsemilla — the collision argument's fine print

$$\mathsf{Sinsemilla}_D(M) = -\mathsf{Sinsemilla}_D(M') \implies [2^{\ell+1}]\,Q(D) + \sum_j [\chi_j(m) + \chi_j(m')]\,P[j] = O$$

- Taking x-coordinates identifies P with $-P$, so a short-form collision is the + case above or this − case, whose Q-coefficient $2^{\ell+1}$ is non-zero modulo p_{Vesta} — a non-trivial relation whether or not any chunk differs.
- Blinded forms: subtracting two openings adds $[r - r']\,H_D$ (or $[r + r']\,H_D$); the message coefficients or the Q-coefficient keep the relation non-trivial. Two distinct notes cannot share cm, nor cmx.
- An undefined incomplete addition is an equality between accumulator points — itself a relation among the same generators.
- Fixed length is load-bearing: zero-padding makes two messages of different bit-lengths collide with no relation at all, so every deployed use fixes its input length.
- A Bitcoin hash is used at every length; here each domain hashes one fixed length, and the theorem is stated for exactly that.

Ironwood Guide, "Collision resistance of the Sinsemilla instances"

Backup: the seal — three rules, in full

Three consensus rules seal the Orchard pool to spend-only, and all three bind every transaction *regardless of its version* (ZIP-258):

1. **No issuance.** A coinbase transaction carries an *empty* Orchard component.
2. **No circulation.** Every Orchard-pool Action has $\mathsf{enableCrossAddress} = 0$. Enforced by the Action-circuit verifying key, which consensus selects by branch, so a v5 transaction cannot bypass it; ZIP-229 states the version-independence explicitly.
3. **No entry.** Per transaction, $v^{\mathsf{Orchard}}_{\mathsf{balance}} \geq 0$: net value may leave the pool, never enter.

Backup: why the seal lives in consensus

- The second is the deepest, because it lives in the circuit: an Orchard-pool spend can pay only its own receiver. The motive is recoverability — lead-byte-`0x02` notes cannot be recovered after a quantum break, so consensus stops their circulation and funnels the value out, while owners keep spending.
- Bitcoin retires no output type: an early P2PK coin is spendable today on the terms it was created with. Zcash retires a pool by sealing its doors and metering its exit.

Ironwood Guide, "Chain state and pool rules"

Backup: the cross-address restriction, A10, in full

With $\delta := \mathsf{disableCrossAddress}$ and $x(\cdot), y(\cdot)$ the affine coordinates of a Pallas point, A10 is four equalities in $\mathbb{F}_{p_{\mathsf{Pallas}}}$:

$$\delta \cdot (x(\mathsf{g_{d_{old}}}) - x(\mathsf{g_{d_{new}}})) = 0, \qquad \delta \cdot (y(\mathsf{g_{d_{old}}}) - y(\mathsf{g_{d_{new}}})) = 0,$$
$$\delta \cdot (x(\mathsf{pk_{d_{old}}}) - x(\mathsf{pk_{d_{new}}})) = 0, \qquad \delta \cdot (y(\mathsf{pk_{d_{old}}}) - y(\mathsf{pk_{d_{new}}})) = 0.$$

- The input δ is a public input of unfixed position, beside anchor, the two coordinates of cv, nf, the two of rk, cmx, enableSpends, enableOutputs. With $\delta \neq 0$ the receivers are equal as full points, not merely in x. The shape is A3's $v_{\text{old}} \cdot (\mathsf{root} - \mathsf{anchor}) = 0$ on four more rows of the same gate.
- On the wire the flag has the opposite polarity: bit 2 of the flags byte is enableCrossAddress (bits 0b001 spends, 0b010 outputs, 0b100 cross-address); canonical bytes 0b011 Orchard pool, 0b111 Ironwood. Bit $2 = 0$ is what a v5 signer treating it as reserved-zero already produces: hardware signers keep signing Orchard-pool unshielding unchanged.
- A Bitcoin soft fork narrows which scripts are valid; here the narrowing is a new verifying key selected by branch, and the wire change is one formerly reserved bit.

Ironwood Guide, "Orchard Action statement"; "Flag values and expanded receiver"

Backup: spending under the restriction

$$(\mathsf{addr}_{\mathrm{old}},\, v_{\mathrm{old}}) \longrightarrow (\mathsf{addr}_{\mathrm{new}} = \mathsf{addr}_{\mathrm{old}},\, v_{\mathrm{new}} = 0), \qquad v^{\mathsf{Orchard}}_{\mathrm{balance}} = v_{\mathrm{old}} > 0.$$

- An Action spends one note and creates one; under the restriction the created note must pay the spent note's own receiver. An owner therefore pays anyone by *exiting*: the real value leaves through a positive $v^{\mathsf{Orchard}}_{\mathrm{balance}}$, and the output slot holds a *fabricated* zero-valued note to the spent note's own receiver — which is also what keeps the Orchard tree growing.
- Self-sends stay possible by construction: paying an Orchard-pool receiver requires a spend at that receiver, hence its spending key. Wallet rules close the gap by forbidding sends to *external* Orchard-pool receivers (ZIP-326).
- The nearest Bitcoin concept is a covenant restricting where an output may go, which Bitcoin script does not offer; here it is one gate in a fixed circuit.

Ironwood Guide, "Meaning of A10"

Backup: the fabricated output, and the restriction's standing

- The fabrication is thorough: the fabricated output's C_{enc} is random bytes. A real encryption would let any ivk holder — including a later quantum adversary who recovered ivk from a published address — decrypt it and link the Action's nullifier to the address (ZIP-326).
- Between construction and signing the pretence is dropped: inside a PCZT the fabricated output carries its recipient, value and rseed explicitly, so a signer can classify it as a tolerable dummy rather than an opaque payment.
- Standing: the restriction is deployed and enforced by consensus, yet its owning ZIP-2006 is a Reserved stub. *Specified*: the flag's encoding, its consensus rule, the verifying-key selection by branch (ZIP-229, ZIP-258). *Designed but unspecified*: the normative semantics of "equal receivers", the flag-to-instance mapping, the dummy-spend treatment.
- A Bitcoin signer sees every output in the clear; a shielded signer needs the format to tell it which output is a dummy.

Ironwood Guide, "Status"; ZIP-326, "Fabricated same-address outputs and randomized note ciphertexts"

Backup: the turnstile and migration

$$\mathsf{IronwoodPoolBalance} := -\sum_{\mathrm{tx}} v^{\mathrm{Ironwood}}_{\mathrm{balance}}(\mathrm{tx}) \;\geq\; 0, \qquad v^{\mathrm{Orchard}}_{\mathrm{balance}} \geq 0 \text{ per transaction.}$$

- The turnstile is the instrument that retired Sprout: a one-way, publicly metered gate, and its consensus content is exactly these two rules — the seal's third, and the Ironwood pool-balance row under ZIP-209's rule (zero before the upgrade, never negative, all pool balances summing to at most MAX_MONEY).
- A migrating transaction has a fixed shape: it spends Orchard-pool notes with $v^{\mathrm{Orchard}}_{\mathrm{balance}} > 0$ into its transparent funds — where fees and transparent outputs are paid too — and absorbs them with $v^{\mathrm{Ironwood}}_{\mathrm{balance}} < 0$. Both fields are cleartext: the amount crossing is public in every transaction.
- Consensus caps nothing migration-specific; ZIP-318 — a Reserved stub at the pin, a Draft at the later recheck — recommends migration outputs of at most 10 000 ZEC. The Ironwood tree has its twin's 2^{32} capacity; the history tree gains three Ironwood fields.
- Bitcoin keeps no per-type ledger row because every amount is already public; the pool balance is the shielded ledger's one public number per pool.

Ironwood Guide, "Chain state and pool rules"; "Chain value pool balances"

Backup: the quantum footnote — what Part I left out

- The retroactive break's edge: one external ivk does not open the separate internal-scope ivk, where change lands (Part D), nor any scope whose address the adversary never learns — the attack is keyed by the address, not by the ciphertext.
- The binding bound's price: the classical collision bound $O(q^2/N)$ on the rcm derivation lifts, by Zhandry's compressed-oracle technique, to $O(q^3/N)$ in the QROM — at $q = 2^{60}$ and $N \approx 2^{254}$, about 2^{-134} classically against 2^{-74}: the lift costs a factor q.
- The Recovery Statement's key half: BindKeys — ask, ak, nk from sk — plus the $\mathsf{r}_{\mathsf{ivk}}$ branch: $\mathsf{r}_{\mathsf{ivk}}$ from sk by PRFexpand, or, where no single sk serves (FROST-generated keys, hardware-held keys), from a *quantum key* qk, one BLAKE3 compression of a secret qsk, with signatures of knowledge over the sighash tying the claimed keys to the spend.
- Bitcoin's exposure is of Part I's table's kind — a forged signature under an exposed public key must be *accepted* — with two differences: a forged RedPallas signature alone spends nothing here, theft also needing the note's witness or a proof forgery; and counterfeit, not theft, is what the pool's exit caps.

Ironwood Guide, "Purpose of the hashed trapdoor"; *Wallet Guide*, "Separation of scopes under trial decryption"; ZIP-2005, "QROM lift of the security arguments"; "Proposed Recovery Statement"